INSTITUTO POLITÉCNICO NACIONAL

Unidad Profesional Interdisciplinaria de Ingeniería
y Ciencias Sociales y Administrativas (UPIICSA)

Ingeniería en Informática

RUTA LIBRE

Mapa de lugares y rutas accesibles

MANUAL TÉCNICO

Versión del documento: 1.0

Versión de la aplicación: 0.1.0 (prototipo MVP)

Elaboró: José Carlos Bernal Bautista

Ciudad de México, octubre de 2026

Contenido

Control del documento....8
Historial de cambios....8
Convenciones tipográficas....8
1. Introducción....10
1.1 Propósito del documento....10
1.2 Alcance....10
1.3 Audiencia....11
1.4 Documentos relacionados....11
1.5 Definiciones y acrónimos....11
2. Descripción general del sistema....13
2.1 Planteamiento del problema....13
2.2 Solución propuesta....13
2.3 Objetivos....13
2.4 Funciones principales....14
2.5 Zona piloto....15
2.6 Perfiles de usuario....15
2.7 Restricciones del sistema....15
3. Requisitos....17
3.1 Requisitos funcionales....17
3.2 Requisitos no funcionales....18
3.3 Requisitos de accesibilidad y su relación con WCAG 2.2....19
3.4 Actores....20
3.5 Casos de uso....20
4. Arquitectura del sistema....25
4.1 Visión general....25
4.2 Descripción de las capas....25
4.3 Patrón de gestión de estado....26

4.4 Inyección de dependencias....26
4.5 Flujo de arranque....27
4.6 Decisiones de arquitectura....27
5. Tecnologías y dependencias....29
5.1 Lenguaje y marco de trabajo....29
5.2 Paquetes de Dart....29
5.3 Servicios externos....30
5.4 Herramientas de desarrollo....30
6. Estructura del proyecto....31
6.1 Árbol de directorios....31
6.2 Descripción de archivos....32
6.3 Convenciones de código....32
7. Modelo de datos....34
7.1 Place (lugar)....34
7.2 Report (reporte de peligro)....35
7.3 RouteResult y RouteStep....37
7.4 AppSettings (ajustes)....38
7.5 Persistencia....38
8. Lógica de negocio (lib/core)....40
8.1 Cálculos geográficos (geo.dart)....40
8.2 Interpretación de OpenStreetMap (osm_parser.dart)....41
8.3 Petición y respuesta de rutas (route_api.dart)....43
8.4 Avisos de peligro (hazard_alerts.dart)....43
8.5 Comandos de voz (voice_commands.dart)....44
8.6 Descripciones habladas (describe.dart)....45
9. Servicios (lib/services)....46
9.1 LocationService....46
9.2 PlacesService....46
9.3 ReportRepository y LocalReportRepository....48

9.4 RoutingService....48
9.5 VoiceService....49
10. Estado y navegación....51
10.1 AppState....51
10.2 Mapa de navegación....52
10.3 Escala de texto....52
11. Sistema de diseño....53
11.1 Tokens de color....53
11.2 Tipografía....54
11.3 Verificación de contraste....55
11.4 Formas, espaciado y elevación....56
11.5 Componentes compartidos (widgets/ui.dart)....56
11.6 Mapa en modo oscuro....57
12. Pantallas....58
12.1 Bienvenida y perfil (ProfileScreen)....58
12.2 Mapa (MapScreen)....59
12.3 Ficha del lugar (PlaceScreen)....62
12.4 Ruta (RouteScreen)....63
12.5 Reportar peligro (NewReportScreen)....65
12.6 Peligro reportado (ReportScreen)....66
12.7 Cerca de ti (NearbyScreen)....67
12.8 Ajustes (SettingsScreen)....68
12.9 Leyenda....69
13. Accesibilidad implementada....70
13.1 Lector de pantalla (TalkBack, VoiceOver, NVDA)....70
13.2 Voz propia de la aplicación....70
13.3 Visión....71
13.4 Motricidad....71
13.5 Retroalimentación háptica....71

14. Integración con servicios externos....72
14.1 Teselas de OpenStreetMap....72
14.2 Overpass API....72
14.3 openrouteservice....72
15. Seguridad y privacidad....74
15.1 Datos personales que maneja la aplicación....74
15.2 Permisos....74
15.3 Manejo de secretos....75
15.4 Firma de la aplicación....75
16. Entorno de desarrollo....76
16.1 Requisitos del equipo....76
16.2 Instalación automatizada....76
16.3 Instalación manual paso a paso....77
16.4 Problemas conocidos de instalación en Windows....77
16.5 Configuración de Visual Studio Code....78
17. Compilación y despliegue....79
17.1 Comandos de compilación....79
17.2 Integración continua (GitHub Actions)....79
17.3 Instalación del APK en un teléfono....80
17.4 Versionado....80
18. Pruebas....81
18.1 Estrategia....81
18.2 Ejecución....81
18.3 Catálogo de pruebas automatizadas....81
18.4 Plan de pruebas manuales....83
18.5 Cobertura pendiente....84
19. Mantenimiento y extensión....85
19.1 Cambiar la zona piloto....85
19.2 Agregar un tipo de reporte....85

19.3 Agregar una categoría de lugar....85
19.4 Agregar un comando de voz....85
19.5 Cambiar colores o tipografía....86
19.6 Servidor de reportes compartidos....86
19.7 Actualizar dependencias....86
20. Solución de problemas técnicos....87
21. Limitaciones conocidas y trabajo futuro....88
21.1 Limitaciones....88
21.2 Hoja de ruta....88
Anexo A. Glosario....89
Anexo B. Archivos de configuración....91
B.1 pubspec.yaml....91
B.2 analysis_options.yaml....91
B.3 .github/workflows/ci.yml....92
B.4 .gitignore....93
B.5 setup/generar_apk.ps1....94
B.6 setup/revisar.ps1....94
Anexo C. Código fuente....96
lib/main.dart....97
lib/app.dart....98
lib/config.dart....99
lib/app_state.dart....99
lib/models/place.dart....104
lib/models/report.dart....105
lib/models/route_result.dart....107
lib/models/settings.dart....108
lib/core/geo.dart....110
lib/core/osm_parser.dart....112
lib/core/route_api.dart....114

lib/core/hazard_alerts.dart....116
lib/core/voice_commands.dart....116
lib/core/describe.dart....117
lib/core/text_utils.dart....118
lib/core/theme.dart....119
lib/services/location_service.dart....122
lib/services/places_service.dart....123
lib/services/report_repository.dart....125
lib/services/routing_service.dart....126
lib/services/voice_service.dart....127
lib/widgets/base_map.dart....129
lib/widgets/map_markers.dart....130
lib/widgets/ui.dart....134
lib/widgets/report_photo.dart....141
lib/features/onboarding/profile_screen.dart....142
lib/features/map/map_screen.dart....146
lib/features/nearby/nearby_screen.dart....160
lib/features/place/place_screen.dart....162
lib/features/route/route_screen.dart....167
lib/features/report/new_report_screen.dart....173
lib/features/report/report_screen.dart....179
lib/features/settings/settings_screen.dart....182
test/geo_test.dart....187
test/osm_parser_test.dart....188
test/report_test.dart....189
test/route_api_test.dart....191
test/describe_test.dart....193
test/widget_test.dart....194
tool/patch_platforms.dart....195

Anexo D. Referencias..... 198

Control del documento

Campo	Valor
Proyecto	Ruta Libre — mapa de lugares y rutas accesibles
Documento	Manual técnico
Versión del documento	1.0
Versión de la aplicación	0.1.0+1 (prototipo MVP)
Identificador del paquete	`com.drak028.ruta_libre`
Autor	José Carlos Bernal Bautista
Institución	IPN — UPIICSA, Ingeniería en Informática
Repositorio	Cuenta de GitHub DraK028 (repositorio `ruta_libre`)
Fecha	Octubre de 2026
Clasificación	Uso académico. Proyecto de portafolio con fines de impacto social

Historial de cambios

Versión	Fecha	Descripción
0.1	Octubre 2026	Primer prototipo funcional: mapa, ficha, reportes, rutas y voz con tema oscuro de alto contraste.
0.2	Octubre 2026	Entorno de pruebas en Windows y GitHub Actions; APK de depuración.
1.0	Octubre 2026	Rediseño de interfaz (tema claro y oscuro, tipografía Atkinson Hyperlegible, mapa a pantalla completa con hoja inferior). APK de pruebas en versión release. Este documento.

Convenciones tipográficas

En este manual se usan las siguientes convenciones para distinguir los distintos tipos de información:

Convención	Significado	Ejemplo
`texto monoespaciado`	Nombres de archivos, clases, funciones, comandos y valores literales.	`lib/app_state.dart`, `flutter test`
Negritas	Elementos de la interfaz (botones, pantallas, secciones) y términos clave.	Botón **Enviar reporte**
Cursivas	Términos en inglés o palabras que se definen en el glosario.	*widget*, *geocoding*

Convención	Significado	Ejemplo
Bloque gris	Fragmentos de código fuente, salidas de consola o archivos de configuración.	Ver capítulo 16
Recuadro «Nota»	Información complementaria.	—
Recuadro «Importante»	Advertencias que, si se ignoran, pueden producir fallos o pérdida de datos.	—

1. Introducción

1.1 Propósito del documento

Este manual técnico describe de manera completa la arquitectura, el diseño, la implementación, la configuración, las pruebas y el mantenimiento de **Ruta Libre**, una aplicación móvil multiplataforma que ayuda a personas con movilidad reducida y con discapacidad visual a encontrar lugares accesibles, calcular rutas que eviten obstáculos y compartir reportes de peligros en la vía pública.

El documento tiene tres objetivos. El primero es permitir que cualquier persona con conocimientos de desarrollo móvil pueda instalar el entorno, compilar la aplicación y ejecutarla sin depender del autor. El segundo es explicar las decisiones de diseño tomadas y sus razones, de modo que los cambios futuros respeten las restricciones de accesibilidad, privacidad y uso de servicios externos que dieron forma al sistema. El tercero es servir como referencia para evaluar el proyecto en un contexto académico, mostrando la trazabilidad entre los requisitos, el código y las pruebas.

1.2 Alcance

El manual cubre la versión 0.1.0 de la aplicación, correspondiente al prototipo del producto mínimo viable (MVP). Se describe:

- La arquitectura en capas y la responsabilidad de cada módulo del código fuente en Dart.
- Los modelos de datos, su ciclo de vida y su representación persistente en formato JSON.
- Los algoritmos geográficos (distancia de Haversine, rumbo, dirección relativa) y de negocio (caducidad de reportes, avisos de peligro, interpretación de comandos de voz).
- La integración con OpenStreetMap (teselas y Overpass API) y con openrouteservice para el cálculo de rutas.
- El sistema de diseño de la interfaz: paleta de colores en modo claro y oscuro, tipografía, componentes y criterios de contraste.
- La implementación de accesibilidad: etiquetas semánticas para lectores de pantalla, síntesis y reconocimiento de voz, escalado de texto y señales que no dependen del color.
- El procedimiento de instalación del entorno de desarrollo en Windows, la compilación para Android y web, y la integración continua con GitHub Actions.
- La estrategia de pruebas automatizadas y el plan de pruebas manuales.
- Las tareas de mantenimiento más comunes y la ruta de evolución hacia un servidor compartido de reportes.

Quedan fuera del alcance el diseño del servidor de reportes compartidos (que se plantea como trabajo futuro), la publicación en tiendas de aplicaciones y la compilación nativa para iOS, que requiere un equipo con macOS.

1.3 Audiencia

Perfil	Uso principal del manual	Capítulos recomendados
Desarrollador que se integra al proyecto	Instalar el entorno, entender la estructura y hacer cambios con seguridad.	4, 5, 6, 16, 17, 18, 19
Evaluador académico	Verificar la trazabilidad entre requisitos, diseño, código y pruebas.	2, 3, 4, 7, 8, 18
Especialista en accesibilidad	Revisar las decisiones de diseño inclusivo y su implementación.	3.3, 11, 13
Responsable de operación	Desplegar, monitorear el uso de servicios externos y atender incidencias.	14, 15, 17, 20

1.4 Documentos relacionados

- **Manual de usuario de Ruta Libre** — describe el uso de la aplicación desde el punto de vista de la persona usuaria.
- **README.md** del repositorio — resumen de funciones, estructura y comandos básicos.
- **Maqueta de diseño** «Ruta Libre — Rediseño de interfaz» — seis pantallas de referencia en modo claro y oscuro.
- Política de uso de teselas de la OpenStreetMap Foundation (operations.osmfoundation.org/policies/tiles).
- Documentación de la API de openrouteservice v2 (openrouteservice.org/dev).
- Pautas de Accesibilidad para el Contenido Web (WCAG) 2.2 del W3C.

1.5 Definiciones y acrónimos

Término	Definición
API	Interfaz de programación de aplicaciones: contrato para que dos programas se comuniquen.
APK	Paquete de instalación de aplicaciones Android.
CI	Integración continua: compilación y pruebas automáticas en cada cambio del código.
GeoJSON	Formato JSON estándar para representar geometrías geográficas (RFC 7946).
GPS	Sistema de posicionamiento global; en este documento, cualquier fuente de ubicación del teléfono.

Término	Definición
MVP	Producto mínimo viable: versión con las funciones mínimas para validar la idea con usuarios reales.
ORS	openrouteservice, servicio de cálculo de rutas basado en datos de OpenStreetMap.
OSM	OpenStreetMap, mapa colaborativo y libre del mundo.
Overpass	API de solo lectura para consultar datos de OSM con un lenguaje de consultas propio (Overpass QL).
STT	Speech to text: reconocimiento de voz (voz a texto).
Tesela	Imagen cuadrada de 256 × 256 píxeles que forma parte del mapa de fondo.
TTS	Text to speech: síntesis de voz (texto a voz).
TTL	Time to live: tiempo de vida de un reporte antes de caducar si nadie lo confirma.
WCAG	Web Content Accessibility Guidelines, pautas internacionales de accesibilidad.
Widget	En Flutter, cualquier elemento de la interfaz: un botón, un texto, una pantalla completa.

2. Descripción general del sistema

2.1 Planteamiento del problema

En la Ciudad de México, como en la mayoría de las ciudades del país, una persona que usa silla de ruedas, bastón, andadera o que tiene baja visión no puede saber con anticipación si un lugar tiene una entrada sin escalón, si la banqueta del camino está en buen estado o si una rampa está bloqueada por un automóvil. Las aplicaciones de mapas generales optimizan el tiempo de traslado de una persona sin discapacidad: proponen escaleras, cruces sin rampa y banquetas estrechas sin advertirlo.

La información existe de manera parcial y dispersa. OpenStreetMap permite registrar la accesibilidad de un lugar con la etiqueta `wheelchair`, el estado de un borde de banqueta con `kerb` y la presencia de guía podotáctil con `tactile_paving`; sin embargo, esa información no está presentada de forma útil para quien la necesita. Por otro lado, los obstáculos temporales —una obra, una coladera abierta, un coche sobre la rampa— cambian cada día y ningún mapa estático los refleja.

2.2 Solución propuesta

Ruta Libre combina tres fuentes de información en una sola aplicación pensada desde el principio para personas con discapacidad motriz o visual:

1. **Datos abiertos de OpenStreetMap** sobre la accesibilidad de comercios, servicios, paradas de transporte y cruces peatonales, presentados con color e ícono para que se distingan sin depender solo del color.
2. **Rutas calculadas por openrouteservice** con el perfil de silla de ruedas (sin escalones, pendiente máxima de 6 %) o a pie, evitando las zonas donde la comunidad reportó un peligro.
3. **Reportes de la comunidad** sobre peligros temporales, que caducan solos si nadie confirma que siguen ahí, para que el mapa no se llene de avisos obsoletos.

Toda la información del mapa está disponible también en forma de lista ordenada por distancia y puede escucharse en voz alta, lo que permite usar la aplicación completa con un lector de pantalla o sin mirar el teléfono.

2.3 Objetivos

Objetivo general

Desarrollar una aplicación móvil accesible que permita a personas con movilidad reducida o discapacidad visual identificar lugares accesibles, planear rutas seguras y compartir información sobre obstáculos en su entorno inmediato, usando datos abiertos y servicios gratuitos.

Objetivos específicos

1. Mostrar en un mapa los lugares de una zona con su nivel de accesibilidad para silla de ruedas, obtenidos de OpenStreetMap.
2. Calcular rutas a pie o en silla de ruedas que eviten escaleras y peligros reportados.
3. Permitir que cualquier persona reporte un peligro en menos de tres toques, y que la comunidad lo confirme o lo descarte.
4. Hacer que todas las funciones puedan usarse con lector de pantalla y por voz.
5. Adaptar el comportamiento de la aplicación al perfil de movilidad de cada persona.
6. Proteger la privacidad de la ubicación de las personas usuarias.

2.4 Funciones principales

Función	Descripción	Pantalla
Mapa accesible	Lugares con color e ícono según accesibilidad (✓, −, ✕, ?). Cruces como cuadrados. Filtros rápidos.	Mapa
Cerca de ti	Lista de lugares y peligros ordenada por distancia, con dirección relativa y lectura en voz alta.	Mapa (hoja) y Cerca de ti
Búsqueda	Búsqueda por nombre o categoría sin importar acentos; también por voz.	Mapa
Ficha del lugar	Estado de accesibilidad, baño accesible, guía podotáctil, notas, «¿Es correcta?».	Lugar
Ruta accesible	Ruta con perfil de silla de ruedas o a pie que evita peligros; indicaciones paso a paso habladas.	Ruta
Reportar peligro	Ocho tipos de peligro con gravedad, foto y nota opcionales.	Reportar peligro
Confirmar o descartar	«Sigue ahí» renueva la vigencia; dos «Ya no está» cierran el reporte.	Peligro reportado
Aviso de peligro	Voz y vibración al acercarse a 30 m de un peligro vigente.	Cualquiera
Comandos de voz	«Qué hay cerca», «reportar», «dónde estoy», «quitar ruta», «ajustes», o búsqueda libre.	Mapa
Perfil de movilidad	Silla de ruedas, bastón o andadera, baja visión o ceguera.	Bienvenida y Ajustes
Apariencia	Tema claro, oscuro o del sistema; tamaño de letra hasta 200 %.	Ajustes

2.5 Zona piloto

El prototipo se configuró para un piloto de un kilómetro de radio alrededor de UPIICSA, en la colonia Granjas México de la alcaldía Iztacalco. El centro de la zona está definido en `AppConfig.pilotCenter` con las coordenadas aproximadas 19.3958° N, 99.0912° O. La zona incluye una avenida principal (Av. Té), comercios de barrio, paradas de transporte público y el propio campus, lo que permite probar los distintos tipos de lugares y peligros.

> **Importante:** Las coordenadas de `pilotCenter` son aproximadas. Antes de hacer pruebas de campo deben verificarse en openstreetmap.org y corregirse en `lib/config.dart` si hace falta.

Fuera de la zona piloto la aplicación sigue funcionando: al desplazar el mapa más allá del 80 % del radio de cualquier zona ya descargada aparece el botón **Buscar lugares aquí**, que descarga los lugares de la nueva zona.

2.6 Perfiles de usuario

La aplicación define cuatro perfiles de movilidad (`MobilityProfile`). El perfil se elige la primera vez que se abre la aplicación y cambia el comportamiento de varias funciones:

Perfil	Perfil ORS	Evita escalones	Voz por defecto	Solo accesibles por defecto
Silla de ruedas	`wheelchair`	Sí	No	Sí
Bastón o andadera	`foot-walking`	Sí	No	No
Baja visión	`foot-walking`	No	Sí	No
Ceguera	`foot-walking`	No	Sí	No

Además, el perfil de silla de ruedas agrega a la petición de ruta restricciones de pendiente máxima (6 %), altura máxima de borde de banqueta (6 cm) y calidad mínima de superficie («good»).

2.7 Restricciones del sistema

- **Sin servidor propio.** El prototipo no tiene un servidor; los reportes se guardan solo en el teléfono de quien los crea. Esto se diseñó así para validar la interfaz antes de invertir en infraestructura, y se resolvió con una interfaz (`ReportRepository`) que permite cambiar el almacenamiento sin tocar las pantallas.
- **Servicios gratuitos con límites.** La instancia pública de Overpass no debe usarse como servidor de una aplicación, por lo que los lugares se guardan en caché siete días. openrouteservice tiene un límite de 2,000 rutas al día y 40 por minuto en su plan gratuito.

- **Sin brújula.** La dirección relativa («a tu derecha») se calcula con el rumbo del GPS, que solo es confiable cuando la persona camina a más de 0.5 m/s. Si está quieta, se usan puntos cardinales.
- **Plataformas.** Android 7.0 (API 24) o superior y navegadores web modernos. La compilación para iOS es posible con el mismo código, pero requiere macOS y Xcode.
- **Conexión.** El mapa de fondo y las rutas requieren conexión a Internet. Los lugares ya descargados y los reportes se pueden consultar sin conexión.

3. Requisitos

Los requisitos se identifican con un código para poder rastrearlos en el diseño, el código y las pruebas. La columna «Implementación» indica el archivo principal donde se cumple el requisito y la columna «Verificación» indica cómo se comprueba.

3.1 Requisitos funcionales

ID	Requisito	Implementación	Verificación
RF-01	El sistema debe solicitar el perfil de movilidad la primera vez que se abre y guardarlo.	`profile_screen.dart`, `app_state.dart`	Prueba manual PM-01
RF-02	El sistema debe mostrar un mapa con los lugares de la zona coloreados según su accesibilidad, con un ícono distinto por estado.	`map_screen.dart`, `map_markers.dart`	`widget_test.dart`, PM-03
RF-03	El sistema debe descargar los lugares de OpenStreetMap y guardarlos para reutilizarlos durante 7 días.	`places_service.dart`, `osm_parser.dart`	`osm_parser_test.dart`, PM-04
RF-04	El sistema debe permitir filtrar el mapa para mostrar solo lugares accesibles o de acceso limitado.	`app_state.dart` (`visiblePlaces`)	PM-05
RF-05	El sistema debe permitir filtrar por farmacias, baños y transporte.	`map_screen.dart` (`PlaceFilter`)	PM-05
RF-06	El sistema debe permitir buscar lugares por nombre o categoría sin distinguir acentos ni mayúsculas.	`text_utils.dart`, `map_screen.dart`	`describe_test.dart`, PM-06
RF-07	El sistema debe mostrar una lista de lugares y peligros ordenada por distancia, con distancia y dirección.	`map_screen.dart`, `nearby_screen.dart`, `ui.dart`	PM-07
RF-08	El sistema debe leer en voz alta la lista de lo más cercano.	`map_screen.dart` (`_speakList`)	PM-07
RF-09	El sistema debe mostrar la ficha de un lugar con estado, baño accesible, guía podotáctil y notas.	`place_screen.dart`	PM-08
RF-10	El sistema debe calcular una ruta según el perfil, evitando escalones si corresponde y los peligros de gravedad media o alta.	`route_api.dart`, `routing_service.dart`	`route_api_test.dart`, PM-09
RF-11	El sistema debe mostrar la ruta en el mapa con tiempo, distancia e indicaciones paso a paso, y leerlas en voz alta.	`route_screen.dart`	PM-09

ID	Requisito	Implementación	Verificación
RF-12	El sistema debe permitir reportar un peligro con tipo, gravedad, ubicación GPS y, opcionalmente, foto y nota.	`new_report_screen.dart`	PM-10
RF-13	Cada tipo de reporte debe caducar después de un tiempo propio si nadie lo confirma.	`report.dart`	`report_test.dart`
RF-14	El sistema debe permitir confirmar («Sigue ahí») o descartar («Ya no está») un reporte; dos descartes seguidos lo cierran.	`report.dart`, `report_screen.dart`	`report_test.dart`, PM-11
RF-15	El sistema debe avisar con voz y vibración al acercarse a 30 m de un peligro vigente de gravedad media o alta, sin repetir el aviso en 10 minutos.	`hazard_alerts.dart`, `app_state.dart`	`report_test.dart`, PM-12
RF-16	El sistema debe reconocer comandos de voz para las acciones principales.	`voice_commands.dart`, `voice_service.dart`	`describe_test.dart`, PM-13
RF-17	El sistema debe permitir indicar si la información de un lugar es correcta.	`place_screen.dart`, `app_state.dart`	PM-08
RF-18	El sistema debe permitir elegir tema claro, oscuro o del sistema y el tamaño del texto.	`settings_screen.dart`, `app.dart`	PM-14
RF-19	El sistema debe permitir configurar la clave de openrouteservice desde la aplicación.	`settings_screen.dart`	PM-15
RF-20	El sistema debe permitir descargar los lugares de una zona fuera de la zona piloto.	`map_screen.dart` (`_loadHere`)	PM-16

3.2 Requisitos no funcionales

ID	Categoría	Requisito	Cómo se cumple
RNF-01	Accesibilidad	Todo elemento interactivo debe tener una etiqueta para lector de pantalla.	Widgets `Semantics` con `label`, `button` y `selected` en todos los controles propios.
RNF-02	Accesibilidad	El contraste de texto debe ser de al menos 4.5:1 (WCAG AA).	Paleta verificada: mínimo 4.92:1 (sección 11.3).
RNF-03	Accesibilidad	La información no debe depender solo del color.	Cada estado tiene ícono propio; los cruces tienen forma distinta.
RNF-04	Accesibilidad	Los controles deben medir al menos 44 × 44 puntos.	Tamaños mínimos de 44–56 puntos en el tema y componentes.
RNF-05	Accesibilidad	El texto debe poder crecer hasta el 200 % sin perder funciones.	Escala configurable de 1.0 a 2.0, nunca menor que la del sistema.

ID	Categoría	Requisito	Cómo se cumple
RNF-06	Rendimiento	Arrastrar el mapa no debe reconstruir los marcadores en cada cuadro.	`onPositionChanged` solo llama a `setState` si cambia algo visible.
RNF-07	Disponibilidad	Sin conexión, la app debe mostrar los últimos datos descargados.	Caché por zona; si Overpass falla se usan datos vencidos.
RNF-08	Uso responsable	Respetar las políticas de uso de OSM y Overpass.	User-Agent propio, atribución visible, caché de 7 días, sin descarga masiva de teselas.
RNF-09	Privacidad	La ubicación de la persona no debe salir del teléfono.	Solo se envían a ORS el origen y destino de una ruta pedida; nunca el recorrido.
RNF-10	Seguridad	La clave de ORS no debe quedar en el código fuente.	Se pasa con `--dart-define` o se escribe en Ajustes y se guarda localmente.
RNF-11	Mantenibilidad	La lógica de negocio debe ser probada sin dispositivo.	Lógica pura en `lib/core` sin dependencias de Flutter; 36 pruebas.
RNF-12	Portabilidad	Un solo código para Android, iOS y web.	Flutter; ajustes por plataforma solo en voz y fotos.
RNF-13	Robustez	La app no debe quedarse detenida si el motor de voz no responde.	Tiempos límite en `speak()` y `stop()`.

3.3 Requisitos de accesibilidad y su relación con WCAG 2.2

Aunque WCAG se escribió para contenido web, sus criterios son la referencia más aceptada para aplicaciones móviles. La tabla siguiente relaciona los criterios relevantes con la implementación en Ruta Libre.

Criterio WCAG	Nivel	Aplicación en Ruta Libre
1.1.1 Contenido no textual	A	Marcadores, íconos y fotos tienen etiqueta semántica o texto alternativo («Foto del peligro»).
1.3.1 Información y relaciones	A	Encabezados marcados con `Semantics(header: true)`; grupos de opciones con `inMutuallyExclusiveGroup`.
1.4.1 Uso del color	A	Estados con ícono (✓ − ✕ ?), cruces cuadrados, peligros triangulares.
1.4.3 Contraste mínimo	AA	Todas las combinaciones de texto superan 4.5:1.
1.4.4 Cambio de tamaño del texto	AA	Escala hasta 200 % desde Ajustes; respeta la escala del sistema.
1.4.11 Contraste de elementos no textuales	AA	Marcadores con aro blanco y sombra; bordes de 1–2 px en controles.

Criterio WCAG	Nivel	Aplicación en Ruta Libre
2.5.8 Tamaño del objetivo (mínimo)	AA	Objetivos de 44 × 44 puntos como mínimo.
3.3.1 Identificación de errores	A	Errores de ubicación, red y rutas se muestran en texto y se dicen en voz alta.
4.1.2 Nombre, función, valor	A	Botones propios con `button: true`, estado `selected` y etiqueta.
4.1.3 Mensajes de estado	AA	`liveRegion` en «Descargando lugares…», «Calculando ruta» y confirmaciones.

3.4 Actores

Actor	Tipo	Descripción
Persona usuaria	Primario	Persona con movilidad reducida o discapacidad visual, o acompañante, que consulta el mapa, pide rutas y reporta peligros.
Comunidad	Primario	Otras personas usuarias que confirman o descartan reportes (en el MVP, en el mismo teléfono).
OpenStreetMap / Overpass	Sistema externo	Provee teselas y datos de lugares.
openrouteservice	Sistema externo	Calcula rutas.
Sistema operativo	Sistema externo	Provee GPS, voz, reconocimiento de voz, cámara, almacenamiento y lector de pantalla.

3.5 Casos de uso

Los casos de uso describen las interacciones principales desde el punto de vista de la persona usuaria. Cada uno se relaciona con uno o más requisitos funcionales.

CU-01. Elegir perfil de movilidad (RF-01)

Elemento	Descripción
Actor principal	Persona usuaria
Precondiciones	La app se abre por primera vez o la persona entra a Ajustes → Perfil.
Flujo principal	1. La app muestra la bienvenida y lee en voz alta las opciones. 2. La persona toca una de las cuatro tarjetas. 3. La app marca la tarjeta, vibra y dice el nombre del perfil. 4. La persona toca Continuar. 5. La app guarda el perfil, ajusta voz y filtro según el perfil y muestra el mapa.

Elemento	Descripción
Flujos alternos	3a. La persona toca otra tarjeta: la selección cambia. 4a. Sin selección, el botón dice «Elige una opción» y está deshabilitado. 5a. Desde Ajustes: se guarda solo el perfil y se regresa a Ajustes.
Postcondiciones	El perfil queda guardado y afecta rutas, filtros y voz.

CU-02. Consultar lugares cercanos (RF-02, RF-07, RF-08)

Elemento	Descripción
Actor principal	Persona usuaria
Precondiciones	Perfil elegido; permiso de ubicación concedido o denegado.
Flujo principal	1. La app obtiene la ubicación y carga los lugares de la zona. 2. El mapa muestra los lugares con color e ícono, y la hoja «Cerca de ti» los lista por distancia. 3. La persona toca Escuchar. 4. La app lee los cinco lugares más cercanos con estado, distancia y dirección.
Flujos alternos	1a. Sin permiso: se avisa y se mide desde el centro de la zona piloto. 1b. Sin conexión y sin caché: se avisa «No hay conexión…». 3a. Pestaña Peligros: se leen los tres peligros más cercanos.
Postcondiciones	La persona conoce lo que hay a su alrededor.

CU-03. Buscar y filtrar lugares (RF-04, RF-05, RF-06)

Elemento	Descripción
Actor principal	Persona usuaria
Precondiciones	Lugares cargados.
Flujo principal	1. La persona escribe en la barra de búsqueda o toca un filtro. 2. La app sube la hoja y muestra «Resultados» con los lugares que coinciden, sin importar acentos. 3. El mapa muestra solo esos lugares.
Flujos alternos	1a. La persona toca el micrófono y dice el nombre: la frase se usa como búsqueda y se lee el resultado más cercano. 2a. Sin coincidencias: «No encontré lugares con ese filtro.»
Postcondiciones	Se ven solo los lugares de interés.

CU-04. Ver la ficha de un lugar (RF-09, RF-17)

Elemento	Descripción
Actor principal	Persona usuaria

Elemento	Descripción
Precondiciones	Lugares visibles en el mapa o la lista.
Flujo principal	1. La persona toca un lugar. 2. La app muestra la ficha y la lee en voz alta. 3. La persona responde «¿Es correcta esta información?». 4. La app guarda la respuesta y agradece.
Flujos alternos	3a. La persona toca «Ver o corregir en OpenStreetMap»: se abre el navegador.
Postcondiciones	La respuesta queda guardada para enviarse a OSM.

CU-05. Calcular una ruta accesible (RF-10, RF-11)

Elemento	Descripción
Actor principal	Persona usuaria
Precondiciones	Ubicación disponible; clave de openrouteservice configurada; conexión.
Flujo principal	1. En la ficha, la persona toca Cómo llegar. 2. La app anuncia «Calculando una ruta accesible» y envía la petición con el perfil y los peligros a evitar. 3. La app muestra la ruta, el tiempo, la distancia, las etiquetas y los pasos, y lee el resumen. 4. La persona toca Iniciar guía por voz. 5. La app lee todos los pasos.
Flujos alternos	2a. Sin ubicación: «Aún no sé dónde estás.» 2b. Sin clave: «Falta la clave…». 3a. Error del servicio: mensaje e «Intentar otra vez». 5a. La persona toca un paso: se lee solo ese paso.
Postcondiciones	La ruta queda activa en el mapa principal con su tarjeta.

CU-06. Reportar un peligro (RF-12, RF-13)

Elemento	Descripción
Actor principal	Persona usuaria
Precondiciones	Ubicación disponible.
Flujo principal	1. La persona toca Reportar peligro (o lo pide por voz). 2. La app muestra la ubicación y los ocho tipos. 3. La persona elige el tipo; la app propone la gravedad. 4. Opcionalmente cambia la gravedad, toma una foto o escribe una nota. 5. La persona toca Enviar reporte. 6. La app guarda el reporte, vibra, agradece y regresa al mapa con el peligro visible.

Elemento	Descripción
Flujos alternos	2a. Sin ubicación: el botón Enviar está deshabilitado y se muestra «Buscando tu ubicación…». 4a. La persona cancela la cámara: no se agrega foto.
Postcondiciones	El reporte es visible y caduca según su tipo.

CU-07. Confirmar o descartar un peligro (RF-14)

Elemento	Descripción
Actor principal	Comunidad
Precondiciones	Existe un reporte activo.
Flujo principal	1. La persona toca el triángulo en el mapa o la fila en Peligros. 2. La app muestra el detalle y pregunta «¿Sigue ahí?». 3. La persona toca Sigue ahí. 4. La app renueva la vigencia y agradece.
Flujos alternos	3a. La persona toca Ya no está: se suma un voto; con dos votos seguidos el reporte se cierra.
Postcondiciones	El mapa refleja el estado real del obstáculo.

CU-08. Recibir un aviso de peligro (RF-15)

Elemento	Descripción
Actor principal	Persona usuaria
Precondiciones	Avisar peligros cercanos activado; existe un peligro activo de gravedad media o alta.
Flujo principal	1. La persona se desplaza. 2. Al quedar a 30 m o menos del peligro, el teléfono vibra fuerte y dice «Atención. …» con tipo, distancia y dirección.
Flujos alternos	2a. Ya se avisó de ese peligro en los últimos 10 minutos: no se repite. 2b. Varios peligros cercanos: se dice solo el más cercano.
Postcondiciones	La persona conoce el peligro antes de llegar a él.

CU-09. Usar comandos de voz (RF-16)

Elemento	Descripción
Actor principal	Persona usuaria
Precondiciones	Permiso de micrófono; servicio de reconocimiento disponible.

Elemento	Descripción
Flujo principal	1. La persona toca el micrófono de la barra de búsqueda. 2. La app dice «Te escucho» y vibra. 3. La persona dice un comando. 4. La app ejecuta la acción correspondiente.
Flujos alternos	3a. Silencio: «No escuché nada. Intenta otra vez.» 4a. Frase que no es comando: se busca como lugar.
Postcondiciones	La acción se realizó sin tocar más controles.

CU-10. Ajustar apariencia y voz (RF-18, RF-19)

Elemento	Descripción
Actor principal	Persona usuaria
Precondiciones	Ninguna.
Flujo principal	1. La persona toca el botón de perfil de la barra de búsqueda. 2. La app muestra Ajustes. 3. La persona cambia el tema, el tamaño de letra, la voz, los avisos o la clave de rutas. 4. La app aplica el cambio de inmediato y lo guarda.
Flujos alternos	3a. Actualizar lugares: se fuerza la descarga y se dice el resultado.
Postcondiciones	La app se adapta a las preferencias de la persona.

4. Arquitectura del sistema

4.1 Visión general

Ruta Libre es una aplicación cliente que se ejecuta completamente en el dispositivo de la persona usuaria. No existe un servidor propio: la aplicación consume directamente tres servicios públicos (teselas de OpenStreetMap, Overpass API y openrouteservice) y guarda sus datos en el almacenamiento local del teléfono mediante `SharedPreferences`.

El código se organiza en cuatro capas con dependencias en un solo sentido: la presentación depende del estado, el estado depende de los servicios y los servicios dependen de la lógica pura y de los modelos. Ninguna capa inferior conoce a una superior. Esta separación permite probar la lógica más delicada (cálculos geográficos, interpretación de datos de OSM, armado de peticiones de rutas, ciclo de vida de los reportes) sin necesidad de un teléfono ni de conexión a Internet.

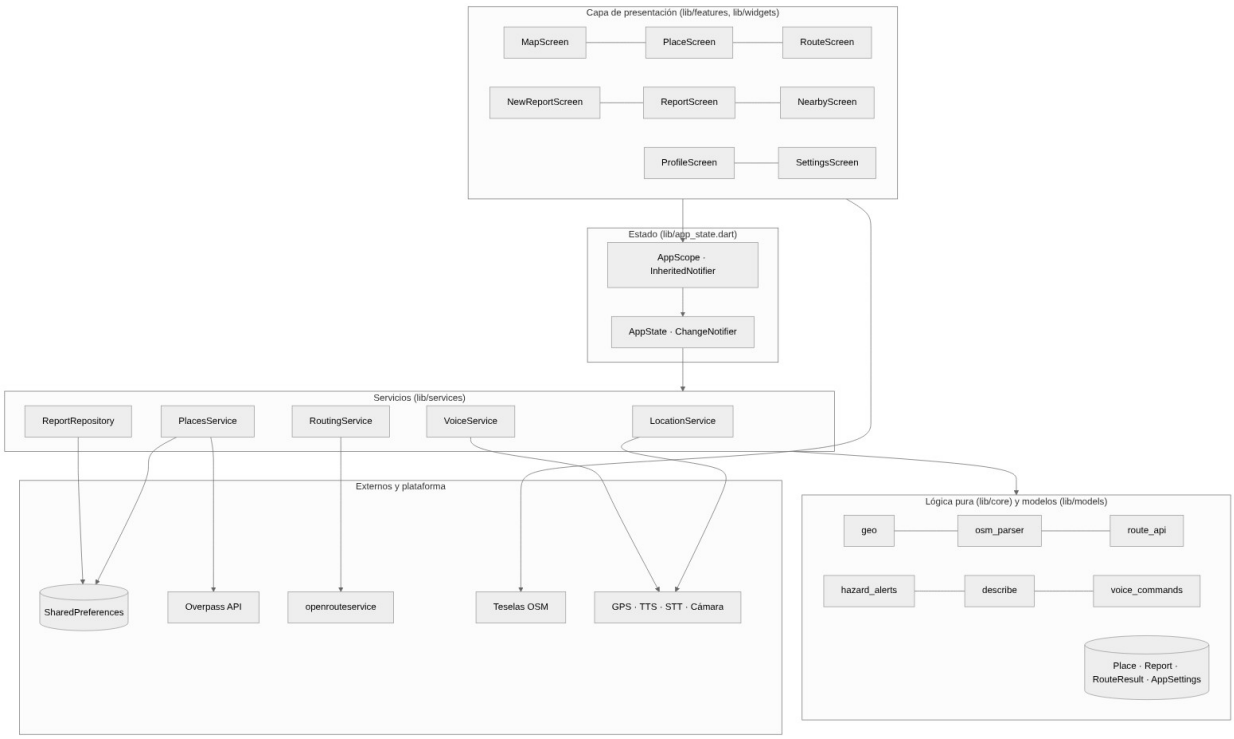


Figura 1. Arquitectura en capas de Ruta Libre y sus dependencias externas.

4.2 Descripción de las capas

Capa	Carpeta	Responsabilidad	Depende de
Presentación	`lib/features`, `lib/widgets`	Pantallas y componentes visuales. Traducen acciones de la persona en llamadas al estado y muestran sus datos.	Estado, modelos, lógica pura (para formatear)

Capa	Carpeta	Responsabilidad	Depende de
Estado	`lib/app_state.dart`	Fuente única de verdad: ajustes, lugares, reportes y ruta activa. Coordina los servicios y dispara los avisos de peligro.	Servicios, modelos, lógica pura
Servicios	`lib/services`	Acceso a recursos externos: GPS, red, almacenamiento, voz. Cada uno es independiente y reemplazable.	Lógica pura, modelos, paquetes de Flutter
Lógica pura	`lib/core`	Funciones sin estado ni dependencias de Flutter: geografía, parser de OSM, API de rutas, textos hablados.	Modelos, `latlong2`
Modelos	`lib/models`	Estructuras de datos inmutables con serialización JSON.	`latlong2`

4.3 Patrón de gestión de estado

Se eligió el patrón más sencillo que ofrece Flutter sin paquetes adicionales: una clase `AppState` que extiende `ChangeNotifier` y se distribuye por el árbol de *widgets* con un `InheritedNotifier` llamado `AppScope`. Cualquier pantalla obtiene el estado con:

```
final app = AppScope.of(context);   // se reconstruye cuando cambia el estado
final app = AppScope.read(context); // solo lectura, para callbacks e initState
```

Esta elección responde al tamaño del proyecto: hay un solo estado global, pocas pantallas y ningún flujo asíncrono complejo que justifique paquetes como Provider, Riverpod o BLoC. Si el proyecto crece (por ejemplo, con sesión de usuario y sincronización con un servidor) la migración a Riverpod es directa, porque las pantallas solo dependen de los métodos públicos de `AppState`.

> **Nota:** `AppScope.read` usa `getElementForInheritedWidgetOfExactType`, que no registra dependencia. Por eso puede llamarse dentro de `initState` y en manejadores de eventos sin provocar reconstrucciones innecesarias.

4.4 Inyección de dependencias

Los servicios se crean una sola vez en `main.dart` y se pasan a `AppState` por su constructor. Esta inyección manual hace explícito qué usa la aplicación y permite sustituir cualquier servicio en pruebas o en producción. El ejemplo más importante es el repositorio de reportes: basta con pasar otra implementación de `ReportRepository` para que los reportes se compartan entre personas.

```
final state = AppState(
  prefs: prefs,
  voice: VoiceService(),
  location: LocationService(),
  places: PlacesService(prefs),
  reportsRepo: LocalReportRepository(prefs),   // cambiar aquí por un repositorio HTTP
  routing: RoutingService(),
```

```
);
await state.init();
runApp(RutaLibreApp(state: state));
```

4.5 Flujo de arranque

1. `main()` inicializa el enlace de Flutter y obtiene `SharedPreferences`.
2. Se construye `AppState`, que lee los ajustes guardados (`_loadSettings`) y se suscribe a los cambios de posición.
3. `state.init()` configura el motor de voz, carga los reportes guardados y, si existen, los lugares en caché de la zona piloto.
4. `RutaLibreApp` decide la pantalla inicial: si no hay perfil guardado muestra la **Bienvenida**; si lo hay, el **Mapa**. La decisión se toma una sola vez (`late final _needsProfile`) para que elegir el perfil no reemplace la pantalla de forma inesperada.
5. Al mostrarse el mapa, `_start()` pide permiso de ubicación, inicia el seguimiento del GPS y descarga (o lee de caché) los lugares de la zona piloto. Al terminar anuncia por voz cuántos lugares accesibles y peligros hay.

4.6 Decisiones de arquitectura

Las decisiones importantes se documentan como registros de decisión de arquitectura (ADR), con su contexto, la decisión y sus consecuencias.

ADR-01. Flutter como marco de desarrollo

Contexto. Se necesitaba una aplicación para Android y iOS con un solo código, con buen soporte de accesibilidad y que también pudiera publicarse en web para demostraciones. **Decisión.** Usar Flutter 3 con Dart. **Consecuencias.** Un solo código para tres plataformas; el árbol de semántica de Flutter se integra con TalkBack, VoiceOver y lectores de pantalla web. A cambio, el tamaño del paquete es mayor que una app nativa (54 MB en release con todas las arquitecturas).

ADR-02. OpenStreetMap como fuente de datos

Contexto. Los mapas comerciales no exponen datos de accesibilidad ni permiten corregirlos. **Decisión.** Usar OpenStreetMap: datos abiertos (ODbL), etiquetas de accesibilidad estandarizadas y posibilidad de que la comunidad corrija los datos. **Consecuencias.** La calidad depende de lo que la comunidad haya registrado; muchos lugares no tienen dato («Sin dato»). Por eso la ficha incluye «¿Es correcta esta información?» y un enlace para corregir en OSM.

ADR-03. Sin servidor en el MVP

Contexto. Construir y operar un servidor tiene costo y tiempo, y aún no se había validado la interfaz con personas usuarias. **Decisión.** Guardar los reportes localmente detrás de la interfaz

`ReportRepository`. **Consecuencias.** Los reportes no se comparten todavía entre teléfonos. La migración está acotada a una clase nueva (capítulo 19).

ADR-04. Caché de lugares por zona

Contexto. La política de la instancia pública de Overpass prohíbe usarla como servidor de una aplicación con muchas consultas. **Decisión.** Descargar cada zona una sola vez y reutilizarla siete días; si el servidor falla, usar los datos vencidos. **Consecuencias.** Muy pocas consultas por persona; los cambios en OSM tardan hasta una semana en verse (se puede forzar desde Ajustes).

ADR-05. Sin brújula

Contexto. Muchos teléfonos económicos no tienen magnetómetro o lo tienen mal calibrado, y una dirección equivocada es peor que una dirección general. **Decisión.** Usar el rumbo del GPS solo cuando la persona camina (velocidad ≥ 0.5 m/s) y puntos cardinales cuando está quieta. **Consecuencias.** Indicaciones relativas confiables al caminar; al estar quieta se dice «al norte» en lugar de «a tu derecha».

ADR-06. Tema claro y oscuro con tokens

Contexto. La primera versión usaba un único tema negro con amarillo de alto contraste, adecuado para baja visión pero poco agradable para otros perfiles. **Decisión.** Definir los colores como *tokens* en una extensión de tema (`RLColors`) con valores para modo claro y oscuro, todos con contraste AA o superior, y seguir el modo del sistema por defecto. **Consecuencias.** Las pantallas no usan colores literales; cambiar la paleta se hace en un solo archivo.

ADR-07. Tipografía Atkinson Hyperlegible

Contexto. Se necesitaba una letra muy legible para baja visión. **Decisión.** Incluir Atkinson Hyperlegible, diseñada por el Braille Institute, con licencia SIL Open Font License, dentro de la app. **Consecuencias.** Letras que no se confunden (I, l, 1; O, 0) y funcionamiento sin conexión; agrega unos 100 KB al paquete.

5. Tecnologías y dependencias

5.1 Lenguaje y marco de trabajo

Componente	Versión	Uso
Flutter (canal estable)	3.47.9	Marco de interfaz multiplataforma.
Dart	3.13.7	Lenguaje de programación. El `pubspec.yaml` exige `sdk: ^3.8.0` y `flutter: >=3.32.0`.
Material 3	Incluido en Flutter	Sistema de componentes base, adaptado con el tema propio.
Android SDK	Plataforma 37, build-tools 36.0.0	Compilación para Android.
Android NDK	28.2.13676358	Requerido por Flutter para compilar código nativo de los complementos.
Java (JBR de Android Studio)	OpenJDK 25	Ejecución de Gradle.
Gradle (Kotlin DSL)	El de la plantilla de Flutter	Sistema de compilación de Android.

5.2 Paquetes de Dart

Las versiones son las que resolvió `flutter pub get` en el equipo de desarrollo (archivo `pubspec.lock`).

Paquete	Versión	Licencia	Para qué se usa
`flutter_map`	8.3.2	BSD-3	Mapa interactivo con capas de teselas, marcadores y polilíneas.
`latlong2`	0.10.1	Apache-2.0	Tipo `LatLng` para coordenadas.
`http`	1.6.0	BSD-3	Peticiones a Overpass y openrouteservice.
`geolocator`	14.1.2	MIT	Permiso y flujo de posiciones del GPS.
`flutter_tts`	4.2.5	MIT	Síntesis de voz en español.
`speech_to_text`	7.5.0	BSD-3	Reconocimiento de voz para comandos.
`image_picker`	1.2.4	Apache-2.0	Tomar la foto de un peligro con la cámara del sistema.
`path_provider`	2.1.6	BSD-3	Carpeta privada de la app para guardar fotos.
`url_launcher`	6.3.3	BSD-3	Abrir la atribución de OSM y la página de un lugar.

Paquete	Versión	Licencia	Para qué se usa
`shared_preferences`	2.5.6	BSD-3	Almacenamiento clave–valor de ajustes, reportes y caché.
`flutter_localizations`	SDK	BSD-3	Textos del sistema en español de México.
`flutter_lints`	6.0.0	BSD-3	Reglas de análisis estático (solo desarrollo).
`flutter_test`	SDK	BSD-3	Pruebas unitarias y de widgets (solo desarrollo).

5.3 Servicios externos

Servicio	Uso	Autenticación	Límites relevantes
Teselas de OpenStreetMap	Imágenes del mapa de fondo.	Ninguna; User-Agent obligatorio.	Uso moderado, sin descarga masiva ni prefetch; atribución visible.
Overpass API (overpass-api.de)	Consulta de lugares por zona.	Ninguna.	No usar como backend; respuestas 429/504 cuando está ocupado.
openrouteservice v2	Rutas a pie y en silla de ruedas.	Clave en el encabezado `Authorization`.	Plan estándar: 2,000 peticiones al día, 40 por minuto.

5.4 Herramientas de desarrollo

- **Visual Studio Code** con la extensión oficial de Flutter (incluye Dart) — editor principal. El repositorio incluye `.vscode/launch.json` con dos configuraciones: Chrome en el puerto 8080 y Android.
- **Android Studio** — solo como proveedor del SDK de Android y de Java (JBR).
- **Git** — control de versiones.
- **GitHub Actions** — integración continua (capítulo 17).
- **PowerShell** — scripts de instalación y compilación en `setup/`.

6. Estructura del proyecto

6.1 Árbol de directorios

```
ruta_libre/
├── lib/
│   ├── main.dart                Punto de entrada: crea servicios y estado
│   ├── app.dart                 MaterialApp, temas, idioma y escala de texto
│   ├── app_state.dart           Estado global (AppState) y AppScope
│   ├── config.dart              Zona piloto, URLs, User-Agent, tiempos
│   ├── core/                    Lógica pura, sin Flutter
│   │   ├── describe.dart        Textos hablados de lugares y reportes
│   │   ├── geo.dart             Distancias, rumbos, direcciones, formatos
│   │   ├── hazard_alerts.dart   Qué peligros anunciar y cuándo
│   │   ├── osm_parser.dart      Consulta Overpass y conversión a Place
│   │   ├── route_api.dart       Petición y respuesta de openrouteservice
│   │   ├── text_utils.dart      Normalización de texto (acentos)
│   │   ├── theme.dart           Tema claro/oscuro y tokens RLColors
│   │   └── voice_commands.dart  Interpretación de comandos de voz
│   ├── models/
│   │   ├── place.dart           Place, Accessibility, PlaceKind
│   │   ├── report.dart          Report, ReportType, Severity
│   │   ├── route_result.dart    RouteResult, RouteStep
│   │   └── settings.dart        AppSettings, MobilityProfile, ThemePreference
│   ├── services/
│   │   ├── location_service.dart GPS y rumbo
│   │   ├── places_service.dart  Overpass + caché por zona
│   │   ├── report_repository.dart Interfaz y versión local
│   │   ├── routing_service.dart Cliente HTTP de openrouteservice
│   │   └── voice_service.dart   Texto a voz y voz a texto
│   ├── features/                Una carpeta por pantalla
│   │   ├── map/map_screen.dart
│   │   ├── nearby/nearby_screen.dart
│   │   ├── onboarding/profile_screen.dart
│   │   ├── place/place_screen.dart
│   │   ├── report/new_report_screen.dart
│   │   ├── report/report_screen.dart
│   │   ├── route/route_screen.dart
│   │   └── settings/settings_screen.dart
│   └── widgets/
│       ├── base_map.dart        Teselas, línea de ruta, crédito de OSM
│       ├── map_markers.dart     Marcadores, insignias y leyenda
│       ├── report_photo.dart    Foto de un reporte (archivo o URL)
│       └── ui.dart              Componentes compartidos
├── test/                        36 pruebas automatizadas
├── assets/fonts/                Atkinson Hyperlegible + licencia OFL
├── tool/patch_platforms.dart    Permisos de Android e iOS
├── setup/                       Scripts de Windows (.bat y .ps1)
├── .github/workflows/ci.yml     Integración continua
├── .vscode/                     Configuración de VS Code
├── analysis_options.yaml        Reglas del analizador
├── pubspec.yaml                 Dependencias y recursos
└── README.md
```

> **Nota:** Las carpetas nativas `android/`, `ios/` y `web/` no se guardan en el repositorio: se generan con `flutter create` y se ajustan con `tool/patch_platforms.dart`. Así el repositorio contiene solo código propio y la plantilla de Flutter se actualiza junto con el SDK.

6.2 Descripción de archivos

Archivo	Líneas	Contenido principal
`main.dart`	27	Crea `SharedPreferences`, los cinco servicios y `AppState`; llama a `init()` y `runApp`.
`app.dart`	78	`RutaLibreApp`: decide pantalla inicial, aplica temas claro/oscuro, idioma es-MX y escala de texto acotada a 2.0.
`app_state.dart`	308	Ajustes, lugares visibles, reportes activos, rutas, retroalimentación de lugares y avisos de peligro.
`config.dart`	33	Constantes: centro y radio del piloto, User-Agent, URLs, TTL de caché, distancia y espera de avisos.
`core/geo.dart`	123	Haversine, rumbo, ángulo relativo, puntos cardinales, dirección relativa, formatos de distancia y tiempo, cuadro alrededor de un punto.
`core/osm_parser.dart`	142	Consulta Overpass QL, tablas de categorías en español, clasificación de cruces, conversión a `Place`.
`core/route_api.dart`	125	Cuerpo de la petición a ORS, conteo de zonas evitadas, lectura de GeoJSON, textos hablados de la ruta.
`core/hazard_alerts.dart`	25	Filtro y orden de peligros que deben anunciarse.
`core/voice_commands.dart`	33	Palabras clave y conversión de frase a comando.
`core/describe.dart`	66	«hace 5 minutos», «a 40 metros, a tu derecha», descripción hablada de lugares y reportes.
`core/text_utils.dart`	37	Normalización sin acentos y utilidades de texto.
`core/theme.dart`	252	`AppTheme.light()`/`dark()`, colores de estado y extensión `RLColors`.
`models/*.dart`	401	Modelos con `toJson`/`fromJson` y enumeraciones con etiquetas en español.
`services/*.dart`	456	GPS, Overpass con caché, reportes locales, ORS y voz.
`features/**`	3,045	Ocho pantallas; la más extensa es el mapa (971 líneas).
`widgets/*.dart`	879	Marcadores, componentes compartidos, mapa base y fotos.

6.3 Convenciones de código

- Nombres de clases, métodos y variables en inglés (convención de Dart); comentarios, textos de interfaz y mensajes en español de México.
- Comentarios de documentación (`///`) en toda clase y función pública, explicando el **porqué** además del qué.

- Clases privadas (prefijo _) para los componentes que solo usa una pantalla.
- Modelos inmutables con constructores `const`; los cambios devuelven una copia (`copyWith`, `confirm`, `voteGone`).
- Reglas del analizador adicionales a `flutter_lints`: `prefer_const_constructors`, `prefer_final_locals`, `always_declare_return_types`.
- El análisis estático debe terminar con «No issues found!» antes de compilar una versión.

7. Modelo de datos

Todos los modelos son clases inmutables con serialización a JSON. No se usa una base de datos relacional: los datos se guardan como cadenas JSON en `SharedPreferences`, suficiente para el volumen del prototipo (cientos de lugares por zona y decenas de reportes).

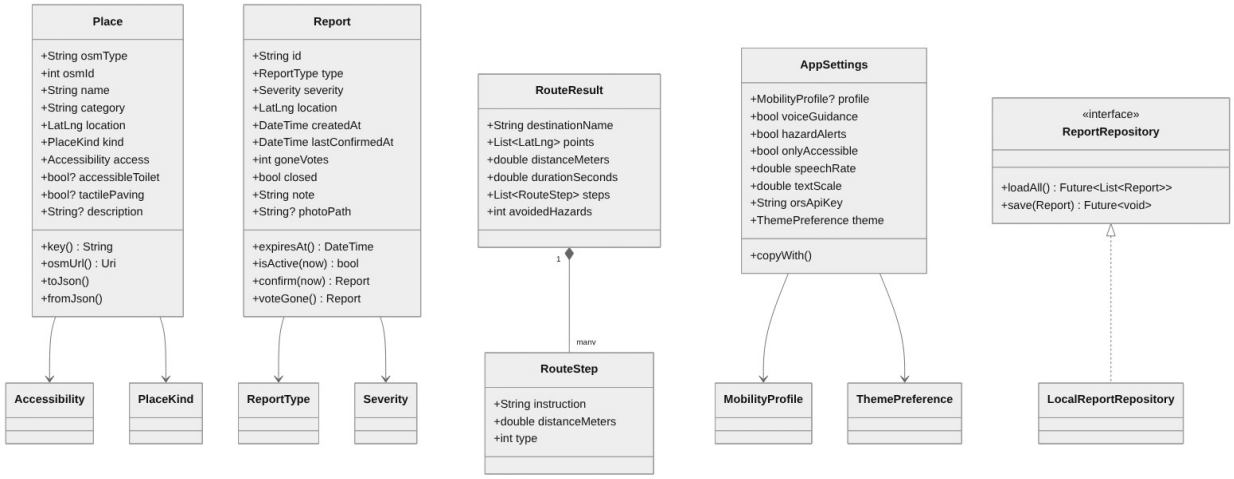


Figura 2. Diagrama de clases de los modelos principales.

7.1 Place (lugar)

Representa un elemento de OpenStreetMap relevante para la accesibilidad: un comercio, un servicio, una parada o un cruce peatonal.

Campo	Tipo	Origen en OSM	Descripción
`osmType`	`String`	tipo del elemento	`node`, `way` o `relation`.
`osmId`	`int`	id	Identificador numérico en OSM. Junto con `osmType` forma la clave única `key`.
`name`	`String`	`name`	Nombre; si no tiene, se usa la categoría.
`category`	`String`	`amenity`, `shop`, `highway`, `railway`	Categoría traducida al español («Farmacia», «Parada de autobús»).
`location`	`LatLng`	`lat`/`lon` o `center`	Coordenadas; para *ways* se usa el centro calculado por Overpass.
`kind`	`PlaceKind`	derivado	`poi` (lugar) o `crossing` (cruce o borde de banqueta).
`access`	`Accessibility`	`wheelchair` o `kerb`	Estado de accesibilidad.

Campo	Tipo	Origen en OSM	Descripción
`accessibleToilet`	`bool?`	`toilets:wheelchair`	Baño accesible; `null` si no hay dato.
`tactilePaving`	`bool?`	`tactile_paving`	Guía podotáctil; `null` si no hay dato.
`description`	`String?`	`wheelchair:description`	Nota libre de accesibilidad.

Enumeración Accessibility

Valor	Etiqueta	Explicación	Etiqueta OSM	Color	Ícono
`yes`	Accesible	entrada sin escalón	`yes`, `designated`	#157F3D	✓ (check)
`limited`	Acceso limitado	escalón pequeño o acceso parcial	`limited`	#A16207	− (remove)
`no`	No accesible	tiene escalones o áreas sin acceso	`no`	#B42318	× (close)
`unknown`	Sin dato	nadie ha registrado su accesibilidad	ausente u otro	#5B6773	? (question_mark)

Ejemplo de Place en JSON

```
{
  "osmType": "node",
  "osmId": 1,
  "name": "Farmacia Ejemplo",
  "category": "Farmacia",
  "lat": 19.396,
  "lon": -99.091,
  "kind": "poi",
  "access": "yes",
  "accessibleToilet": false,
  "tactilePaving": null,
  "description": "Rampa por la entrada lateral"
}
```

7.2 Report (reporte de peligro)

Un peligro u obstáculo reportado por la comunidad. Es inmutable: confirmar o votar devuelve un reporte nuevo que reemplaza al anterior en el repositorio.

Campo	Tipo	Descripción
`id`	`String`	Microsegundos desde la época Unix al crearlo; único por dispositivo.
`type`	`ReportType`	Tipo de peligro (ver tabla siguiente).

Campo	Tipo	Descripción
`severity`	`Severity`	Baja, media o alta. Se propone la del tipo y la persona puede cambiarla.
`location`	`LatLng`	Ubicación GPS de quien reporta en el momento de enviarlo.
`createdAt`	`DateTime`	Fecha de creación.
`lastConfirmedAt`	`DateTime`	Última confirmación «Sigue ahí» (al crearlo, igual a `createdAt`).
`goneVotes`	`int`	Votos «Ya no está» seguidos desde la última confirmación.
`closed`	`bool`	Verdadero cuando `goneVotes` llega a `votesToClose` (2).
`note`	`String`	Nota opcional de hasta 140 caracteres.
`photoPath`	`String?`	Ruta del archivo de la foto en la carpeta privada (o URL temporal en web).

Tipos de reporte y tiempos de vida

El tiempo de vida (TTL) refleja qué tan rápido suele desaparecer cada tipo de obstáculo: un coche sobre una rampa se mueve en horas, una banqueta rota dura meses.

Valor	Etiqueta	Gravedad por defecto	TTL
`blockedRamp`	Rampa bloqueada	Alta	6 horas
`openDrain`	Coladera o registro abierto	Alta	3 días
`brokenSidewalk`	Banqueta rota o levantada	Media	30 días
`construction`	Obra o desvío	Media	14 días
`noRamp`	Sin rampa en el cruce	Media	90 días
`elevatorOut`	Elevador fuera de servicio	Alta	2 días
`flooding`	Inundación o encharcamiento	Alta	1 día
`other`	Otro peligro	Media	7 días

Ciclo de vida

Un reporte está **activo** mientras no esté cerrado y la fecha actual sea anterior a `expiresAt = lastConfirmedAt + type.ttl`. Las transiciones son:

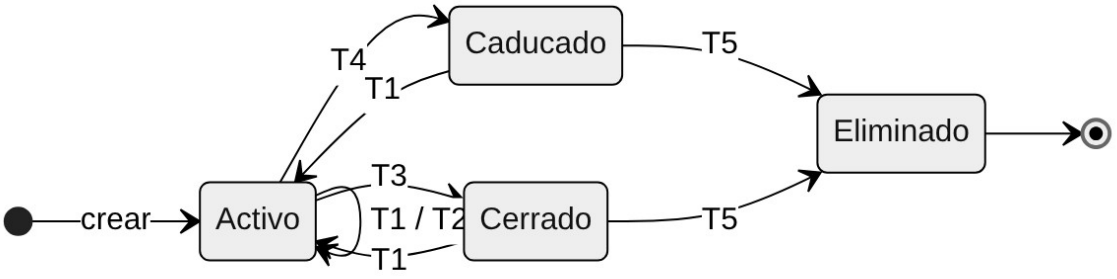


Figura 3. Diagrama de estados de un reporte.

Transición	Evento	Efecto
T1	«Sigue ahí» (`confirm(now)`)	`lastConfirmedAt = now`, `goneVotes = 0`, `closed = false`. Reactiva incluso un reporte caducado o cerrado.
T2	Primer «Ya no está» (`voteGone()`)	`goneVotes = 1`; sigue activo.
T3	Segundo «Ya no está» seguido	`goneVotes = 2`, `closed = true`.
T4	Pasa el tiempo	Cuando `now ≥ expiresAt` deja de estar activo; no se muestra en el mapa.
T5	Limpieza	Al cargar, `LocalReportRepository` descarta reportes con más de 30 días después de `expiresAt`.

7.3 RouteResult y RouteStep

Campo	Tipo	Descripción
`destinationName`	`String`	Nombre del lugar de destino.
`points`	`List<LatLng>`	Geometría de la ruta en orden.
`distanceMeters`	`double`	Distancia total.
`durationSeconds`	`double`	Duración estimada por ORS según el perfil.
`steps`	`List<RouteStep>`	Indicaciones: texto, distancia y tipo de maniobra.
`avoidedHazards`	`int`	Cuántos peligros se enviaron como zonas a evitar.

El campo `RouteStep.type` conserva el código de maniobra de ORS, que la pantalla de ruta convierte en un ícono:

Código ORS	Maniobra	Ícono en la app
0, 2, 4, 12	Izquierda, izquierda cerrada, ligera izquierda, mantener izquierda	`turn_left`
1, 3, 5, 13	Derecha y variantes	`turn_right`
6	Recto	`straight`
7, 8	Entrar o salir de glorieta	`roundabout_left`
9	Vuelta en U	`u_turn_left`
10	Llegada	`flag_outlined`
11	Salida (inicio)	`directions_walk`

7.4 AppSettings (ajustes)

Campo	Tipo	Valor por defecto	Descripción
`profile`	`MobilityProfile?`	`null`	Perfil de movilidad; `null` hasta la primera elección.
`voiceGuidance`	`bool`	`true`	Anunciar pantallas y acciones (`announce`).
`hazardAlerts`	`bool`	`true`	Avisos de peligros cercanos.
`onlyAccessible`	`bool`	`false`	Filtro «Solo accesibles» (en silla de ruedas inicia en `true`).
`speechRate`	`double`	0.5	Velocidad de voz (0.3 lenta a 0.7 rápida).
`textScale`	`double`	1.2	Escala de texto (1.0 a 2.0).
`orsApiKey`	`String`	`""`	Clave de openrouteservice escrita por la persona.
`theme`	`ThemePreference`	`system`	Sistema, claro u oscuro.

7.5 Persistencia

Todos los datos se guardan en `SharedPreferences` (en Android, un archivo XML privado de la app; en web, `localStorage`). Las claves son:

Clave	Contenido	Escrita por
`settings`	JSON de `AppSettings`.	`AppState.updateSettings`
`reports`	Arreglo JSON de `Report`.	`LocalReportRepository.save`

Clave	Contenido	Escrita por
`zone_{lat}_{lon}_{radio}`	Objeto con `savedAt` y `places` (arreglo de `Place`). Lat/lon redondeadas a 3 decimales (~100 m).	`PlacesService.load`
`place_feedback`	Lista de cadenas JSON: lugar, estado mostrado, si es correcto y fecha.	`AppState.saveFeedback`

```
// Ejemplo de la clave de caché de la zona piloto
zone_19.396_-99.091_1000
{
  "savedAt": "2026-10-30T13:40:12.512",
  "places": [ { "osmType": "node", "osmId": 1, ... }, ... ]
}
```

> **Importante:** Las fotos no se guardan en `SharedPreferences`: se copian a la carpeta de documentos de la app (`getApplicationDocumentsDirectory`) con el nombre `reporte_<milisegundos>.jpg`, y el reporte guarda solo la ruta. En web, la foto es una URL temporal del navegador que deja de existir al cerrar la pestaña; `ReportPhoto` muestra «La foto ya no está disponible» en ese caso.

8. Lógica de negocio (lib/core)

La carpeta `core` concentra los algoritmos del sistema. Ningún archivo importa Flutter, por lo que se prueban con `flutter test` en milisegundos y sin emulador.

8.1 Cálculos geográficos (geo.dart)

Distancia de Haversine

La distancia entre dos puntos sobre la superficie terrestre se calcula con la fórmula de Haversine, que trata la Tierra como una esfera de radio R = 6,371,000 m. Para las distancias de una zona urbana (menos de unos kilómetros) el error frente a un elipsoide es menor a 0.5 %, despreciable para mostrar «120 m».

```
h = sin²(Δφ/2) + cos φ1 · cos φ2 · sin²(Δλ/2)
d = 2 · R · asin(√h)

donde φ es la latitud y λ la longitud, en radianes.
```

Rumbo inicial

El rumbo de A hacia B, en grados desde el norte en el sentido de las manecillas del reloj, se obtiene con:

```
y = sin(Δλ) · cos φ2
x = cos φ1 · sin φ2 − sin φ1 · cos φ2 · cos(Δλ)
rumbo = (atan2(y, x) en grados + 360) mod 360
```

Dirección relativa

Cuando se conoce hacia dónde camina la persona (`heading`), el ángulo relativo es `rumbo − heading` normalizado al intervalo (−180, 180]. Negativo significa a la izquierda; positivo, a la derecha. El ángulo se convierte en una expresión natural:

Ángulo relativo absoluto	Expresión
0° a 22.5°	«al frente»
22.5° a 67.5°	«adelante a la derecha» / «adelante a la izquierda»
67.5° a 112.5°	«a tu derecha» / «a tu izquierda»
112.5° a 157.5°	«atrás a la derecha» / «atrás a la izquierda»
157.5° a 180°	«atrás»

Si no hay rumbo, se usa uno de ocho puntos cardinales: el índice es `((rumbo mod 360) + 22.5) div 45 mod 8` sobre la lista «al norte, al noreste, al este, al sureste, al sur, al suroeste, al oeste, al noroeste».

Formatos de distancia y tiempo

Función	Uso	Reglas	Ejemplos
`formatDistance`	Voz	< 10 m: «menos de 10 metros»; < 100 m: múltiplos de 5; < 1 km: múltiplos de 10; ≥ 1 km: un decimal.	42 → «40 metros»; 1,260 → «1.3 kilómetros»
`formatDistanceShort`	Pantalla	< 1 km: múltiplos de 10 con «m»; ≥ 1 km: un decimal con «km».	118 → «120 m»; 1,000 → «1 km»
`formatDuration`	Pantalla	Minutos redondeados hacia arriba, mínimo 1; desde 60 min, horas y minutos.	120 s → «2 min»; 3,900 s → «1 h 5 min»

El redondeo es intencional: decir «cuarenta y dos metros» da una falsa sensación de precisión cuando el GPS tiene un error típico de 5 a 15 metros.

Cuadro alrededor de un peligro

`squareRing(centro, metros)` devuelve un anillo GeoJSON cerrado de cinco vértices en orden [lon, lat]. La mitad del lado en grados de latitud es `metros / 2 / 111,320`; en longitud se divide además entre `cos(latitud)` para compensar que los meridianos se juntan hacia los polos. Se usa para pedir a ORS que evite un cuadro de 30 m alrededor de cada peligro.

8.2 Interpretación de OpenStreetMap (osm_parser.dart)

Consulta Overpass QL

La consulta pide, dentro de un radio alrededor del centro, todos los elementos con etiqueta `wheelchair`, los servicios y comercios más útiles, paradas de autobús y elevadores, estaciones y los nodos con información de bordes de banqueta o guía podotáctil. `out center tags` hace que las vías y relaciones incluyan su punto central.

```
[out:json][timeout:25];
(
  nwr["wheelchair"](around:1000,19.3958,-99.0912);
  nwr["amenity"~"^(pharmacy|hospital|clinic|doctors|dentist|bank|atm|restaurant|cafe|
fast_food|toilets|school|college|university|police|post_office|marketplace|townhall|library|
place_of_worship)$"](around:...);
  nwr["shop"~"^(supermarket|convenience|chemist|bakery|mall|department_store)$"](around:...);
  node["highway"~"^(bus_stop|elevator)$"](around:...);
  nwr["railway"="station"](around:...);
  node["kerb"](around:...);
  node["tactile_paving"](around:...);
);
out center tags;
```

Traducción de categorías

Etiqueta OSM	Categoría en la app
amenity=pharmacy, shop=chemist	Farmacia

Etiqueta OSM	Categoría en la app
amenity=hospital / clinic / doctors / dentist	Hospital / Clínica / Consultorio / Dentista
amenity=bank / atm	Banco / Cajero
amenity=restaurant / cafe / fast_food	Restaurante / Cafetería / Comida rápida
amenity=toilets	Baños públicos
amenity=school / college / university	Escuela / Escuela / Universidad
amenity=police / post_office / marketplace	Policía / Correos / Mercado
amenity=townhall / library / place*of*worship	Oficina de gobierno / Biblioteca / Templo
shop=supermarket / convenience / bakery	Supermercado / Tienda / Panadería
shop=mall / department_store	Centro comercial / Tienda departamental
shop=* (otro)	Comercio
highway=bus_stop / elevator	Parada de autobús / Elevador
railway=station, public_transport=station	Estación
kerb, crossing, tactile_paving sin nombre	Cruce peatonal
building=*	Edificio
cualquier otro	Lugar

Accesibilidad de los cruces

Un cruce no suele tener etiqueta `wheelchair`, pero sí `kerb`, que describe el borde de la banqueta. Se traduce así (si existe `wheelchair` explícito, tiene prioridad):

kerb	Significado	Accesibilidad
`flush`, `no`	Al ras de la calle	Accesible
`lowered`	Rebajado (rampa)	Accesible
`rolled`	Redondeado	Acceso limitado
`raised`, `yes`	Elevado (escalón)	No accesible
otro o ausente	—	Sin dato

Reglas de descarte

- Elementos sin etiquetas o sin coordenadas (ni `lat/lon` ni `center`).
- Elementos sin nombre, cuya categoría es «Lugar» y sin dato de accesibilidad: no aportan nada a la persona usuaria.

8.3 Petición y respuesta de rutas (route_api.dart)

Cuerpo de la petición

```
{
  "coordinates": [[-99.0912, 19.3958], [-99.0905, 19.3970]],   // [lon, lat]
  "language": "es",
  "instructions": true,
  "units": "m",
  "options": {
    "avoid_features": ["steps"],                  // si el perfil evita escalones
    "profile_params": {                           // solo silla de ruedas
      "restrictions": {
        "maximum_incline": 6,
        "maximum_sloped_kerb": 0.06,
        "smoothness_type": "good"
      }
    },
    "avoid_polygons": {                           // si hay peligros que evitar
      "type": "MultiPolygon",
      "coordinates": [ [ [[lon,lat], ... 5 vértices ...] ], ... ]
    }
  }
}
```

Solo se envían como zonas a evitar los peligros de gravedad media o alta que estén a más de 25 m del origen y del destino. La razón: openrouteservice rechaza la petición si el origen o el destino caen dentro de un polígono a evitar, y una persona que está junto a un peligro necesita poder salir de ahí.

Lectura de la respuesta

La respuesta es un `FeatureCollection` GeoJSON. De la primera *feature* se toman la geometría (coordenadas [lon, lat] convertidas a `LatLng`), el resumen (`distance`, `duration`) y los pasos de todos los segmentos (`instruction`, `distance`, `type`). Si no hay *features* se lanza `FormatException`, que el servicio convierte en el mensaje «La respuesta de la ruta no se pudo leer».

Textos hablados

- `spokenStep`: «Gira a la derecha en Av. Té. Sigue 80 metros.» (no agrega distancia en la llegada ni en pasos de menos de 5 m).
- `spokenSummary`: «Ruta a Farmacia: 120 metros, unos 2 minutos caminando. Evita 1 peligro reportado.»

8.4 Avisos de peligro (hazard_alerts.dart)

`hazardsToAnnounce` recibe todos los reportes, la posición actual, la hora, el mapa de últimos avisos, el radio y el tiempo de espera, y devuelve los reportes que deben anunciarse, el más cercano primero. Un reporte se anuncia si cumple las cuatro condiciones:

1. Está activo (no cerrado ni caducado).

2. Su gravedad no es baja.
3. Está a 30 m o menos (`AppConfig.hazardAlertMeters`).
4. No se anunció en los últimos 10 minutos (`AppConfig.hazardAlertCooldown`).

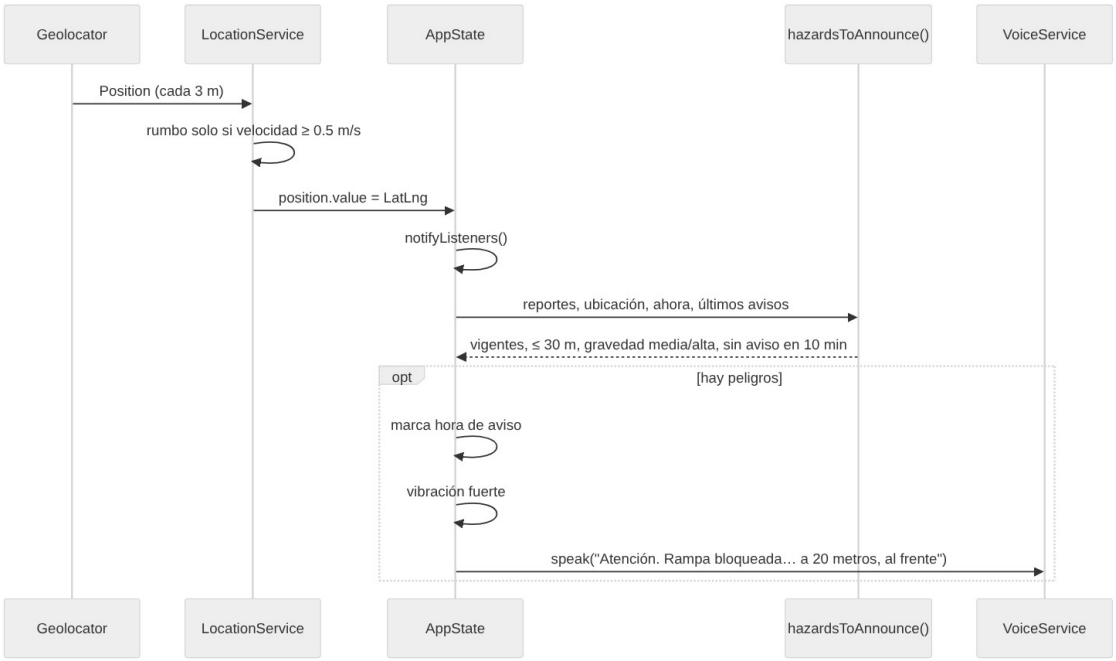


Figura 4. Secuencia de un aviso de peligro al moverse.

Solo se dice en voz alta el más cercano, pero todos los que cumplen se marcan como avisados para no encadenar varios mensajes seguidos. El aviso se dice aunque la guía por voz esté desactivada, porque se trata de seguridad; solo se omite si la persona apagó **Avisar peligros cercanos**. Quien crea un reporte ya conoce el peligro, por lo que se marca como avisado al crearlo.

8.5 Comandos de voz (voice_commands.dart)

La frase reconocida se normaliza (minúsculas, sin acentos) y se busca la primera palabra clave contenida, en este orden de prioridad:

Prioridad	Comando	Palabras clave	Acción en el mapa
1	`report`	reportar, reporta, peligro, obstaculo, avisar	Abre **Reportar peligro**.
2	`clearRoute`	quitar ruta, cancelar ruta, borrar ruta	Quita la ruta activa.
3	`whereAmI`	donde estoy, mi ubicacion	Dice el lugar más cercano y su dirección.
4	`nearby`	cerca, que hay, alrededor, lugares, buscar, farmacia, bano	Abre **Cerca de ti**.
5	`settings`	ajuste, configuracion, perfil	Abre **Ajustes**.

Prioridad	Comando	Palabras clave	Acción en el mapa
—	`unknown`	cualquier otra frase	Se usa como texto de búsqueda.

El orden importa: «hay un peligro cerca» contiene «peligro» y «cerca»; debe interpretarse como un reporte, no como una consulta. Esta regla está cubierta por la prueba «un peligro cerca es un reporte, no una consulta».

8.6 Descripciones habladas (describe.dart)

Generan el texto que se lee al abrir una ficha o al escuchar la lista, y también la etiqueta semántica de los marcadores para el lector de pantalla. Ejemplo para un lugar:

```
«Farmacia Ejemplo, farmacia. Accesible: entrada sin escalón. Su baño no es accesible.
Rampa por la entrada lateral. A 40 metros, a tu derecha.»
```

Y para un reporte: «Rampa bloqueada, gravedad alta. A 20 metros, al frente. Coche sobre la rampa. Confirmado hace 5 minutos.»

9. Servicios (lib/services)

Cada servicio encapsula un recurso externo. Ninguno conoce a los demás ni a las pantallas; `AppState` los coordina. Todos reciben sus dependencias por constructor (cliente HTTP, `SharedPreferences`, motores de voz) para poder sustituirlas en pruebas.

9.1 LocationService

Gestiona el permiso de ubicación y el flujo de posiciones del GPS mediante el paquete `geolocator`.

Miembro	Tipo	Descripción
`position`	`ValueNotifier<LatLng?>`	Última posición; `null` hasta la primera lectura. Las pantallas pueden escucharlo con `ValueListenableBuilder`.
`accuracy`	`double?`	Precisión de la última lectura en metros (se muestra al reportar).
`heading`	`double?`	Rumbo de desplazamiento; `null` si la velocidad es menor a 0.5 m/s o el GPS no lo reporta.
`start()`	`Future<String?>`	Verifica que la ubicación esté encendida, pide permiso y comienza el flujo. Devuelve un mensaje si no es posible.
`dispose()`	`void`	Cancela la suscripción y libera el notificador.

Configuración del flujo: precisión `LocationAccuracy.best` y `distanceFilter: 3`, es decir, una nueva posición cada 3 metros de desplazamiento. Este valor equilibra la frescura de las distancias y el consumo de batería, y evita reconstruir la interfaz cuando el GPS «tiembla» con el teléfono quieto.

Situación	Mensaje devuelto
Servicio de ubicación apagado	«La ubicación del teléfono está apagada.»
Permiso negado o negado para siempre	«Ruta Libre necesita tu ubicación para mostrar qué hay cerca.»
Error en el flujo	Se ignora (`onError` vacío); se conserva la última posición válida.

9.2 PlacesService

Descarga los lugares de una zona desde Overpass y los guarda en caché. Expone tres métodos:

Método	Descripción
`cached(centro, radio)`	Devuelve los lugares guardados de la zona aunque estén vencidos, o `null`. Se usa al arrancar para mostrar algo de inmediato.
`isFresh(centro, radio)`	Indica si la zona se guardó hace menos de `AppConfig.placesCacheTtl` (7 días).
`load(centro, {radio, force})`	Devuelve la caché si está vigente y no se fuerza; si no, descarga, guarda y devuelve. Si la descarga falla y existe caché vieja, la devuelve.

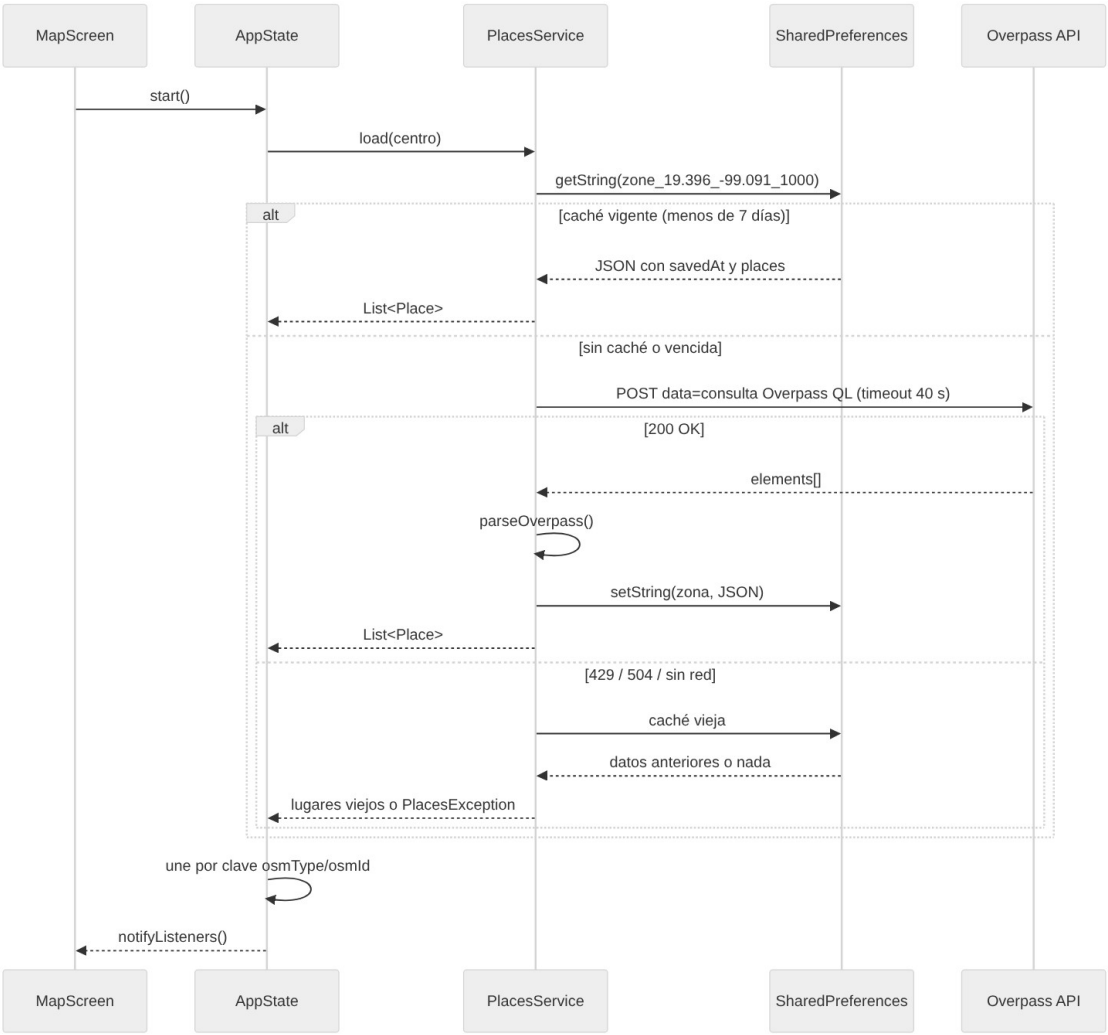


Figura 5. Secuencia de carga de lugares con caché por zona.

Clave de la zona

Las coordenadas del centro se redondean a tres decimales (aproximadamente 110 m de latitud) para que pequeños desplazamientos del centro reutilicen la misma caché. La clave incluye el radio porque una zona de 1 km no es la misma que una de 500 m.

Manejo de errores

Situación	Excepción	Mensaje
Sin conexión o tiempo agotado (40 s)	`PlacesException`	«No hay conexión para descargar los lugares de esta zona.»
HTTP 429 o 504	`PlacesException`	«El servidor de OpenStreetMap está ocupado. Intenta en un minuto.»
Otro código distinto de 200	`PlacesException`	«No se pudieron descargar los lugares (error N).»

En web no se envía el encabezado `User-Agent` porque los navegadores no permiten modificarlo; el navegador envía el suyo.

9.3 ReportRepository y LocalReportRepository

`ReportRepository` es una interfaz con dos operaciones asíncronas: `loadAll()` y `save(report)`. La implementación local guarda todos los reportes como un arreglo JSON bajo la clave `reports`.

- `loadAll()` descarta los reportes cuya fecha de caducidad pasó hace más de 30 días, para que el almacenamiento no crezca sin límite.
- `save(report)` reemplaza el reporte con el mismo `id` o lo agrega al final.

> **Nota:** Que las operaciones sean asíncronas aunque `SharedPreferences` responda casi al instante es deliberado: una implementación remota (HTTP) tendrá la misma firma y las pantallas no tendrán que cambiar.

9.4 RoutingService

Cliente HTTP de openrouteservice. Su método `route` recibe la clave, el origen, el destino, su nombre, el perfil y la lista de peligros, y devuelve un `RouteResult` o lanza `RoutingException` con un mensaje listo para mostrar y decir en voz alta.

Aspecto	Valor
URL	`https://api.openrouteservice.org/v2/directions/{perfil}/geojson`
Método	POST
Encabezados	`Authorization: <clave>`, `Content-Type: application/json; charset=utf-8`, `Accept: application/geo+json, application/json`
Tiempo límite	25 segundos
Cuerpo	Generado por `buildRouteRequest` (sección 8.3)

Respuesta	Mensaje para la persona
Clave vacía (antes de llamar)	«Falta la clave de openrouteservice. Agrégala en Ajustes, en la sección Rutas.»
Sin conexión o tiempo agotado	«No hay conexión para calcular la ruta.»
200 con respuesta ilegible	«La respuesta de la ruta no se pudo leer.»
401 o 403	«La clave de openrouteservice no es válida.»
429	«Se alcanzó el límite de rutas por ahora. Intenta en un minuto.»
Otro error con mensaje	«No se encontró una ruta accesible (mensaje del servidor).»
Otro error sin mensaje	«No se encontró una ruta accesible hasta ese lugar.»

9.5 VoiceService

Encapsula la síntesis de voz (`flutter_tts`) y el reconocimiento de voz (`speech_to_text`) en español.

Inicialización

`init(speechRate)` intenta usar `es-MX`; si el motor no lo tiene instalado usa `es-ES`. Cada ajuste (idioma, velocidad, volumen, esperar fin de habla y, en iOS, instancia compartida) se aplica dentro de un bloque `attempt` independiente: si un motor o un navegador no soporta uno, los demás se aplican y la aplicación no se detiene.

Hablar

Método	Comportamiento
`speak(texto)`	Detiene lo que se estaba diciendo (con tope de 2 s) y dice el texto. Espera a que termine con un tope de 70 ms por letra, entre 3 y 30 segundos. Siempre habla.
`announce(texto)`	Igual que `speak` pero solo si `guidanceEnabled` es verdadero. Se usa para anuncios de navegación («Ajustes», «Mapa…»).
`stop()`	Detiene la voz.

> **Importante:** Los topes de tiempo existen porque en algunos navegadores el motor de voz nunca avisa que terminó. Sin ellos, el primer `await speak(...)` dejaba la pantalla de bienvenida detenida en la versión web. No deben quitarse.

Escuchar

`listenOnce()` inicializa el reconocedor la primera vez, elige el idioma `es_MX` (o el primer español disponible), detiene la voz, vibra para indicar que empezó a escuchar y espera una frase de hasta

8 segundos, terminando antes si hay una pausa de 3 segundos. Devuelve la frase o `null` si no se entendió nada. El notificador `listening` permite a la interfaz mostrar «Escuchando».

10. Estado y navegación

10.1 AppState

Es la única clase que conocen las pantallas. Agrupa sus responsabilidades en secciones:

Sección	Miembros	Descripción
Arranque	`init()`, `start()`	Prepara voz, reportes y caché; luego GPS y descarga de la zona piloto.
Ajustes	`settings`, `profile`, `updateSettings()`, `chooseProfile()`	Guarda los ajustes y los aplica a la voz. `chooseProfile` fija además voz y filtro según el perfil.
Lugares	`allPlaces`, `visiblePlaces`, `loadPlaces()`, `placesNearby()`, `loadingPlaces`, `placesError`	Une los lugares de varias zonas sin duplicar (por `key`). `visiblePlaces` aplica «Solo accesibles».
Reportes	`activeReports()`, `reportsNearby()`, `addReport()`, `confirmReport()`, `voteGone()`, `reportById()`	Crea, confirma y descarta reportes; los guarda en el repositorio.
Rutas	`route`, `routingKey`, `hasRoutingKey`, `routeTo()`, `clearRoute()`	La clave de Ajustes tiene prioridad sobre la de compilación.
Retroalimentación	`saveFeedback()`	Guarda «¿Es correcta?» para enviarlo a OSM más adelante.
Avisos	`_onMove()`	En cada posición nueva: redibuja y evalúa avisos de peligro.

Notificaciones de cambio

`AppState` llama a `notifyListeners()` al cambiar ajustes, lugares, reportes o ruta, y en cada posición nueva del GPS. Como el GPS solo avisa cada 3 m, esto no representa una carga importante. Para evitar reconstruir la aplicación completa al moverse, `RutaLibreApp` no escucha todo el estado: solo reconstruye `MaterialApp` cuando cambia la preferencia de tema.

10.2 Mapa de navegación

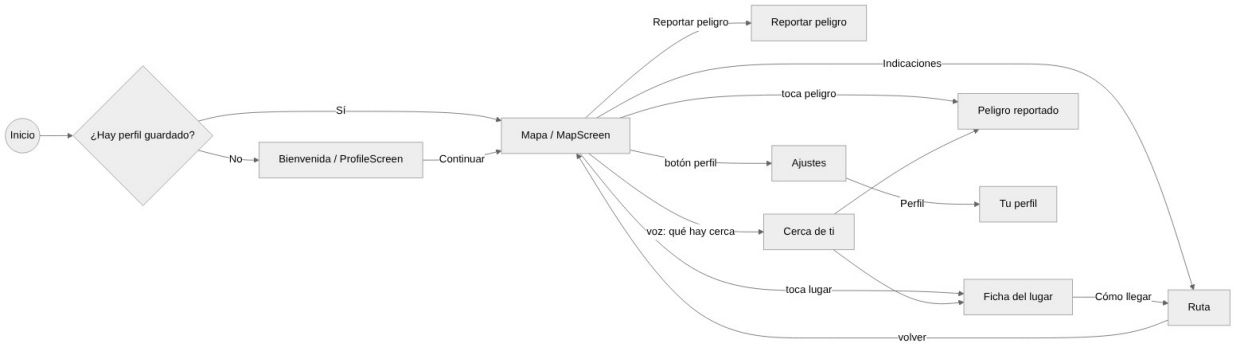


Figura 6. Navegación entre pantallas.

La navegación usa el `Navigator` estándar con `MaterialPageRoute`. No se usan rutas con nombre porque no hay enlaces profundos ni navegación web por URL en el MVP. La única sustitución de pantalla (`pushReplacement`) ocurre al terminar la bienvenida, para que el botón Atrás no regrese a ella.

10.3 Escala de texto

En `MaterialApp.builder` se calcula la escala efectiva como `min(max(escalaDelSistema, escalaDeAjustes), 2.0)`. Así la app nunca muestra letras más pequeñas que las que la persona eligió en el sistema operativo, puede agrandarse más desde Ajustes y no rebasa el 200 %, punto en el que algunos diseños empiezan a desbordarse.

11. Sistema de diseño

La interfaz se rediseñó a partir de una maqueta con seis pantallas de referencia, con dos objetivos: que fuera agradable para todos los perfiles (no solo para baja visión) y que conservara un contraste alto en cualquier modo. El resultado es un sistema de *tokens* de color, una tipografía muy legible y un conjunto de componentes compartidos.

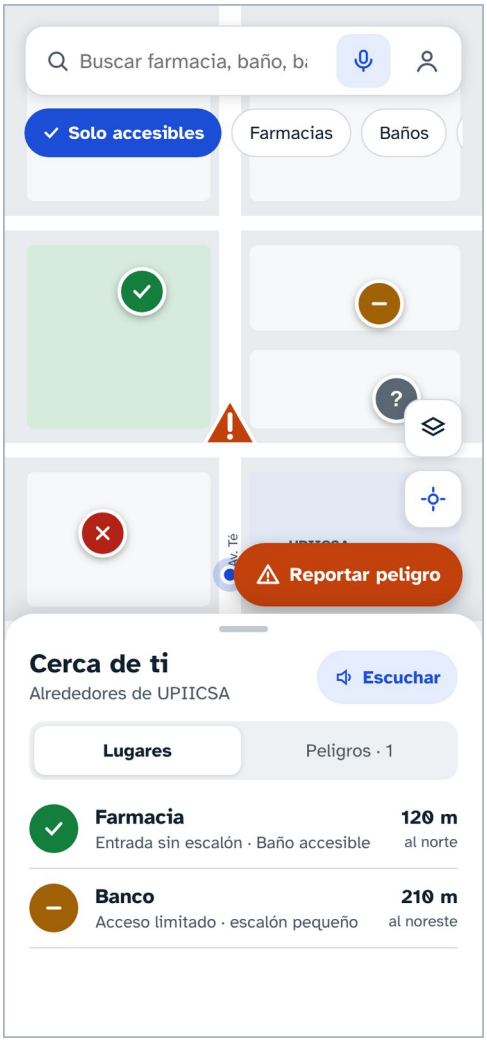


Figura 7. Mapa en modo claro (maqueta).



Figura 8. Mapa en modo oscuro (maqueta).

11.1 Tokens de color

Los colores se definen en la extensión de tema `RLColors` (archivo `lib/core/theme.dart`) y se leen en cualquier *widget* con `context.rl`. Ninguna pantalla usa colores literales salvo los de estado, que son iguales en ambos modos.

Token	Claro	Oscuro	Uso
`background`	#F4F6F8	#0F151B	Fondo de las pantallas.
`surface`	#FFFFFF	#161D25	Tarjetas, hojas, barra de búsqueda.
`surface2`	#EEF1F4	#1F2832	Pista de controles segmentados, fondos secundarios.
`border`	#D5DBE1	#2C3742	Bordes y divisores.
`handle`	#C3CAD2	#3A4652	Barrita para arrastrar hojas.
`text`	#10202E	#E6EDF3	Texto principal.
`text2`	#4B5966	#A9B6C2	Texto secundario.
`primary`	#1D4ED8	#8AB4FF	Acción principal, enlaces, selección.
`primarySoft`	#E6EDFC	#1E2D4A	Fondo de botones secundarios (Escuchar, micrófono).
`onPrimary`	#FFFFFF	#0B1B33	Texto sobre `primary`.
`okSoft` / `okText`	#E7F4EC / #0F5B2C	#12301F / #8EE0AE	Tarjeta «Accesible», etiqueta «Sin escaleras».
`hazardSoft` / `hazardText`	#FDEEE6 / #8A2C07	#3A1D10 / #FFB48F	Tipo de peligro seleccionado, «Evita N peligros».

Colores de estado (iguales en ambos modos)

Constante	Color	Uso
`AppTheme.accessYes`	#157F3D	Lugar accesible.
`AppTheme.accessLimited`	#A16207	Acceso limitado; peligros de gravedad baja.
`AppTheme.accessNo`	#B42318	No accesible; errores.
`AppTheme.accessUnknown`	#5B6773	Sin dato.
`AppTheme.hazard`	#C2410C	Peligros de gravedad media y alta; botón Reportar.
`AppTheme.userDot`	#1D4ED8	Punto de «tú estás aquí».

11.2 Tipografía

Se usa **Atkinson Hyperlegible** (regular 400 y negrita 700), incluida en `assets/fonts` y declarada en `pubspec.yaml`. La escala tipográfica del tema es:

Estilo	Tamaño	Peso	Uso
`displaySmall`	32	Negrita	Título de bienvenida.
`headlineMedium`	28	Negrita	Nombre del lugar en su ficha.
`headlineSmall`	22	Negrita	Títulos de hoja («Cerca de ti») y de reporte.
`titleLarge`	19	Negrita	Subtítulos.
`titleMedium`	17	Negrita	Elementos de listas.
`bodyLarge`	17	Regular	Texto de lectura.
`bodyMedium`	15	Regular	Texto secundario.
`labelLarge`	17	Negrita	Botones.

Todos los tamaños se multiplican por la escala de texto efectiva (1.2 por defecto).

11.3 Verificación de contraste

Los cocientes de contraste se calcularon con la fórmula de luminancia relativa de WCAG 2.x: (L1 + 0.05) / (L2 + 0.05). El mínimo para texto normal (AA) es 4.5:1 y para texto grande 3:1. **Todas las combinaciones superan 4.5:1.**

Modo	Combinación	Primer plano	Fondo	Contraste	Resultado
Claro	Texto sobre superficie	#10202E	#FFFFFF	16.56:1	AAA
Claro	Texto secundario sobre superficie	#4B5966	#FFFFFF	7.19:1	AAA
Claro	Texto secundario sobre fondo	#4B5966	#F4F6F8	6.64:1	AA
Claro	Primario sobre superficie	#1D4ED8	#FFFFFF	6.70:1	AA
Claro	Blanco sobre primario	#FFFFFF	#1D4ED8	6.70:1	AA
Claro	Primario sobre primario suave	#1D4ED8	#E6EDFC	5.71:1	AA
Claro	Texto ok sobre ok suave	#0F5B2C	#E7F4EC	7.27:1	AAA
Claro	Texto peligro sobre peligro suave	#8A2C07	#FDEEE6	7.59:1	AAA
Oscuro	Texto sobre superficie	#E6EDF3	#161D25	14.37:1	AAA
Oscuro	Texto secundario sobre superficie	#A9B6C2	#161D25	8.21:1	AAA
Oscuro	Primario sobre superficie	#8AB4FF	#161D25	8.13:1	AAA
Oscuro	Texto sobre primario	#0B1B33	#8AB4FF	8.25:1	AAA
Oscuro	Primario sobre primario suave	#8AB4FF	#1E2D4A	6.57:1	AA
Oscuro	Texto ok sobre ok suave	#8EE0AE	#12301F	9.14:1	AAA

Modo	Combinación	Primer plano	Fondo	Contraste	Resultado
Oscuro	Texto peligro sobre peligro suave	#FFB48F	#3A1D10	8.93:1	AAA
Ambos	Blanco sobre Accesible	#FFFFFF	#157F3D	5.08:1	AA
Ambos	Blanco sobre Acceso limitado	#FFFFFF	#A16207	4.92:1	AA
Ambos	Blanco sobre No accesible	#FFFFFF	#B42318	6.57:1	AA
Ambos	Blanco sobre Sin dato	#FFFFFF	#5B6773	5.78:1	AA
Ambos	Blanco sobre Peligro	#FFFFFF	#C2410C	5.18:1	AA

11.4 Formas, espaciado y elevación

- Radios: 24 en hojas inferiores, 16 en tarjetas y botones principales, 14 en botones flotantes y campos, 10–12 en elementos pequeños; píldoras con borde de estadio.
- Márgenes laterales de 16 a 24 puntos; separación entre secciones de 12 a 24.
- Sombras suaves (opacidad 12 % a 14 %) solo en elementos que flotan sobre el mapa.
- Tamaño mínimo de controles: 44 × 44 puntos; botones principales de 56 de alto.

11.5 Componentes compartidos (widgets/ui.dart)

Componente	Descripción	Accesibilidad
`SheetHandle`	Barrita de 40 × 5 para arrastrar hojas.	Excluida del lector de pantalla.
`MapIconButton`	Botón cuadrado flotante de 48 × 48 con borde y sombra.	Botón con etiqueta igual al tooltip.
`FilterPill`	Píldora de filtro; activa en color primario con palomita.	«Filtro: X», con estado seleccionado.
`Segmented<T>`	Control segmentado genérico (Lugares | Peligros, Baja | Media | Alta, Sistema | Claro | Oscuro).	Cada segmento es botón con `selected`.
`Tag`	Etiqueta de color suave con ícono opcional.	Texto legible.
`AccessBadge`	Círculo de estado con ícono; cuadrado para cruces; aro blanco en el mapa.	La etiqueta la pone el contenedor.
`HazardTriangle`	Triángulo de peligro pintado con `CustomPainter`.	Ídem.
`PlaceRow` / `HazardRow`	Fila de lista con insignia, nombre, resumen, distancia y dirección.	Toda la fila es un botón con la descripción hablada completa.
`ListenButton`	Botón «Escuchar» (con texto o solo ícono).	Tooltip «Escuchar».

Componente	Descripción	Accesibilidad
`EmptyNote`	Mensaje centrado de lista vacía.	Texto simple.
`OsmCredit`	Crédito «© OpenStreetMap» que abre la página de derechos.	Enlace con etiqueta completa.

11.6 Mapa en modo oscuro

Las teselas de OSM son claras. En modo oscuro se aplica `darkModeTileBuilder` de `flutter_map`, que invierte y ajusta los colores de cada tesela con una matriz de color. Así el mapa no deslumbra de noche y los marcadores, con su aro blanco, siguen resaltando.

12. Pantallas

Esta sección describe cada pantalla: su propósito, su estructura, el estado que maneja y sus interacciones. Las figuras corresponden a la maqueta de diseño a partir de la cual se implementó cada pantalla; pueden existir pequeñas diferencias de texto con la aplicación compilada.

12.1 Bienvenida y perfil (ProfileScreen)



Figura 9. Bienvenida (primera vez).



Figura 10. Cambio de perfil desde Ajustes.

Aspecto	Detalle
Archivo	`lib/features/onboarding/profile_screen.dart`
Parámetro	`firstTime` (`true` al abrir la app sin perfil).
Estado local	`_selected` (perfil marcado) y `_saving`.

Aspecto	Detalle
Estructura	Ícono y título (solo primera vez), texto explicativo, cuatro tarjetas de opción, botón fijo inferior y crédito de OSM.
Interacciones	Tocar una tarjeta la marca y dice su nombre. **Continuar** (o **Guardar**) se habilita al haber selección.
Al confirmar	Primera vez: `chooseProfile` y reemplazo por el mapa. Después: `updateSettings` y regresar.
Semántica	Tarjetas con `button`, `selected` e `inMutuallyExclusiveGroup`; etiqueta «Perfil. Descripción».
Voz	Al abrir lee la bienvenida y las cuatro opciones (siempre, aunque la guía esté apagada).

12.2 Mapa (MapScreen)

Es la pantalla principal y la más compleja (971 líneas). Se compone de cuatro capas en un `Stack`:

1. **Mapa** (`FlutterMap`) a pantalla completa con teselas, línea de ruta y marcadores.
2. **Controles flotantes**: crédito de OSM a la izquierda; leyenda, centrar y **Reportar peligro** a la derecha. Suben y bajan con la hoja y se ocultan cuando la hoja pasa del 62 % de la altura.
3. **Hoja «Cerca de ti»** (`DraggableScrollableSheet`) con mínimo de unos 190 puntos, punto intermedio de 36 % y máximo de 92 %.
4. **Barra superior**: búsqueda con micrófono y perfil; debajo, filtros o la tarjeta de ruta activa, y avisos de estado.



Figura 11. Búsqueda con filtro Farmacias.

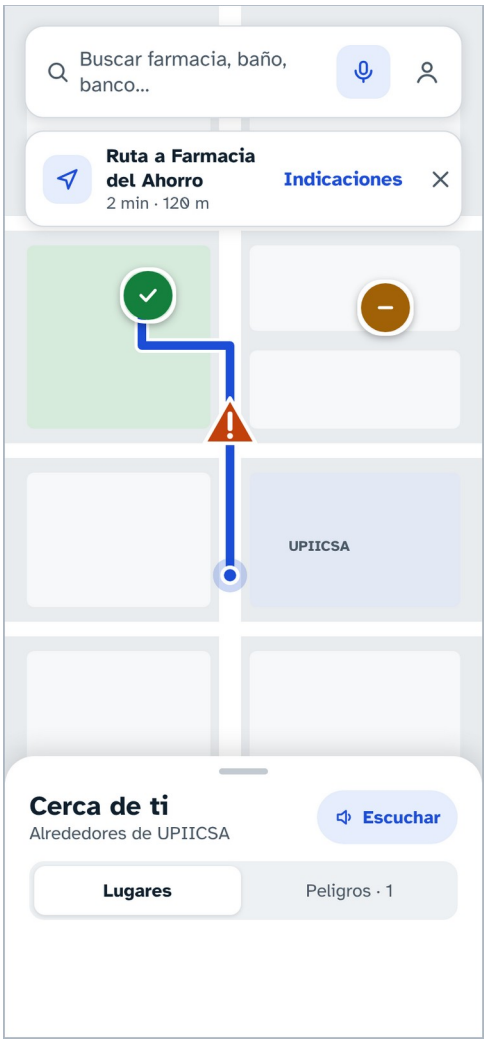


Figura 12. Ruta activa en el mapa.

Estado local	Uso
`_showCrossings`	Los cruces solo se muestran con zoom ≥ 17 para no saturar el mapa.
`_followMe`	El mapa sigue a la persona hasta que lo arrastra con el dedo.
`_listening`	El micrófono está escuchando.
`_showLoadHere`	La cámara está fuera de toda zona descargada (más de 80 % del radio).
`_showHazards`	Pestaña activa de la hoja: Lugares o Peligros.
`_filter`	Filtro de categoría (Farmacias, Baños, Transporte) o ninguno.
`_query`	Texto de búsqueda.
`_sheetSize`	`ValueNotifier` con la altura de la hoja, para posicionar los controles sin reconstruir el mapa.

Filtros de categoría

Filtro	Criterio
Farmacias	Categoría «Farmacia» (incluye `shop=chemist`).
Baños	Categoría «Baños públicos» o cualquier lugar con baño accesible.
Transporte	Parada de autobús, estación o elevador.

Con un filtro o una búsqueda activos, se ocultan los cruces y la hoja muestra «Resultados» con el filtro y el texto buscado como subtítulo.

Optimización del redibujado

`onPositionChanged` se llama en cada cuadro mientras se arrastra el mapa. Llamar a `setState` en cada uno reconstruiría cientos de marcadores sesenta veces por segundo. Por eso solo se llama cuando cambia algo visible: si se cruza el zoom de cruces, si se sale o entra a una zona descargada, o si se deja de seguir a la persona. La posición de los controles flotantes depende de `_sheetSize` mediante un `ValueListenableBuilder`, que solo reconstruye esos controles.

12.3 Ficha del lugar (PlaceScreen)



Figura 13. Ficha de un lugar accesible.

Zona	Contenido
Mapa superior (260 pt)	Mapa fijo sin interacción, con el lugar al centro (insignia de 52 con halo) y la posición de la persona.
Encabezado	Nombre, «categoría · distancia y dirección · tiempo a pie» y botón de escuchar.
Tarjeta de estado	Insignia, etiqueta y explicación con el color suave del estado.
Datos	Cuadrícula de dos columnas: Baño accesible (solo lugares) y Guía podotáctil, con «Sí», «No» o «Sin dato».
Nota	`wheelchair:description`, si existe.

Zona	Contenido
Acciones	**Cómo llegar** (abre la pantalla de ruta), «¿Es correcta esta información? Sí / No», enlace «Ver o corregir en OpenStreetMap».

El tiempo a pie se estima con una velocidad tranquila de 1 m/s (`walkingSpeed`), adecuada para silla de ruedas y bastón. La ficha se dice completa en voz alta al abrirse si la guía por voz está activa.

12.4 Ruta (RouteScreen)

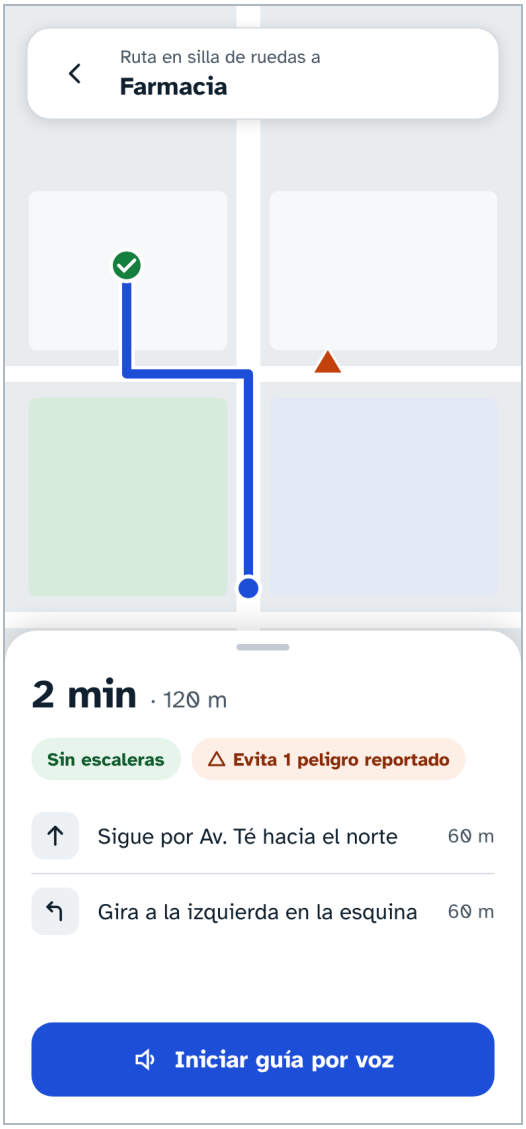


Figura 14. Ruta accesible con indicaciones.

Muestra la ruta sobre un mapa a pantalla completa, ajustado para que toda la ruta quepa entre la tarjeta superior y la hoja (`CameraFit.bounds` con relleno inferior de la mitad de la pantalla). La

hoja contiene tiempo y distancia, etiquetas «Sin escaleras» (si el perfil evita escalones) y «Evita N peligros reportados», los pasos con ícono y distancia y un aviso sobre la calidad de los datos. El botón fijo **Iniciar guía por voz** lee el resumen y todos los pasos; tocar un paso lo lee individualmente.

Estado	Qué muestra la hoja
Calculando	Indicador de progreso con `liveRegion` y «Buscando un camino sin obstáculos…».
Error	Ícono, mensaje del servicio y **Intentar otra vez** (si hay destino).
Sin ruta	«No hay una ruta activa.» (al abrir Indicaciones sin ruta).
Ruta	Resumen, etiquetas, pasos y botón de guía.

12.5 Reportar peligro (NewReportScreen)



Figura 15. Reportar un peligro.

Zona	Contenido
Barra	Botón cerrar (✕) y título.
Ubicación	Tarjeta «Se reportará donde estás · Ubicación por GPS · precisión N m», o «Buscando tu ubicación…» en color de advertencia.
¿Qué encontraste?	Ocho opciones en dos columnas con ícono; la elegida en naranja con borde de 2 pt.
Gravedad	Control segmentado Baja / Media / Alta; al elegir un tipo se propone su gravedad por defecto.
Opcional	Agregar foto (cámara del sistema), aviso sobre rostros y placas, nota de hasta 140 caracteres.

Zona	Contenido
Pie fijo	Botón naranja **Enviar reporte** (deshabilitado sin tipo o sin GPS) y «Caduca en N si nadie confirma que sigue ahí».

Al enviar se vibra fuerte, se agradece por voz y se regresa al mapa, donde el peligro ya aparece. La foto se copia a la carpeta privada de la app porque la cámara la guarda en una caché temporal que el sistema puede borrar.

12.6 Peligro reportado (ReportScreen)

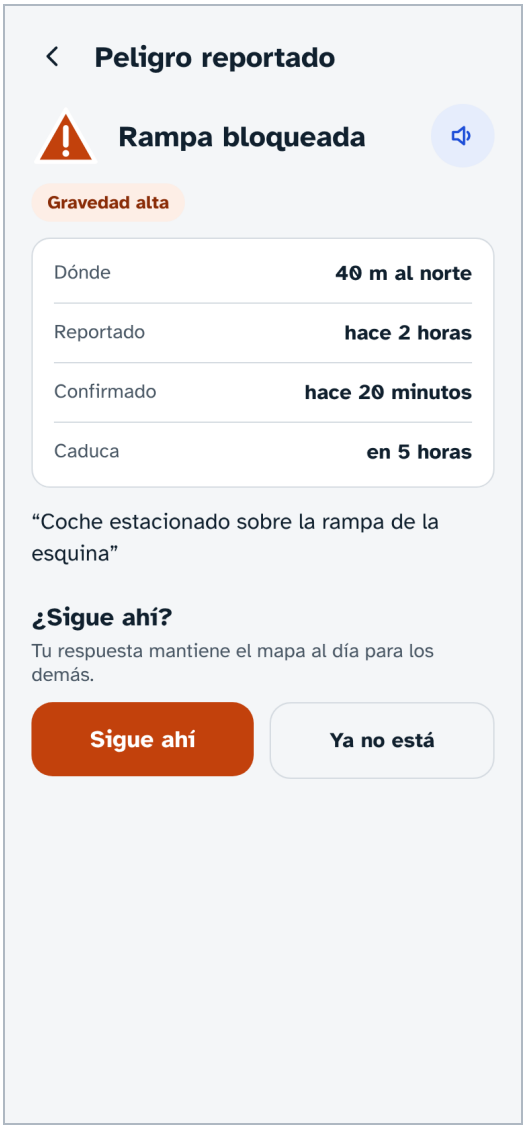


Figura 16. Detalle de un peligro.

Muestra el triángulo, el tipo, la gravedad, una tabla con dónde está, cuándo se reportó, cuándo se confirmó y cuándo caduca, la nota y la foto. Si está activo, pregunta «¿Sigue ahí?» con **Sigue ahí**

(renueva la vigencia) y **Ya no está** (suma un voto). Al responder se muestra y se dice un agradecimiento. Si el reporte ya no existe (por limpieza) se muestra «Este reporte ya no existe».

12.7 Cerca de ti (NearbyScreen)

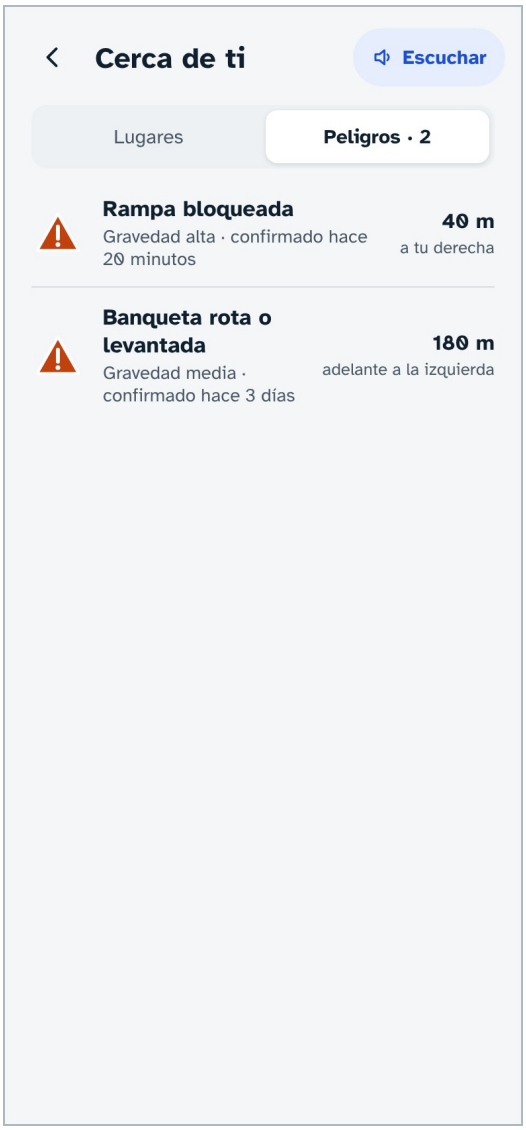


Figura 17. Cerca de ti, pestaña Peligros.

Versión de pantalla completa de la hoja del mapa. Se abre con el comando de voz «qué hay cerca», porque cambiar de pantalla es más fácil de seguir con lector de pantalla que una hoja que se expande. Incluye pestañas, campo de búsqueda y botón Escuchar. Se reconstruye en cada posición nueva para mantener vigentes las distancias.

12.8 Ajustes (SettingsScreen)

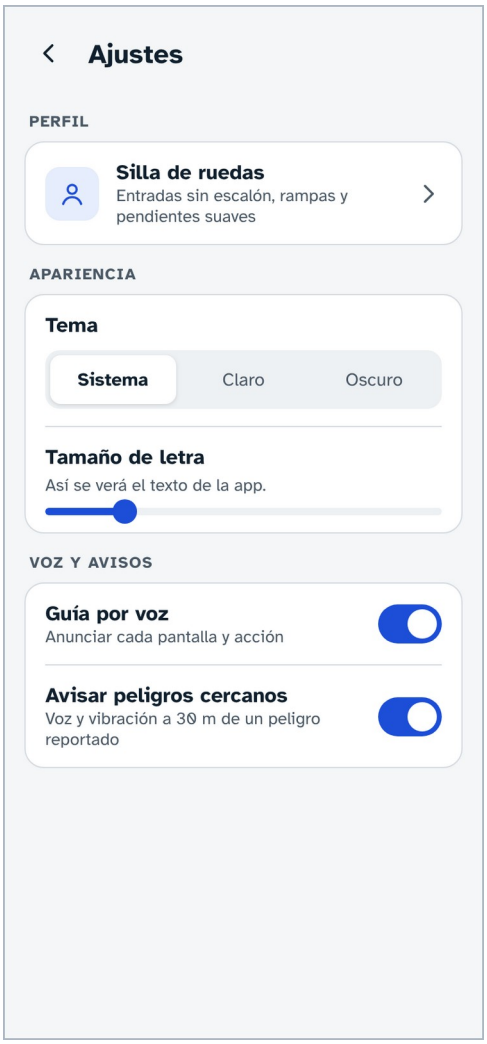


Figura 18. Ajustes (parte superior).

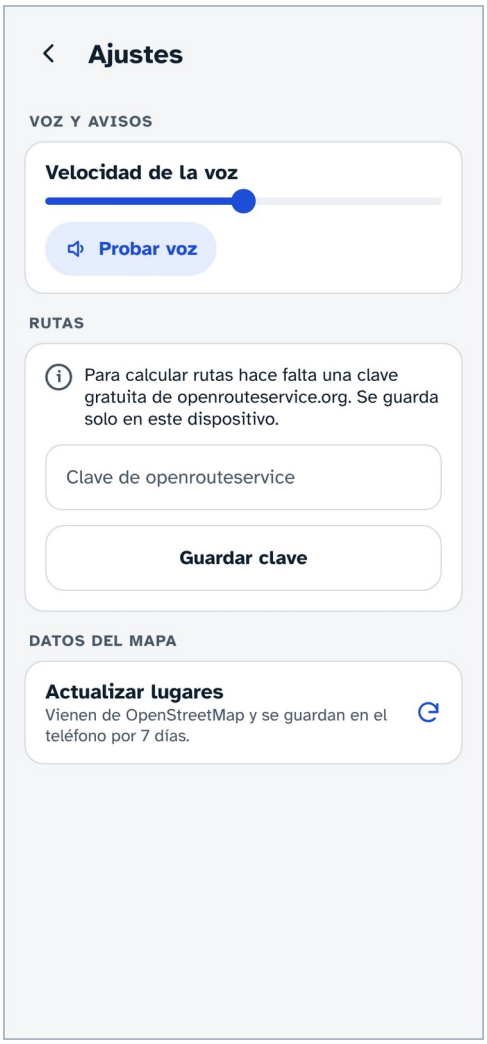


Figura 19. Ajustes (parte inferior).

Sección	Controles
Perfil	Fila con el perfil actual que abre **Tu perfil**.
Apariencia	Tema Sistema / Claro / Oscuro; tamaño de letra de 100 % a 200 % en 5 pasos.
Voz y avisos	Guía por voz; Avisar peligros cercanos; velocidad de la voz (lenta, normal, rápida); Probar voz.
Rutas	Estado de la clave, campo oculto con botón para mostrarla, Guardar clave.
Datos del mapa	Actualizar lugares (fuerza la descarga de la zona piloto).
Privacidad	Explicación del uso de la ubicación y los reportes.

12.9 Leyenda



Figura 20. Leyenda de marcas.

Hoja modal (`showModalBottomSheet`) con el significado de cada insignia, el triángulo de peligro y la diferencia entre cruces (cuadrados) y lugares (círculos). Cada fila es un `MergeSemantics` para que el lector la lea completa.

13. Accesibilidad implementada

La accesibilidad no es una capa añadida sino un criterio que atraviesa todas las decisiones. Esta sección resume cómo se implementó en cada frente.

13.1 Lector de pantalla (TalkBack, VoiceOver, NVDA)

- Todo control propio se envuelve en `Semantics(button: true, label: …, excludeSemantics: true)` para que se lea como un solo elemento con una etiqueta clara, en lugar de leer por separado ícono y texto.
- Los marcadores del mapa tienen como etiqueta la descripción hablada completa: «Farmacia Ejemplo, farmacia. Acceso limitado: escalón pequeño o acceso parcial. A 40 metros, al norte.»
- Las filas de las listas usan la misma descripción, de modo que la lista es equivalente al mapa.
- Controles de opción (perfiles, tipos de reporte, segmentos, filtros) exponen `selected`; los perfiles y tipos además `inMutuallyExclusiveGroup`.
- Títulos de sección marcados con `header: true` para navegar por encabezados.
- Mensajes de estado con `liveRegion: true` («Descargando lugares…», «Calculando ruta», «Gracias por tu ayuda»), que el lector anuncia sin mover el foco.
- Elementos decorativos (barritas, mapas de las fichas) excluidos con `ExcludeSemantics`.
- Los controles flotantes ocultos cuando la hoja está expandida también se excluyen de la semántica.

13.2 Voz propia de la aplicación

Además del lector de pantalla, la app habla por sí misma, lo que la hace útil para personas con baja visión que no usan lector y para quien prefiere no mirar el teléfono en la calle:

Momento	Qué dice	Depende de la guía por voz
Bienvenida	Saludo y opciones de perfil.	No (siempre)
Abrir el mapa	Cuántos lugares accesibles y peligros hay.	Sí
Abrir una ficha o un peligro	Descripción completa.	Sí
Filtros, centrar, descargar zona	Confirmación breve.	Sí
Escuchar	Los 5 lugares o 3 peligros más cercanos.	No (lo pidió la persona)

Momento	Qué dice	Depende de la guía por voz
Ruta calculada	Resumen.	No
Error de red, ubicación o ruta	El mensaje de error.	No
Aviso de peligro	«Atención. …»	No (solo «Avisar peligros cercanos»)
Enviar reporte, confirmar, descartar	Agradecimiento.	No

13.3 Visión

- Contraste AA o superior en todas las combinaciones (sección 11.3).
- Tipografía Atkinson Hyperlegible.
- Escala de texto hasta 200 %, nunca menor que la del sistema.
- Estados distinguibles sin color: ícono distinto y forma (círculo, cuadrado, triángulo).
- Modo oscuro para reducir el deslumbramiento, incluido el mapa.

13.4 Motricidad

- Objetivos táctiles de 44 a 56 puntos.
- Reportar un peligro en tres toques: tipo, (gravedad), enviar.
- Botones principales en la parte inferior, al alcance del pulgar.
- Comandos de voz para las acciones principales.
- Ninguna acción requiere gestos complejos; arrastrar la hoja es opcional porque su contenido también se recorre con desplazamiento.

13.5 Retroalimentación háptica

Acción	Vibración
Elegir un perfil o un tipo de reporte	`selectionClick`
Confirmar el perfil; empezar a escuchar	`mediumImpact`
Enviar un reporte; aviso de peligro	`heavyImpact`

14. Integración con servicios externos

14.1 Teselas de OpenStreetMap

El mapa de fondo se obtiene de `https://tile.openstreetmap.org/{z}/{x}/{y}.png`, el servidor de teselas de la OpenStreetMap Foundation. Su uso es gratuito pero está sujeto a una política que la aplicación cumple de la siguiente manera:

Requisito de la política	Cumplimiento en Ruta Libre
Identificar la aplicación con un User-Agent válido	`userAgentPackageName: com.drak028.ruta_libre` en `TileLayer`; `flutter_map` arma un User-Agent con ese identificador.
Mostrar la atribución de forma visible	Crédito «© OpenStreetMap» siempre visible sobre el mapa, que abre la página de derechos; también en la bienvenida.
No descargar teselas en masa ni para uso sin conexión	Solo se piden las teselas visibles; no hay precarga ni descarga de zonas.
Respetar la caché HTTP	La caché la gestiona `flutter_map` y el sistema.
Zoom máximo 19	`maxZoom: 19`.

> **Importante:** Si la aplicación crece a muchas personas usuarias, la política obliga a cambiar a un proveedor comercial de teselas (por ejemplo MapTiler, Stadia Maps o un servidor propio). El cambio se hace en un solo lugar: la función `osmTiles` en `lib/widgets/base_map.dart`.

14.2 Overpass API

Aspecto	Valor
URL	`https://overpass-api.de/api/interpreter` (`AppConfig.overpassUrl`)
Método	POST con formulario `data=<consulta>`
Formato	`[out:json][timeout:25]`
Tiempo límite del cliente	40 segundos
Frecuencia	Una consulta por zona cada 7 días como máximo (o al forzar desde Ajustes, o al pedir una zona nueva).
Tamaño típico de respuesta	Entre 100 KB y 1 MB para 1 km de radio en zona urbana.

14.3 openrouteservice

Se usa el servicio de rutas v2 con salida GeoJSON. Para obtener una clave:

1. Crear una cuenta gratuita en openrouteservice.org.
2. En el panel (Dashboard), solicitar un token del plan estándar (Standard).
3. Copiar la clave y escribirla en **Ajustes** → **Rutas** de la app, o pasarla al compilar con `--dart-define=ORS_API_KEY=...`.

Perfil de la app	Perfil ORS	Opciones adicionales
Silla de ruedas	`wheelchair`	`avoid_features: [steps]`; pendiente máxima 6 %; borde máximo 0.06 m; superficie «good».
Bastón o andadera	`foot-walking`	`avoid_features: [steps]`.
Baja visión	`foot-walking`	Ninguna.
Ceguera	`foot-walking`	Ninguna.

En todos los perfiles se agregan como `avoid_polygons` los peligros activos de gravedad media o alta, excepto los que están a 25 m o menos del origen o destino.

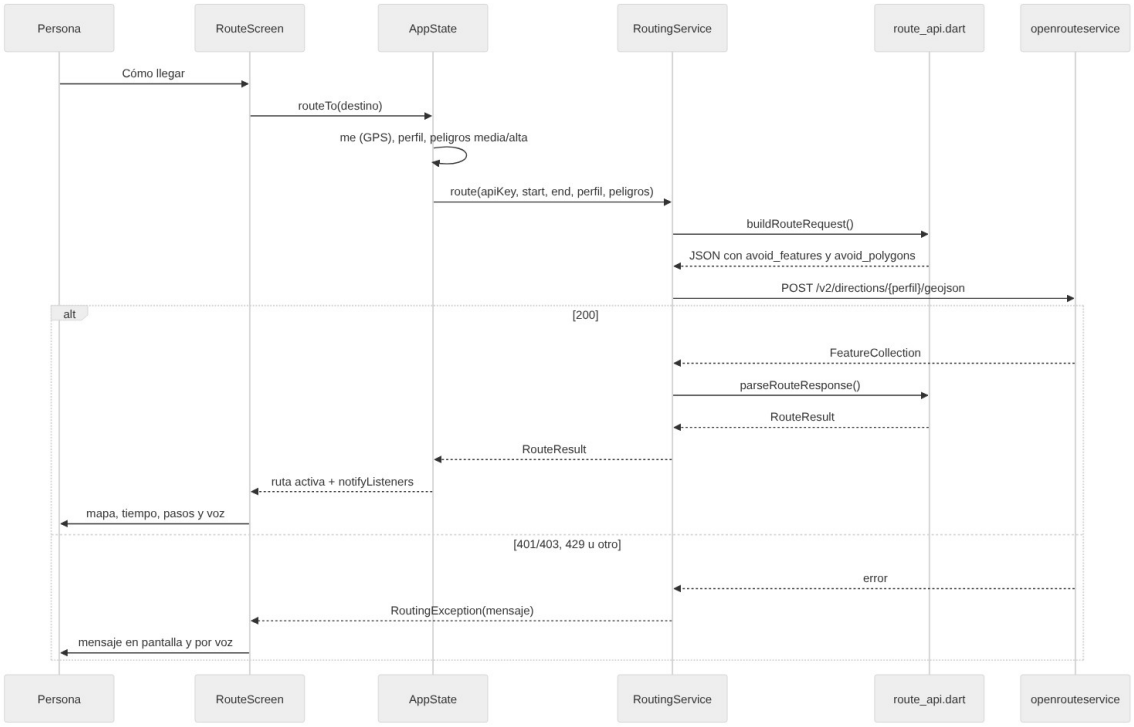


Figura 21. Secuencia del cálculo de una ruta.

Consumo esperado

Con 2,000 rutas diarias en el plan gratuito, y suponiendo un promedio de cinco rutas por persona al día, el plan alcanza para unas 400 personas activas diarias, más que suficiente para el piloto. Para crecer, ORS ofrece planes de colaboración para proyectos sin fines de lucro, o puede instalarse un servidor propio con su imagen de Docker.

15. Seguridad y privacidad

15.1 Datos personales que maneja la aplicación

Dato	¿Se guarda?	¿Sale del teléfono?	Detalle
Ubicación en tiempo real	No (solo en memoria)	Solo origen y destino al pedir una ruta, a openrouteservice	Nunca se guarda ni se envía el recorrido.
Ubicación de un peligro reportado	Sí, local	No en el MVP	Cuando exista servidor, se enviará solo la ubicación del peligro.
Fotos de peligros	Sí, carpeta privada	No en el MVP	La pantalla pide evitar rostros y placas.
Perfil de movilidad	Sí, local	No	Es un dato sensible (relacionado con discapacidad); no se comparte.
Clave de openrouteservice	Sí, local	Solo a openrouteservice	Se muestra oculta en Ajustes.
Voz (micrófono)	No	Según el motor del sistema	El reconocimiento lo hace el servicio de voz del sistema operativo.

15.2 Permisos

Los permisos se agregan a las carpetas nativas con `tool/patch_platforms.dart`.

Plataforma	Permiso	Para qué
Android	`INTERNET`	Mapa, lugares y rutas.
Android	`ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION`	Ubicación para distancias, avisos y reportes.
Android	`RECORD_AUDIO`	Comandos de voz.
Android	Consultas de intents (TTS, reconocimiento, cámara, https)	Encontrar los servicios del sistema en Android 11+.
iOS	`NSLocationWhenInUseUsageDescription`	Ubicación mientras se usa.
iOS	`NSCameraUsageDescription`, `NSPhotoLibraryUsageDescription`	Foto del peligro.
iOS	`NSMicrophoneUsageDescription`, `NSSpeechRecognitionUsageDescription`	Comandos de voz.

> **Nota:** No se declara `CAMERA` en Android a propósito: `image_picker` usa la app de cámara del sistema mediante un intent, y declarar el permiso obligaría a solicitarlo en tiempo de ejecución sin necesidad.

15.3 Manejo de secretos

- La clave de openrouteservice **no** está en el código. Se lee de `String.fromEnvironment('ORS_API_KEY')` (definida al compilar) o de los ajustes guardados.
- La versión web publicada en GitHub Pages se compila **sin** clave, para no exponerla en el JavaScript público; cada persona escribe la suya en Ajustes.
- Los archivos de firma (`*.jks`, `key.properties`) están en `.gitignore`.

15.4 Firma de la aplicación

La plantilla de Flutter firma la compilación `release` con la llave de depuración. Esto basta para instalar el APK de pruebas en teléfonos, pero no para publicar en Google Play. Para publicar:

```
keytool -genkey -v -keystore ruta-libre.jks -keyalg RSA -keysize 2048 \
        -validity 10000 -alias ruta-libre

# android/key.properties (no se sube al repositorio)
storePassword=...
keyPassword=...
keyAlias=ruta-libre
storeFile=C:/ruta/segura/ruta-libre.jks
```

y configurar `signingConfigs.release` en `android/app/build.gradle.kts` para leer ese archivo. Después se compila con `flutter build appbundle --release`.

16. Entorno de desarrollo

16.1 Requisitos del equipo

Recurso	Mínimo	Recomendado
Sistema operativo	Windows 10 de 64 bits	Windows 11 (probado en 25H2)
Memoria RAM	8 GB	16 GB
Espacio en disco	15 GB libres	30 GB libres (SDK, NDK, emulador)
Procesador	4 núcleos	8 núcleos
Conexión	Necesaria para instalar y para ejecutar	—
Teléfono de pruebas	Android 7.0 con GPS	Android 12 o superior

16.2 Instalación automatizada

La carpeta `setup/` contiene scripts que automatizan la instalación en Windows. Los archivos `.bat` solo abren el `.ps1` correspondiente con la política de ejecución adecuada, así que basta con hacer doble clic en el `.bat`.

Script	Qué hace	Salida
`instalar_entorno.bat`	Instala Git (winget), descarga y descomprime Flutter estable en `C:\src\flutter` verificando su SHA-256, configura el Android SDK, instala command-line tools y platform-tools, acepta licencias, genera las carpetas nativas, aplica permisos, `pub get`, análisis y pruebas.	`setup/instalacion.log`, `setup/estado.txt`
`preparar_android.bat`	Crea el enlace `C:\src\android-sdk` al SDK (sin espacios), configura Flutter, acepta licencias y compila un APK de depuración.	`setup/salida/*.txt`
`instalar_ndk.bat`	Descarga e instala el NDK 28.2.13676358 desde el repositorio oficial de Google verificando SHA-1, y recompila el APK.	`setup/salida/ndk_estado.txt`
`revisar.bat`	`pub get`, análisis, pruebas, `flutter doctor` y dispositivos.	`setup/salida/analyze.txt`, `test.txt`, `doctor.txt`
`generar_apk.bat`	Análisis, pruebas y, si el análisis pasa, APK release copiado a `C:\src\ruta_libre\RutaLibre.apk`.	`setup/salida/apk_estado.txt`
`ejecutar_web.bat`	`flutter run -d chrome --web-port 8080` (o Edge si no hay Chrome).	Navegador en localhost:8080

Script	Qué hace	Salida
`ejecutar_android.bat`	Lista dispositivos y ejecuta en el teléfono conectado por USB.	Consola
`abrir_vscode.bat`	Instala la extensión de Flutter y abre el proyecto en VS Code.	—

16.3 Instalación manual paso a paso

Si se prefiere instalar a mano, o en macOS/Linux, el procedimiento equivalente es:

1. Instalar Git.
2. Descargar Flutter estable desde docs.flutter.dev y descomprimirlo en una ruta **sin espacios** (por ejemplo `C:\src\flutter`).
3. Agregar `C:\src\flutter\bin` a la variable PATH del usuario.
4. Instalar Android Studio y, desde **More Actions → SDK Manager**, instalar Android SDK Platform, Command-line Tools y Platform-Tools.
5. Ejecutar `flutter doctor --android-licenses` y aceptar todas.
6. Clonar el repositorio en una ruta sin espacios.
7. Generar carpetas nativas: `flutter create --org com.drak028 --project-name ruta_libre --platforms android,web .`
8. Aplicar permisos: `dart run tool/patch_platforms.dart`.
9. Descargar dependencias: `flutter pub get`.
10. Verificar: `flutter analyze` y `flutter test`.
11. Ejecutar: `flutter run` (teléfono) o `flutter run -d chrome --web-port 8080`.

16.4 Problemas conocidos de instalación en Windows

Ruta del SDK con espacios

Cuando el nombre de la cuenta de Windows contiene espacios (por ejemplo `C:\Users\Asus Vivobook`), el SDK de Android queda en una ruta con espacios y algunas herramientas del NDK fallan. La solución, aplicada por `preparar_android.ps1`, es crear un enlace de directorio sin espacios que apunte al mismo SDK, sin copiar nada ni requerir permisos de administrador:

```
mklink /J "C:\src\android-sdk" "%LOCALAPPDATA%\Android\Sdk"
flutter config --android-sdk="C:\src\android-sdk"
setx ANDROID_HOME C:\src\android-sdk
```

El NDK no se instala solo

Al compilar por primera vez, Gradle intenta instalar el NDK que pide Flutter y el `sdkmanager` reciente puede responder «Package ndk not found». `instalar_ndk.ps1` lee el catálogo oficial

`repository2-3.xml`, descarga el paquete para Windows, verifica su SHA-1, lo descomprime en `ndk\28.2.13676358` y escribe el `package.xml` que el SDK necesita para reconocerlo.

Las salidas de PowerShell no se guardan

`Start-Transcript` no captura la salida de programas nativos como `flutter`. Por eso los scripts redirigen cada comando a su propio archivo en `setup/salida/` con `*>`. Esos archivos se guardan en UTF-16; para leerlos desde otra herramienta conviene convertirlos (por ejemplo, `iconv -f utf-16 -t utf-8`).

16.5 Configuración de Visual Studio Code

```
// .vscode/launch.json
{
  "configurations": [
    { "name": "Ruta Libre (Chrome)",  "request": "launch", "type": "dart",
      "deviceId": "chrome", "args": ["--web-port", "8080"] },
    { "name": "Ruta Libre (Android)", "request": "launch", "type": "dart" }
  ]
}
```

Con estas configuraciones, **F5** ejecuta la app con recarga en caliente (*hot reload*): al guardar un archivo, la interfaz se actualiza en uno o dos segundos sin perder el estado.

17. Compilación y despliegue

17.1 Comandos de compilación

Objetivo	Comando	Resultado
APK de depuración	`flutter build apk --debug`	`build/app/outputs/flutter-apk/app-debug.apk` (~166 MB, incluye depurador).
APK de pruebas (release)	`flutter build apk --release`	`app-release.apk` (~54 MB); firmado con llave de depuración.
APK por arquitectura	`flutter build apk --release --split-per-abi`	Tres APK de ~20 MB (arm64-v8a, armeabi-v7a, x86_64).
Paquete para Google Play	`flutter build appbundle --release`	`.aab`; requiere firma propia (15.4).
Web	`flutter build web --release --base-href /ruta_libre/`	`build/web`, listo para publicar en un servidor estático.
Con clave de rutas	agregar `--dart-define=ORS_API_KEY=clave`	La clave queda compilada (no usar en web pública).

> **Consejo:** Para compartir el APK por WhatsApp o correo conviene `--split-per-abi` y enviar solo `app-arm64-v8a-release.apk`, que funciona en casi todos los teléfonos Android de los últimos años.

17.2 Integración continua (GitHub Actions)

El archivo `.github/workflows/ci.yml` define cuatro trabajos. En cada *push* a `main` se ejecutan todos; en cada *pull request* solo el de pruebas.

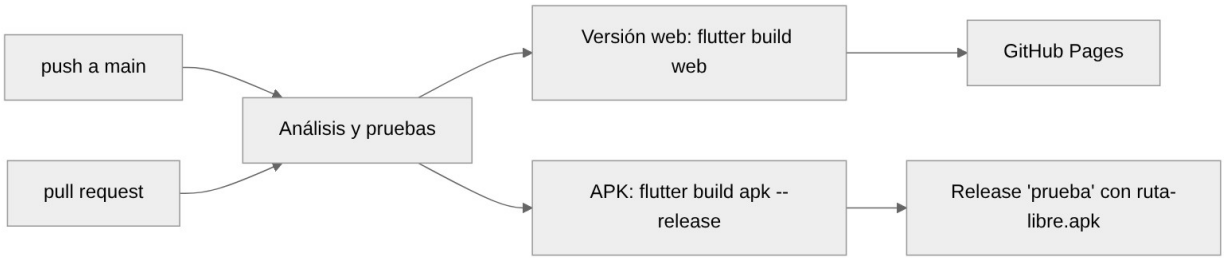


Figura 22. Flujo de integración continua.

Trabajo	Depende de	Pasos principales
Análisis y pruebas	—	Instala Flutter estable con caché, `pub get`, `flutter analyze --no-fatal-infos`, `flutter test --reporter expanded`.

Trabajo	Depende de	Pasos principales
Versión web	Pruebas	Genera la carpeta `web/`, compila con `--base-href` igual al nombre del repositorio y sube el artefacto de Pages.
Publicar en GitHub Pages	Web	Despliega el artefacto; la URL queda en el entorno `github-pages`.
APK de prueba	Pruebas	Java 17, genera `android/`, aplica permisos, `flutter build apk --release`, sube el artefacto y crea la versión preliminar «prueba» con `ruta-libre.apk`.

La versión «prueba» se borra y se vuelve a crear en cada corrida, de modo que el enlace de descarga siempre apunta al APK más reciente. La opción `concurrency` cancela corridas anteriores de la misma rama.

17.3 Instalación del APK en un teléfono

1. Copiar el APK al teléfono (cable USB, Google Drive o WhatsApp como documento) o descargarlo desde la versión «prueba» de GitHub.
2. Abrir el archivo desde el administrador de archivos o la app con la que se recibió.
3. Permitir «Instalar apps desconocidas» para esa app cuando Android lo pida.
4. Tocar **Instalar**. Si existe una versión anterior firmada con otra llave, desinstalarla primero.

17.4 Versionado

La versión se define en `pubspec.yaml` como `version: 0.1.0+1`, donde `0.1.0` es la versión visible (`versionName`) y `1` el número de compilación (`versionCode`). Cada APK que se instale encima de otro debe tener un número de compilación mayor. Se sigue versionado semántico: el primer número cambia con cambios incompatibles, el segundo con funciones nuevas y el tercero con correcciones.

18. Pruebas

18.1 Estrategia

La estrategia sigue la pirámide de pruebas: muchas pruebas unitarias rápidas sobre la lógica pura, algunas pruebas de *widgets* sobre los componentes críticos de accesibilidad y un plan de pruebas manuales en dispositivo para lo que depende del hardware (GPS, voz, cámara) y de servicios externos.

Nivel	Herramienta	Cantidad	Qué cubre
Unitarias	`flutter_test` (`test`)	34	Geografía, parser de OSM, API de rutas, ciclo de vida de reportes, avisos, comandos de voz, textos.
De widgets	`flutter_test` (`testWidgets`)	2	Etiqueta semántica e ícono de los marcadores; leyenda completa.
Análisis estático	`flutter analyze`	—	Errores de tipo, código muerto, estilo. Debe dar «No issues found!».
Manuales	Teléfono Android	18 casos	Flujos completos con GPS, voz, cámara y red.

18.2 Ejecución

```
flutter test                        # todas
flutter test test/geo_test.dart     # un archivo
flutter test --reporter expanded    # nombre de cada prueba
flutter test --coverage             # genera coverage/lcov.info
```

Resultado de la última ejecución en el equipo de desarrollo (30 de octubre de 2026): **36 pruebas, todas aprobadas**, en unos 2 segundos.

18.3 Catálogo de pruebas automatizadas

Archivo	Prueba	Qué verifica
geo_test	distancia de haversine	~111.2 m al norte, ~104.9 m al este y 0 m al mismo punto, con tolerancia de 1 m.
geo_test	rumbo	Norte, este y sur dan 0°, 90° y 180° (±0.5°).
geo_test	punto cardinal	0° y 350° «al norte», 44° «al noreste», 90° «al este», 225° «al suroeste».
geo_test	dirección relativa a hacia dónde camina	Frente, derecha, izquierda, diagonal, atrás y cruce del 0° (camina a 350°, destino a 30°).

Archivo	Prueba	Qué verifica
geo_test	sin rumbo usa puntos cardinales	`directionFrom` sin `heading`.
geo_test	distancias redondeadas para voz	4, 42, 48, 234, 1000 y 1260 m.
geo_test	cuadro alrededor de un peligro	Anillo cerrado de 5 vértices con lados de ~30 m.
geo_test	distancias y tiempos cortos para pantalla	«120 m», «1 km», «1.3 km», «1 min», «1 h 5 min», «2 h».
osm*parser*test	convierte elementos de Overpass en lugares	Nombre, categoría, accesibilidad, baño y descripción.
osm*parser*test	usa el centro de un way…	Coordenadas de `center` y categoría como nombre.
osm*parser*test	los cruces se califican por su borde de banqueta	`lowered` accesible, `raised` no accesible.
osm*parser*test	designated cuenta como accesible	Valor `designated`.
osm*parser*test	la consulta incluye el radio y el centro	Texto de la consulta Overpass.
osm*parser*test	Place sobrevive ida y vuelta por JSON	Serialización.
report_test	caduca según su tipo	Rampa bloqueada: activa a las 5 h, inactiva a las 6 h.
report_test	«sigue ahí» reinicia la caducidad…	Confirmación.
report_test	dos «ya no está» seguidos lo cierran	Cierre.
report_test	ida y vuelta por JSON	Serialización.
report_test	avisa solo lo cercano, vigente y no leve	Filtro de avisos.
report_test	no repite el aviso antes del tiempo de espera	Espera de 10 minutos.
report_test	el más cercano va primero	Orden.
route*api*test	coordenadas en orden [lon, lat] e idioma español	Cuerpo de la petición.
route*api*test	silla de ruedas: evita escalones y limita pendiente	Opciones del perfil.
route*api*test	los peligros se envían como zonas a evitar, salvo junto al origen	Regla de 25 m.
route*api*test	lee geometría [lon, lat], resumen y pasos	Lectura de la respuesta.
route*api*test	sin ruta lanza error	`FormatException`.
route*api*test	resumen e indicaciones habladas	Textos.
route*api*test	un minuto en singular	«un minuto».

Archivo	Prueba	Qué verifica
describe_test	tiempo transcurrido y restante	«hace un momento», «hace 15 minutos», «hace una hora», «hace 3 días», «en 5 horas», «ya caducó».
describe_test	descripción hablada de un lugar	Texto completo.
describe_test	descripción hablada de un reporte	Texto completo.
describe_test	reconoce cada comando sin importar acentos	Comandos.
describe_test	un peligro cerca es un reporte	Prioridad.
describe_test	frases sin comando	`unknown`.
widget_test	el marcador expone su descripción al lector de pantalla	Semántica, ícono `remove` y toque.
widget_test	la leyenda explica cada estado	Textos de los cuatro estados y el peligro.

18.4 Plan de pruebas manuales

Se ejecuta en un teléfono Android real dentro de la zona piloto, con conexión de datos y una clave de ORS configurada. Para cada caso se registra si pasa, la versión del APK y el modelo del teléfono.

ID	Caso	Pasos	Resultado esperado
PM-01	Primera apertura	Instalar y abrir.	Se escucha la bienvenida; Continuar deshabilitado hasta elegir; al continuar se pide la ubicación.
PM-02	Permiso negado	Negar la ubicación.	Mensaje en pantalla y por voz; el mapa sigue usable centrado en la zona piloto.
PM-03	Marcadores	Observar el mapa.	Círculos con ícono por estado; cruces cuadrados con zoom ≥ 17.
PM-04	Caché sin conexión	Abrir con datos, cerrar, activar modo avión y abrir.	Los lugares aparecen; no hay error de descarga.
PM-05	Filtros	Activar Solo accesibles y luego Farmacias.	El mapa y la lista se ajustan; se anuncia el número de resultados.
PM-06	Búsqueda	Escribir «panaderia» sin acento.	Aparecen panaderías; la hoja dice «Resultados».
PM-07	Cerca de ti	Arrastrar la hoja y tocar Escuchar.	Se leen los 5 más cercanos con distancia y dirección.
PM-08	Ficha	Tocar un lugar; responder «¿Es correcta?».	Datos correctos; agradecimiento.

ID	Caso	Pasos	Resultado esperado
PM-09	Ruta	Cómo llegar a un lugar a 300 m.	Ruta dibujada; etiquetas; Iniciar guía lee los pasos.
PM-10	Reporte con foto	Reportar Rampa bloqueada con foto.	Aparece el triángulo; «Caduca en 6 horas».
PM-11	Confirmar y descartar	Abrir el peligro: Sigue ahí; luego Ya no está dos veces.	Vigencia renovada; al segundo voto deja de mostrarse.
PM-12	Aviso de peligro	Caminar hacia un peligro de otro teléfono o creado hace 10+ min.	Vibración y «Atención…» a 30 m o menos.
PM-13	Voz	Decir «qué hay cerca», «dónde estoy», «farmacia del ahorro».	Se abre Cerca de ti; se dice el lugar más cercano; se busca el texto.
PM-14	Tema y letra	Cambiar a Oscuro y letra 200 %.	Todo legible, sin cortes; mapa oscuro.
PM-15	Clave inválida	Escribir una clave falsa y pedir ruta.	«La clave de openrouteservice no es válida.»
PM-16	Otra zona	Mover el mapa 2 km y tocar Buscar lugares aquí.	Descarga y anuncia; aparecen lugares nuevos.
PM-17	TalkBack	Activar TalkBack y recorrer la app.	Cada elemento se lee con su etiqueta; se puede reportar sin ver.
PM-18	Rotación y regreso	Salir y volver a la app con ruta activa.	La ruta y la hoja se conservan.

18.5 Cobertura pendiente

- Pruebas de *widgets* de las pantallas completas con un `AppState` de prueba y servicios simulados.
- Pruebas de `PlacesService` y `RoutingService` con `MockClient` del paquete `http/testing.dart`.
- Pruebas de integración (`integration_test`) en dispositivo para el flujo de reporte.
- Pruebas de contraste automatizadas sobre los tokens del tema.

19. Mantenimiento y extensión

19.1 Cambiar la zona piloto

1. Buscar el nuevo centro en openstreetmap.org y copiar sus coordenadas (clic derecho → «Mostrar dirección»).
2. Editar `AppConfig.pilotCenter` y, si hace falta, `pilotRadiusMeters` en `lib/config.dart`.
3. Cambiar el subtítulo «Alrededores de UPIICSA» en `map_screen.dart`.
4. Compilar. La caché de la zona anterior queda guardada pero ya no se usa.

19.2 Agregar un tipo de reporte

1. Agregar el valor a `ReportType` en `lib/models/report.dart` con su etiqueta, gravedad por defecto y TTL.
2. Agregar su ícono en `reportIcon` (`new_report_screen.dart`). El compilador marca error si falta, porque el `switch` es exhaustivo.
3. Agregar una prueba de caducidad en `report_test.dart`.

> **Importante:** Nunca renombrar ni quitar un valor existente de `ReportType`: los reportes guardados se leen con `ReportType.values.byName`, que fallaría con un nombre que ya no existe.

19.3 Agregar una categoría de lugar

1. Agregar la etiqueta a la expresión regular de `buildOverpassQuery`.
2. Agregar su traducción en `_amenity` o `_shop` de `osm_parser.dart`.
3. Si debe tener filtro rápido, agregarlo a `PlaceFilter` en `map_screen.dart`.
4. Forzar la descarga desde **Ajustes** → **Actualizar lugares** para ver el cambio.

19.4 Agregar un comando de voz

1. Agregar el valor a `VoiceCommand` y sus palabras clave (sin acentos) a `_keywords`, cuidando la prioridad.
2. Atender el nuevo caso en el `switch` de `_listen` en `map_screen.dart`.
3. Agregar la prueba en `describe_test.dart`.

19.5 Cambiar colores o tipografía

Todos los colores están en `RLColors.light`, `RLColors.dark` y las constantes de estado de `AppTheme`. Al cambiarlos hay que volver a verificar el contraste con la fórmula de la sección 11.3. La tipografía se cambia en `AppTheme.fontFamily` y en la sección `fonts` de `pubspec.yaml`.

19.6 Servidor de reportes compartidos

Es el siguiente paso del proyecto. La arquitectura propuesta usa servicios administrados de AWS para no operar servidores:

Componente	Servicio	Función
API	Amazon API Gateway (HTTP API)	`GET /reports?lat&lon&radio`, `POST /reports`, `POST /reports/{id}/confirm`, `POST /reports/{id}/gone`.
Lógica	AWS Lambda	Validación, ciclo de vida (misma lógica que `report.dart`), límite de votos por dispositivo.
Datos	Amazon DynamoDB	Tabla de reportes con clave de partición por celda geográfica (geohash de 6 caracteres) y TTL nativo.
Fotos	Amazon S3 + URL prefirmadas	Subida directa desde el teléfono; borrado automático por regla de ciclo de vida.
Identidad	Identificador anónimo por instalación	Evitar votos repetidos sin pedir cuenta.

En la app solo hay que crear `HttpReportRepository implements ReportRepository` y pasarla en `main.dart`. Conviene mantener la versión local como caché para funcionar sin conexión y sincronizar al recuperarla.

19.7 Actualizar dependencias

```
flutter upgrade              # actualiza el SDK
flutter pub outdated         # muestra versiones nuevas
flutter pub upgrade          # dentro de las restricciones de pubspec.yaml
flutter pub upgrade --major-versions   # cambia restricciones (revisar notas)
flutter analyze && flutter test
```

Después de actualizar Flutter conviene regenerar las carpetas nativas (borrarlas, `flutter create` y `patch_platforms.dart`), porque la plantilla de Android cambia entre versiones.

20. Solución de problemas técnicos

Síntoma	Causa probable	Solución
`flutter` no se reconoce como comando	PATH no actualizado.	Cerrar y abrir la terminal; verificar `C:\src\flutter\bin` en PATH del usuario.
«Android sdkmanager not found»	Faltan command-line tools.	Instalarlas desde SDK Manager o con `instalar_entorno.bat`.
Error de NDK o rutas con espacios al compilar	SDK en carpeta con espacios.	`preparar_android.bat` (enlace `C:\src\android-sdk`).
«Package ndk;28… not found»	Falla del sdkmanager.	`instalar_ndk.bat`.
Advertencias de «Built-in Kotlin» al compilar	Complementos aún no migrados.	Son advertencias; no impiden compilar.
El mapa sale gris	Sin conexión o teselas bloqueadas.	Revisar conexión; en web, revisar la consola.
«El servidor de OpenStreetMap está ocupado»	Overpass saturado (429/504).	Esperar un minuto; la app usa la caché si existe.
No aparecen lugares en una zona	Zona sin datos en OSM o fuera de las zonas descargadas.	Usar **Buscar lugares aquí**; verificar en openstreetmap.org.
«Falta la clave de openrouteservice»	Sin clave.	Escribirla en Ajustes → Rutas.
«No se encontró una ruta accesible»	No existe camino con las restricciones o el destino está fuera de la red peatonal.	Probar con otro destino; con silla de ruedas las restricciones son estrictas.
La voz no habla	Motor TTS sin español o desactivado.	Instalar voz en español en Ajustes del teléfono → Texto a voz.
El micrófono no entiende	Sin servicio de reconocimiento o sin conexión.	Instalar o actualizar la app de Google; revisar permiso de micrófono.
En web la bienvenida no avanza	Motor de voz del navegador sin respuesta (versiones antiguas).	Ya corregido con topes de tiempo; actualizar el código.
«App no instalada» al instalar el APK	Firma diferente a la instalada.	Desinstalar la versión anterior.
La foto del reporte desaparece en web	URL temporal del navegador.	Comportamiento esperado en web; en Android se guarda.

21. Limitaciones conocidas y trabajo futuro

21.1 Limitaciones

- Los reportes no se comparten entre teléfonos (sin servidor).
- La calidad de la accesibilidad depende de los datos de OpenStreetMap; en la zona piloto muchos lugares tienen «Sin dato».
- Las indicaciones de ruta no se dan paso a paso conforme se avanza: se leen completas al iniciar la guía.
- No hay modo sin conexión para el mapa de fondo, por la política de teselas.
- La dirección relativa depende del movimiento; al estar quieto se usan puntos cardinales.
- El nombre de la aplicación en el lanzador de Android es el de la plantilla («ruta_libre»); debe cambiarse `android:label` a «Ruta Libre».
- La versión de iOS no se ha compilado ni probado.
- Los votos «¿Es correcta?» se guardan pero aún no se envían a OSM.

21.2 Hoja de ruta

Fase	Entregables
1. Piloto con usuarios	Pruebas en campo con 5–10 personas de los cuatro perfiles; corrección de datos en OSM de la zona; ajuste de textos de voz.
2. Reportes compartidos	Servidor en AWS (19.6), sincronización, moderación básica, límite de reportes por dispositivo.
3. Navegación activa	Indicaciones conforme se avanza, recálculo al desviarse, aviso de llegada.
4. Contribución a OSM	Envío de notas de OSM con las respuestas «¿Es correcta?» y con fotos de rampas.
5. Publicación	Firma propia, nombre e ícono definitivos, Google Play (prueba cerrada), versión web pública, compilación para iOS.

Anexo A. Glosario

Accesibilidad
Grado en que un lugar, servicio o aplicación puede ser usado por personas con discapacidad sin ayuda adicional.

Atribución
Mención obligatoria de la fuente de los datos («© colaboradores de OpenStreetMap»).

Borde de banqueta (kerb)
Desnivel entre la banqueta y la calle. Puede estar al ras, rebajado, redondeado o elevado.

Caché
Copia local de datos descargados para reutilizarlos sin volver a pedirlos.

ChangeNotifier
Clase de Flutter que avisa a sus oyentes cuando cambia.

Cruce
En la app, un nodo de OSM con información de borde de banqueta o guía podotáctil, sin nombre.

Guía podotáctil
Franja de piso con relieve que orienta a personas ciegas con el bastón o los pies.

Haversine
Fórmula que calcula la distancia sobre una esfera a partir de latitudes y longitudes.

Hot reload
Recarga en caliente: aplicar cambios de código sin reiniciar la app.

InheritedNotifier
Widget que comparte un objeto observable con sus descendientes y los reconstruye al cambiar.

Lector de pantalla
Software que lee en voz alta lo que hay en pantalla (TalkBack, VoiceOver, NVDA).

liveRegion
Propiedad semántica que pide al lector de pantalla anunciar un cambio sin mover el foco.

Nodo, vía, relación
Tipos de elemento de OSM: un punto, una línea o polígono, y un grupo de elementos.

Overpass QL
Lenguaje de consultas de la API Overpass.

Perfil de movilidad
: Forma en que la persona se desplaza; cambia rutas, filtros y voz.

Polilínea
: Línea formada por varios segmentos; así se dibuja la ruta.

Rumbo
: Ángulo en grados desde el norte hacia una dirección, en el sentido de las manecillas del reloj.

Semántica
: Información que Flutter expone a los servicios de accesibilidad: etiqueta, rol, estado.

SharedPreferences
: Almacenamiento clave–valor persistente de Android, iOS y web.

Tesela
: Imagen cuadrada que forma parte del mapa de fondo.

Token de diseño
: Valor con nombre (color, tamaño) que se usa en toda la interfaz en lugar de un valor literal.

TTL
: Tiempo de vida de un reporte antes de caducar.

Anexo B. Archivos de configuración

B.1 pubspec.yaml

```
name: ruta_libre
description: Mapa de lugares y rutas accesibles con reportes de obstáculos y guía por voz.
publish_to: "none"
version: 0.1.0+1

environment:
  sdk: ^3.8.0
  flutter: ">=3.32.0"

dependencies:
  flutter:
    sdk: flutter
  flutter_localizations:
    sdk: flutter

  # Mapa (OpenStreetMap)
  flutter_map: ^8.3.2
  latlong2: any # la versión la fija flutter_map

  # Red: Overpass y openrouteservice
  http: ^1.2.0

  # Ubicación, voz y fotos
  geolocator: ^14.0.0
  flutter_tts: ^4.2.5
  speech_to_text: ^7.3.0
  image_picker: ^1.2.0
  path_provider: ^2.1.0

  # Enlaces (atribución de OSM, ver en OSM)
  url_launcher: ^6.3.0

  # Datos locales
  shared_preferences: ^2.5.0

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0

flutter:
  uses-material-design: true
  fonts:
    # Atkinson Hyperlegible (Braille Institute, licencia OFL): letras
    # pensadas para baja visión. Ver assets/fonts/OFL.txt.
    - family: Atkinson Hyperlegible
      fonts:
        - asset: assets/fonts/AtkinsonHyperlegible-Regular.ttf
        - asset: assets/fonts/AtkinsonHyperlegible-Bold.ttf
          weight: 700
```

B.2 analysis_options.yaml

```
include: package:flutter_lints/flutter.yaml

linter:
  rules:
    prefer_const_constructors: true
    prefer_final_locals: true
    always_declare_return_types: true
```

B.3 .github/workflows/ci.yml

```
name: CI

# En cada push a main: análisis + pruebas; si pasan, APK de prueba y
# versión web en GitHub Pages. En pull requests solo análisis + pruebas.
on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

env:
  # Las carpetas nativas no se versionan: se generan en cada corrida con la
  # misma plantilla de Flutter y se les aplica tool/patch_platforms.dart.
  CREATE_ARGS: --org com.drak028 --project-name ruta_libre

jobs:
  test:
    name: Análisis y pruebas
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: subosito/flutter-action@v2
        with:
          channel: stable
          cache: true
      - run: flutter --version
      - run: flutter pub get
      - name: Analizar
        run: flutter analyze --no-fatal-infos
      - name: Pruebas
        run: flutter test --reporter expanded

  web:
    name: Versión web
    needs: test
    if: github.event_name != 'pull_request'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: subosito/flutter-action@v2
        with:
          channel: stable
          cache: true
      - run: flutter create $CREATE_ARGS --platforms web .
      - run: flutter pub get
      # Sin clave de rutas: en web la escribe cada quien en Ajustes, así no
      # queda expuesta en el código publicado.
      - run: flutter build web --release --base-href /${{ github.event.repository.name }}/
      - uses: actions/upload-pages-artifact@v3
        with:
          path: build/web

  pages:
    name: Publicar en GitHub Pages
    needs: web
    runs-on: ubuntu-latest
    permissions:
      pages: write
      id-token: write
    environment:
      name: github-pages
      url: ${{ steps.deploy.outputs.page_url }}
    steps:
```

```
      - id: deploy
        uses: actions/deploy-pages@v4

  apk:
    name: APK de prueba
    needs: test
    if: github.event_name != 'pull_request'
    runs-on: ubuntu-latest
    permissions:
      contents: write
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'
      - uses: subosito/flutter-action@v2
        with:
          channel: stable
          cache: true
      - run: flutter create $CREATE_ARGS --platforms android .
      - run: dart run tool/patch_platforms.dart
      - run: flutter pub get
      # La plantilla de Flutter firma el release con la llave de depuración:
      # sirve para instalar en tu teléfono, no para la Play Store.
      - run: flutter build apk --release
      - run: cp build/app/outputs/flutter-apk/app-release.apk ruta-libre.apk
      - uses: actions/upload-artifact@v4
        with:
          name: ruta-libre-apk
          path: ruta-libre.apk
      - name: Publicar como versión de prueba
        env:
          GH_TOKEN: ${{ github.token }}
        run: |
          gh release delete prueba --yes --cleanup-tag || true
          gh release create prueba ruta-libre.apk \
            --prerelease \
            --target "$GITHUB_SHA" \
            --title "Versión de prueba" \
            --notes "APK generado automáticamente desde ${GITHUB_SHA::7}. Descárgalo desde el
teléfono e instálalo (permite instalar apps de origen desconocido)."
```

B.4 .gitignore

```
# Flutter / Dart
.dart_tool/
.flutter-plugins
.flutter-plugins-dependencies
.packages
.pub-cache/
.pub/
build/
coverage/

# IDE
.idea/
.vscode/
*.iml

# Android
android/.gradle/
android/local.properties
android/key.properties
*.jks

# iOS
ios/Pods/
```

```
ios/.symlinks/
ios/Flutter/Generated.xcconfig
ios/Flutter/flutter_export_environment.sh
**/xcuserdata/

.DS_Store

# Carpetas nativas y web: se generan con `flutter create` (ver README y CI)
/android/
/ios/
/web/
/linux/
/macos/
/windows/
.metadata

# Registros locales de la instalación
/setup/instalacion.log
/setup/estado.txt
/setup/salida/
```

B.5 setup/generar_apk.ps1

```
# Revisa el codigo y genera el APK de prueba (release, firmado con la clave de depuracion)
$ErrorActionPreference = 'Continue'
$env:Path = 'C:\src\flutter\bin;' + [Environment]::GetEnvironmentVariable('Path','User') + ';'
+ [Environment]::GetEnvironmentVariable('Path','Machine')
$proyecto = Split-Path -Parent $PSScriptRoot
$salida = Join-Path $PSScriptRoot 'salida'
New-Item -ItemType Directory -Force -Path $salida | Out-Null
Set-Location $proyecto
$estado = Join-Path $salida 'apk_estado.txt'
Set-Content $estado 'en curso: pub get' -Encoding UTF8
Write-Host 'Ruta Libre - generando APK. No cierres esta ventana (tarda unos minutos).'
& flutter pub get *> (Join-Path $salida 'pub_get.txt')
Set-Content $estado 'en curso: analyze' -Encoding UTF8
Write-Host '1/3 Revisando el codigo...'
& flutter analyze --no-fatal-infos *> (Join-Path $salida 'analyze.txt')
$a = $LASTEXITCODE
Set-Content $estado 'en curso: test' -Encoding UTF8
Write-Host '2/3 Corriendo pruebas...'
& flutter test --reporter expanded *> (Join-Path $salida 'test.txt')
$t = $LASTEXITCODE
$b = -1
if ($a -eq 0) {
  Set-Content $estado 'en curso: build apk' -Encoding UTF8
  Write-Host '3/3 Construyendo el APK...'
  & flutter build apk --release *> (Join-Path $salida 'apk.txt')
  $b = $LASTEXITCODE
  $origen = Join-Path $proyecto 'build\app\outputs\flutter-apk\app-release.apk'
  if ($b -eq 0 -and (Test-Path $origen)) {
    Copy-Item $origen (Join-Path $proyecto 'RutaLibre.apk') -Force
  }
}
Set-Content $estado "analyze=$a test=$t apk=$b fin=$(Get-Date -Format s)" -Encoding UTF8
if ($b -eq 0) { Write-Host 'Listo: C:\src\ruta_libre\RutaLibre.apk' } else { Write-Host 'Hubo
un error; revisa setup\salida' }
```

B.6 setup/revisar.ps1

```
# Corre analisis y pruebas y guarda la salida completa en setup\salida\
$ErrorActionPreference = 'Continue'
$env:Path = 'C:\src\flutter\bin;' + [Environment]::GetEnvironmentVariable('Path','User') + ';'
+ [Environment]::GetEnvironmentVariable('Path','Machine')
$proyecto = Split-Path -Parent $PSScriptRoot
$salida = Join-Path $PSScriptRoot 'salida'
New-Item -ItemType Directory -Force -Path $salida | Out-Null
```

```
Set-Location $proyecto
Set-Content (Join-Path $salida 'estado.txt') 'en curso' -Encoding UTF8
& flutter pub get *> (Join-Path $salida 'pub_get.txt')
& flutter analyze --no-fatal-infos *> (Join-Path $salida 'analyze.txt')
$a = $LASTEXITCODE
& flutter test --reporter expanded *> (Join-Path $salida 'test.txt')
$t = $LASTEXITCODE
& flutter doctor -v *> (Join-Path $salida 'doctor.txt')
& flutter devices *> (Join-Path $salida 'devices.txt')
Set-Content (Join-Path $salida 'estado.txt') "analyze=$a test=$t fin=$(Get-Date -Format s)" -
Encoding UTF8
```

Anexo C. Código fuente

Este anexo reproduce el código fuente completo de la versión 0.1.0, tal como se compiló el APK de pruebas del 30 de octubre de 2026. Los archivos se presentan en el orden de las capas: arranque, modelos, lógica pura, servicios, componentes, pantallas, pruebas y herramientas.

Archivo	Líneas	Descripción
lib/main.dart	27	Punto de entrada.
lib/app.dart	78	Aplicación, temas, idioma y escala de texto.
lib/config.dart	33	Configuración del piloto y de servicios.
lib/app_state.dart	308	Estado global y AppScope.
lib/models/place.dart	104	Modelo de lugar.
lib/models/report.dart	123	Modelo de reporte y su ciclo de vida.
lib/models/route_result.dart	37	Modelo de ruta.
lib/models/settings.dart	137	Ajustes, perfiles y tema.
lib/core/geo.dart	123	Cálculos geográficos.
lib/core/osm_parser.dart	142	Consulta y lectura de OpenStreetMap.
lib/core/route_api.dart	125	Petición y respuesta de openrouteservice.
lib/core/hazard_alerts.dart	25	Selección de avisos de peligro.
lib/core/voice_commands.dart	33	Comandos de voz.
lib/core/describe.dart	66	Descripciones habladas.
lib/core/text_utils.dart	37	Utilidades de texto.
lib/core/theme.dart	252	Tema claro y oscuro.
lib/services/location_service.dart	61	Servicio de ubicación.
lib/services/places_service.dart	104	Servicio de lugares con caché.
lib/services/report_repository.dart	51	Repositorio de reportes.
lib/services/routing_service.dart	95	Cliente de rutas.
lib/services/voice_service.dart	145	Voz: hablar y escuchar.
lib/widgets/base_map.dart	63	Mapa base, línea de ruta y crédito.
lib/widgets/map_markers.dart	259	Marcadores y leyenda.
lib/widgets/ui.dart	505	Componentes compartidos.
lib/widgets/report_photo.dart	52	Foto de un reporte.
lib/features/onboarding/ profile_screen.dart	255	Pantalla de bienvenida y perfil.

Archivo	Líneas	Descripción
lib/features/map/map_screen.dart	971	Pantalla del mapa.
lib/features/nearby/nearby_screen.dart	158	Pantalla Cerca de ti.
lib/features/place/place_screen.dart	353	Ficha del lugar.
lib/features/route/route_screen.dart	404	Pantalla de ruta.
lib/features/report/new_report_screen.dart	384	Pantalla para reportar.
lib/features/report/report_screen.dart	211	Detalle de un reporte.
lib/features/settings/settings_screen.dart	309	Ajustes.
test/geo_test.dart	76	Pruebas de geografía.
test/osm_parser_test.dart	116	Pruebas del parser de OSM.
test/report_test.dart	101	Pruebas de reportes y avisos.
test/route_api_test.dart	120	Pruebas de la API de rutas.
test/describe_test.dart	81	Pruebas de textos y comandos de voz.
test/widget_test.dart	56	Pruebas de widgets.
tool/patch_platforms.dart	120	Configuración nativa de Android e iOS.

lib/main.dart

Punto de entrada.

```
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

import 'app.dart';
import 'app_state.dart';
import 'services/location_service.dart';
import 'services/places_service.dart';
import 'services/report_repository.dart';
import 'services/routing_service.dart';
import 'services/voice_service.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final prefs = await SharedPreferences.getInstance();

  final state = AppState(
    prefs: prefs,
    voice: VoiceService(),
    location: LocationService(),
    places: PlacesService(prefs),
    reportsRepo: LocalReportRepository(prefs),
    routing: RoutingService(),
  );
  await state.init();

  runApp(RutaLibreApp(state: state));
}
```

lib/app.dart

Aplicación, temas, idioma y escala de texto.

```
import 'dart:math' as math;

import 'package:flutter/material.dart';
import 'package:flutter_localizations/flutter_localizations.dart';

import 'app_state.dart';
import 'core/theme.dart';
import 'features/map/map_screen.dart';
import 'features/onboarding/profile_screen.dart';
import 'models/settings.dart';

class RutaLibreApp extends StatefulWidget {
  const RutaLibreApp({super.key, required this.state});

  final AppState state;

  @override
  State<RutaLibreApp> createState() => _RutaLibreAppState();
}

class _RutaLibreAppState extends State<RutaLibreApp> {
  late final bool _needsProfile = widget.state.settings.profile == null;
  late ThemePreference _theme = widget.state.settings.theme;

  @override
  void initState() {
    super.initState();
    widget.state.addListener(_onState);
  }

  /// Solo se reconstruye la app entera si cambia el tema.
  void _onState() {
    final next = widget.state.settings.theme;
    if (next != _theme) setState(() => _theme = next);
  }

  @override
  void dispose() {
    widget.state.removeListener(_onState);
    widget.state.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return AppScope(
      state: widget.state,
      child: MaterialApp(
        title: 'Ruta Libre',
        debugShowCheckedModeBanner: false,
        theme: AppTheme.light(),
        darkTheme: AppTheme.dark(),
        themeMode: switch (_theme) {
          ThemePreference.system => ThemeMode.system,
          ThemePreference.light => ThemeMode.light,
          ThemePreference.dark => ThemeMode.dark,
        },
        locale: const Locale('es', 'MX'),
        supportedLocales: const [Locale('es', 'MX'), Locale('es')],
        localizationsDelegates: GlobalMaterialLocalizations.delegates,
        builder: (context, child) {
          // Nunca más chico que la letra del sistema; tope para no desbordar.
          final media = MediaQuery.of(context);
          final system = media.textScaler.scale(100) / 100;
          final user = AppScope.of(context).settings.textScale;
```

```
          final scale = math.min(math.max(system, user), 2.0);
          return MediaQuery(
            data: media.copyWith(textScaler: TextScaler.linear(scale)),
            child: child!,
          );
        },
        home: _needsProfile
            ? const ProfileScreen(firstTime: true)
            : const MapScreen(),
      ),
    );
  }
}
```

lib/config.dart

Configuración del piloto y de servicios.

```
import 'package:latlong2/latlong.dart';

/// Configuración del piloto. Ajusta estos valores a tu caso.
class AppConfig {
  /// Centro aproximado de UPIICSA (Av. Té 950, Granjas México, Iztacalco).
  /// Verifícalo en openstreetmap.org y ajústalo si hace falta.
  static const pilotCenter = LatLng(19.3958, -99.0912);

  /// Radio de la zona piloto en metros.
  static const pilotRadiusMeters = 1000;

  /// Identificador de la app para OpenStreetMap (User-Agent).
  /// La política de OSM pide uno propio y con forma de contacto.
  static const packageName = 'com.drak028.ruta_libre';
  static const userAgent =
      'RutaLibre/0.1 (prototipo estudiantil; https://github.com/DraK028)';

  static const overpassUrl = 'https://overpass-api.de/api/interpreter';
  static const orsBaseUrl = 'https://api.openrouteservice.org/v2/directions';

  /// Clave de openrouteservice. No se guarda en el código:
  /// `flutter run --dart-define=ORS_API_KEY=tu_clave`
  static const orsApiKey = String.fromEnvironment('ORS_API_KEY');

  /// Cada cuánto se vuelven a descargar los lugares de una zona.
  static const placesCacheTtl = Duration(days: 7);

  /// Distancia a la que se avisa de un peligro reportado.
  static const hazardAlertMeters = 30.0;

  /// Tiempo mínimo antes de repetir el aviso del mismo peligro.
  static const hazardAlertCooldown = Duration(minutes: 10);
}
```

lib/app_state.dart

Estado global y AppScope.

```
import 'dart:convert';

import 'package:flutter/services.dart';
import 'package:flutter/widgets.dart';
import 'package:latlong2/latlong.dart';
import 'package:shared_preferences/shared_preferences.dart';

import 'config.dart';
import 'core/describe.dart';
import 'core/geo.dart';
```

```
import 'core/hazard_alerts.dart';
import 'models/place.dart';
import 'models/report.dart';
import 'models/route_result.dart';
import 'models/settings.dart';
import 'services/location_service.dart';
import 'services/places_service.dart';
import 'services/report_repository.dart';
import 'services/routing_service.dart';
import 'services/voice_service.dart';

/// Estado de la app y acceso a los servicios.
///
/// Las pantallas solo hablan con esta clase; cada servicio sigue siendo
/// independiente y reemplazable.
class AppState extends ChangeNotifier {
  AppState({
    required SharedPreferences prefs,
    required this.voice,
    required this.location,
    required this.places,
    required this.reportsRepo,
    required this.routing,
  })  : _prefs = prefs,
        _settings = _loadSettings(prefs) {
    voice.guidanceEnabled = _settings.voiceGuidance;
    location.position.addListener(_onMove);
  }

  static const _kSettings = 'settings';
  static const _kFeedback = 'place_feedback';

  final SharedPreferences _prefs;
  final VoiceService voice;
  final LocationService location;
  final PlacesService places;
  final ReportRepository reportsRepo;
  final RoutingService routing;

  AppSettings _settings;
  List<Place> _places = [];
  List<Report> _reports = [];
  RouteResult? _route;
  bool _loadingPlaces = false;
  String? _placesError;
  final Map<String, DateTime> _lastAlerted = {};

  AppSettings get settings => _settings;
  MobilityProfile get profile => _settings.profile ?? MobilityProfile.mobilityAid;
  List<Place> get allPlaces => List.unmodifiable(_places);
  RouteResult? get route => _route;
  bool get loadingPlaces => _loadingPlaces;
  String? get placesError => _placesError;
  LatLng? get me => location.position.value;
  double? get heading => location.heading;

  /// Lugares que se muestran según el filtro "solo accesibles".
  List<Place> get visiblePlaces => _settings.onlyAccessible
      ? _places
          .where((p) =>
              p.access == Accessibility.yes || p.access == Accessibility.limited)
          .toList()
      : allPlaces;

  List<Report> activeReports([DateTime? now]) {
    final t = now ?? DateTime.now();
    return _reports.where((r) => r.isActive(t)).toList();
  }

  // ---------- Arranque ----------
```

```
Future<void> init() async {
  await voice.init(speechRate: _settings.speechRate);
  _reports = await reportsRepo.loadAll();
  final cached = places.cached(AppConfig.pilotCenter, AppConfig.pilotRadiusMeters);
  if (cached != null) _places = cached;
  notifyListeners();
}

/// Empieza a seguir la ubicación y descarga la zona piloto.
/// Devuelve un mensaje para el usuario si algo falla.
Future<String?> start() async {
  final locationError = await location.start();
  await loadPlaces(AppConfig.pilotCenter);
  return locationError ?? _placesError;
}

// ---------- Ajustes ----------

static AppSettings _loadSettings(SharedPreferences prefs) {
  final raw = prefs.getString(_kSettings);
  if (raw == null) return const AppSettings();
  return AppSettings.fromJson((jsonDecode(raw) as Map).cast<String, Object?>());
}

Future<void> updateSettings(AppSettings next) async {
  _settings = next;
  voice.guidanceEnabled = next.voiceGuidance;
  await voice.setSpeechRate(next.speechRate);
  await _prefs.setString(_kSettings, jsonEncode(next.toJson()));
  notifyListeners();
}

/// Primera elección del perfil: también fija los valores por defecto.
Future<void> chooseProfile(MobilityProfile p) => updateSettings(
      _settings.copyWith(
        profile: p,
        voiceGuidance: p.voiceByDefault || _settings.voiceGuidance,
        onlyAccessible: p.onlyAccessibleByDefault,
      ),
    );

// ---------- Lugares ----------

Future<void> loadPlaces(LatLng center, {bool force = false}) async {
  _loadingPlaces = true;
  _placesError = null;
  notifyListeners();
  try {
    final fresh = await places.load(center, force: force);
    // Une con lo que ya había (otra zona) sin duplicar.
    final byKey = {for (final p in _places) p.key: p};
    for (final p in fresh) {
      byKey[p.key] = p;
    }
    _places = byKey.values.toList();
  } on PlacesException catch (e) {
    _placesError = e.message;
  } finally {
    _loadingPlaces = false;
    notifyListeners();
  }
}

/// Lugares y peligros ordenados por distancia (para la lista hablada).
List<Place> placesNearby({int limit = 30, bool includeCrossings = false}) {
  final here = me ?? AppConfig.pilotCenter;
  final list = visiblePlaces
      .where((p) => includeCrossings || p.kind == PlaceKind.poi)
      .toList()
```

```
    ..sort((a, b) => distanceMeters(here, a.location)
        .compareTo(distanceMeters(here, b.location)));
  return list.take(limit).toList();
}

List<Report> reportsNearby() {
  final here = me ?? AppConfig.pilotCenter;
  return activeReports()
    ..sort((a, b) => distanceMeters(here, a.location)
        .compareTo(distanceMeters(here, b.location)));
}

/// "¿Es correcto?" — se guarda para enviarlo a OSM como nota en el piloto.
Future<void> saveFeedback(Place p, {required bool correct}) async {
  final list = _prefs.getStringList(_kFeedback) ?? [];
  list.add(jsonEncode({
    'place': p.key,
    'access': p.access.name,
    'correct': correct,
    'at': DateTime.now().toIso8601String(),
  }));
  await _prefs.setStringList(_kFeedback, list);
}

// ---------- Reportes ----------

Future<Report> addReport({
  required ReportType type,
  required Severity severity,
  required LatLng at,
  String note = '',
  String? photoPath,
}) async {
  final now = DateTime.now();
  final report = Report(
    id: '${now.microsecondsSinceEpoch}',
    type: type,
    severity: severity,
    location: at,
    createdAt: now,
    lastConfirmedAt: now,
    note: note,
    photoPath: photoPath,
  );
  await _saveReport(report);
  // Quien reporta ya sabe del peligro: no se le avisa al instante.
  _lastAlerted[report.id] = now;
  return report;
}

Future<Report> confirmReport(Report r) => _saveReport(r.confirm(DateTime.now()));

Future<Report> voteGone(Report r) => _saveReport(r.voteGone());

Report? reportById(String id) {
  for (final r in _reports) {
    if (r.id == id) return r;
  }
  return null;
}

Future<Report> _saveReport(Report r) async {
  await reportsRepo.save(r);
  final i = _reports.indexWhere((x) => x.id == r.id);
  if (i == -1) {
    _reports = [..._reports, r];
  } else {
    _reports = [..._reports]..[i] = r;
  }
  notifyListeners();
```

```
  return r;
}

// ---------- Rutas ----------

/// La clave escrita en Ajustes tiene prioridad sobre la de compilación.
String get routingKey => _settings.orsApiKey.trim().isNotEmpty
    ? _settings.orsApiKey.trim()
    : AppConfig.orsApiKey;

bool get hasRoutingKey => routingKey.isNotEmpty;

Future<RouteResult> routeTo(Place destination) async {
  final start = me;
  if (start == null) {
    throw const RoutingException('Aún no sé dónde estás. Revisa la ubicación.');
  }
  final result = await routing.route(
    apiKey: routingKey,
    start: start,
    end: destination.location,
    destinationName: destination.name,
    profile: profile,
    hazards: activeReports()
        .where((r) => r.severity != Severity.low)
        .map((r) => r.location)
        .toList(),
  );
  _route = result;
  notifyListeners();
  return result;
}

void clearRoute() {
  _route = null;
  notifyListeners();
}

// ---------- Avisos de peligro ----------

void _onMove() {
  // El mapa y las distancias dependen de la posición: se redibujan.
  // El GPS solo avisa cada 3 m, así que no es una carga excesiva.
  notifyListeners();
  final here = me;
  if (here == null || !_settings.hazardAlerts) return;
  final now = DateTime.now();
  final due = hazardsToAnnounce(
    reports: _reports,
    me: here,
    now: now,
    lastAlerted: _lastAlerted,
    radiusMeters: AppConfig.hazardAlertMeters,
    cooldown: AppConfig.hazardAlertCooldown,
  );
  if (due.isEmpty) return;
  final nearest = due.first;
  for (final r in due) {
    _lastAlerted[r.id] = now;
  }
  HapticFeedback.heavyImpact();
  // El aviso de peligro se dice aunque la guía por voz esté apagada.
  voice.speak('Atención. ${describeReport(nearest, now, me: here, heading: heading)}');
}

@override
void dispose() {
  location.position.removeListener(_onMove);
  location.dispose();
  voice.dispose();
```

```
    super.dispose();
  }
}

/// Acceso a [AppState] desde cualquier pantalla.
class AppScope extends InheritedNotifier<AppState> {
  const AppScope({super.key, required AppState state, required super.child})
      : super(notifier: state);

  static AppState of(BuildContext context) =>
      context.dependOnInheritedWidgetOfExactType<AppScope>()!.notifier!;

  /// Sin redibujar al cambiar el estado (para callbacks e initState).
  static AppState read(BuildContext context) =>
      (context.getElementForInheritedWidgetOfExactType<AppScope>()!.widget
              as AppScope)
          .notifier!;
}
```

lib/models/place.dart

Modelo de lugar.

```
import 'package:latlong2/latlong.dart';

/// Qué tan accesible es un lugar para silla de ruedas, según OSM.
enum Accessibility {
  /// `wheelchair=yes` o `designated`: entrada y áreas principales sin escalón.
  yes('Accesible', 'entrada sin escalón'),

  /// `wheelchair=limited`: escalón de hasta 7 cm o acceso parcial.
  limited('Acceso limitado', 'escalón pequeño o acceso parcial'),

  /// `wheelchair=no`: escalón mayor a 7 cm o áreas principales no accesibles.
  no('No accesible', 'tiene escalones o áreas sin acceso'),

  /// Sin etiqueta: nadie lo ha registrado todavía.
  unknown('Sin dato', 'nadie ha registrado su accesibilidad');

  const Accessibility(this.label, this.explanation);
  final String label;
  final String explanation;

  static Accessibility fromTag(String? value) => switch (value) {
        'yes' || 'designated' => Accessibility.yes,
        'limited' => Accessibility.limited,
        'no' => Accessibility.no,
        _ => Accessibility.unknown,
      };
}

/// Tipo de elemento en el mapa.
enum PlaceKind {
  /// Comercio, servicio o edificio.
  poi,

  /// Cruce o borde de banqueta (`kerb`, `tactile_paving`).
  crossing,
}

/// Un lugar de OpenStreetMap con sus datos de accesibilidad.
class Place {
  const Place({
    required this.osmType,
    required this.osmId,
    required this.name,
    required this.category,
    required this.location,
```

```
    required this.kind,
    this.access = Accessibility.unknown,
    this.accessibleToilet,
    this.tactilePaving,
    this.description,
  });

  /// `node`, `way` o `relation`.
  final String osmType;
  final int osmId;
  final String name;
  final String category;
  final LatLng location;
  final PlaceKind kind;
  final Accessibility access;

  /// `toilets:wheelchair`. Null si no hay dato.
  final bool? accessibleToilet;

  /// `tactile_paving`. Null si no hay dato.
  final bool? tactilePaving;

  /// `wheelchair:description`.
  final String? description;

  String get key => '$osmType/$osmId';

  Uri get osmUrl => Uri.parse('https://www.openstreetmap.org/$osmType/$osmId');

  Map<String, Object?> toJson() => {
        'osmType': osmType,
        'osmId': osmId,
        'name': name,
        'category': category,
        'lat': location.latitude,
        'lon': location.longitude,
        'kind': kind.name,
        'access': access.name,
        'accessibleToilet': accessibleToilet,
        'tactilePaving': tactilePaving,
        'description': description,
      };

  factory Place.fromJson(Map<String, Object?> json) => Place(
        osmType: json['osmType'] as String,
        osmId: json['osmId'] as int,
        name: json['name'] as String,
        category: json['category'] as String,
        location: LatLng(
          (json['lat'] as num).toDouble(),
          (json['lon'] as num).toDouble(),
        ),
        kind: PlaceKind.values.byName(json['kind'] as String),
        access: Accessibility.values.byName(json['access'] as String),
        accessibleToilet: json['accessibleToilet'] as bool?,
        tactilePaving: json['tactilePaving'] as bool?,
        description: json['description'] as String?,
      );
}
```

lib/models/report.dart

Modelo de reporte y su ciclo de vida.

```
import 'package:latlong2/latlong.dart';

enum Severity {
  low('Baja'),
```

```
  medium('Media'),
  high('Alta');

  const Severity(this.label);
  final String label;
}

/// Tipos de peligro con su gravedad por defecto y cuánto duran sin confirmar.
///
/// Lo temporal caduca rápido (un coche se mueve); lo estructural dura más.
enum ReportType {
  blockedRamp('Rampa bloqueada', Severity.high, Duration(hours: 6)),
  openDrain('Coladera o registro abierto', Severity.high, Duration(days: 3)),
  brokenSidewalk('Banqueta rota o levantada', Severity.medium, Duration(days: 30)),
  construction('Obra o desvío', Severity.medium, Duration(days: 14)),
  noRamp('Sin rampa en el cruce', Severity.medium, Duration(days: 90)),
  elevatorOut('Elevador fuera de servicio', Severity.high, Duration(days: 2)),
  flooding('Inundación o encharcamiento', Severity.high, Duration(days: 1)),
  other('Otro peligro', Severity.medium, Duration(days: 7));

  const ReportType(this.label, this.defaultSeverity, this.ttl);
  final String label;
  final Severity defaultSeverity;
  final Duration ttl;
}

/// Un peligro u obstáculo reportado por la comunidad.
///
/// Es inmutable: confirmar o votar devuelve un reporte nuevo.
class Report {
  const Report({
    required this.id,
    required this.type,
    required this.severity,
    required this.location,
    required this.createdAt,
    required this.lastConfirmedAt,
    this.goneVotes = 0,
    this.closed = false,
    this.note = '',
    this.photoPath,
  });

  /// Votos "ya no está" seguidos que cierran un reporte.
  static const votesToClose = 2;

  final String id;
  final ReportType type;
  final Severity severity;
  final LatLng location;
  final DateTime createdAt;

  /// Última vez que alguien dijo "sigue ahí" (o la creación).
  final DateTime lastConfirmedAt;

  /// "Ya no está" seguidos desde la última confirmación.
  final int goneVotes;
  final bool closed;
  final String note;
  final String? photoPath;

  DateTime get expiresAt => lastConfirmedAt.add(type.ttl);

  bool isActive(DateTime now) => !closed && now.isBefore(expiresAt);

  /// "Sigue ahí": reinicia la caducidad y borra los votos en contra.
  Report confirm(DateTime now) =>
      _copy(lastConfirmedAt: now, goneVotes: 0, closed: false);

  /// "Ya no está": con [votesToClose] votos seguidos se cierra.
```

```
  Report voteGone() {
    final votes = goneVotes + 1;
    return _copy(goneVotes: votes, closed: votes >= votesToClose);
  }

  Report _copy({DateTime? lastConfirmedAt, int? goneVotes, bool? closed}) =>
      Report(
        id: id,
        type: type,
        severity: severity,
        location: location,
        createdAt: createdAt,
        lastConfirmedAt: lastConfirmedAt ?? this.lastConfirmedAt,
        goneVotes: goneVotes ?? this.goneVotes,
        closed: closed ?? this.closed,
        note: note,
        photoPath: photoPath,
      );

  Map<String, Object?> toJson() => {
        'id': id,
        'type': type.name,
        'severity': severity.name,
        'lat': location.latitude,
        'lon': location.longitude,
        'createdAt': createdAt.toIso8601String(),
        'lastConfirmedAt': lastConfirmedAt.toIso8601String(),
        'goneVotes': goneVotes,
        'closed': closed,
        'note': note,
        'photoPath': photoPath,
      };

  factory Report.fromJson(Map<String, Object?> json) => Report(
        id: json['id'] as String,
        type: ReportType.values.byName(json['type'] as String),
        severity: Severity.values.byName(json['severity'] as String),
        location: LatLng(
          (json['lat'] as num).toDouble(),
          (json['lon'] as num).toDouble(),
        ),
        createdAt: DateTime.parse(json['createdAt'] as String),
        lastConfirmedAt: DateTime.parse(json['lastConfirmedAt'] as String),
        goneVotes: (json['goneVotes'] as int?) ?? 0,
        closed: (json['closed'] as bool?) ?? false,
        note: (json['note'] as String?) ?? '',
        photoPath: json['photoPath'] as String?,
      );
}
```

lib/models/route_result.dart

Modelo de ruta.

```
import 'package:latlong2/latlong.dart';

/// Un paso de la ruta ("Gira a la derecha en Av. Té").
class RouteStep {
  const RouteStep({
    required this.instruction,
    required this.distanceMeters,
    required this.type,
  });

  final String instruction;
  final double distanceMeters;

  /// Código de openrouteservice (0 izquierda, 1 derecha, 6 recto, 10 llegada…).
```

```
  final int type;
}

/// Ruta calculada hacia un destino.
class RouteResult {
  const RouteResult({
    required this.destinationName,
    required this.points,
    required this.distanceMeters,
    required this.durationSeconds,
    required this.steps,
    required this.avoidedHazards,
  });

  final String destinationName;
  final List<LatLng> points;
  final double distanceMeters;
  final double durationSeconds;
  final List<RouteStep> steps;

  /// Cuántos peligros reportados se pidió evitar.
  final int avoidedHazards;
}
```

lib/models/settings.dart

Ajustes, perfiles y tema.

```
/// Cómo se mueve la persona. Cambia rutas, filtros y guía por voz.
enum MobilityProfile {
  wheelchair(
    'Silla de ruedas',
    'Entradas sin escalón, rampas y pendientes suaves',
    orsProfile: 'wheelchair',
    avoidSteps: true,
    voiceByDefault: false,
  ),
  mobilityAid(
    'Bastón o andadera',
    'Evitar escaleras y pisos irregulares',
    orsProfile: 'foot-walking',
    avoidSteps: true,
    voiceByDefault: false,
  ),
  lowVision(
    'Baja visión',
    'Letra grande, alto contraste y avisos de peligros',
    orsProfile: 'foot-walking',
    avoidSteps: false,
    voiceByDefault: true,
  ),
  blind(
    'Ceguera',
    'Todo por voz: qué hay cerca, peligros y rutas',
    orsProfile: 'foot-walking',
    avoidSteps: false,
    voiceByDefault: true,
  );

  const MobilityProfile(
    this.label,
    this.description, {
    required this.orsProfile,
    required this.avoidSteps,
    required this.voiceByDefault,
  });

  final String label;
```

```
  final String description;

  /// Perfil de openrouteservice para calcular rutas.
  final String orsProfile;
  final bool avoidSteps;
  final bool voiceByDefault;

  /// Quienes usan silla de ruedas solo ven en el mapa lo accesible por defecto.
  bool get onlyAccessibleByDefault => this == MobilityProfile.wheelchair;
}

/// Claro, oscuro o lo que diga el sistema.
enum ThemePreference {
  system('Sistema'),
  light('Claro'),
  dark('Oscuro');

  const ThemePreference(this.label);
  final String label;
}

class AppSettings {
  const AppSettings({
    this.profile,
    this.voiceGuidance = true,
    this.hazardAlerts = true,
    this.onlyAccessible = false,
    this.speechRate = 0.5,
    this.textScale = 1.2,
    this.orsApiKey = '',
    this.theme = ThemePreference.system,
  });

  /// Null hasta que la persona elige su perfil la primera vez.
  final MobilityProfile? profile;
  final bool voiceGuidance;

  /// Aviso por voz y vibración al acercarse a un peligro reportado.
  final bool hazardAlerts;

  /// Ocultar en el mapa lo que no es accesible o no tiene dato.
  final bool onlyAccessible;
  final double speechRate;
  final double textScale;

  /// Clave de openrouteservice escrita por la persona (se guarda solo en
  /// este dispositivo). Si está vacía se usa la de compilación, si hay.
  final String orsApiKey;
  final ThemePreference theme;

  AppSettings copyWith({
    MobilityProfile? profile,
    bool? voiceGuidance,
    bool? hazardAlerts,
    bool? onlyAccessible,
    double? speechRate,
    double? textScale,
    String? orsApiKey,
    ThemePreference? theme,
  }) =>
      AppSettings(
        profile: profile ?? this.profile,
        voiceGuidance: voiceGuidance ?? this.voiceGuidance,
        hazardAlerts: hazardAlerts ?? this.hazardAlerts,
        onlyAccessible: onlyAccessible ?? this.onlyAccessible,
        speechRate: speechRate ?? this.speechRate,
        textScale: textScale ?? this.textScale,
        orsApiKey: orsApiKey ?? this.orsApiKey,
        theme: theme ?? this.theme,
      );
```

```
  Map<String, Object?> toJson() => {
        'profile': profile?.name,
        'voiceGuidance': voiceGuidance,
        'hazardAlerts': hazardAlerts,
        'onlyAccessible': onlyAccessible,
        'speechRate': speechRate,
        'textScale': textScale,
        'orsApiKey': orsApiKey,
        'theme': theme.name,
      };

  factory AppSettings.fromJson(Map<String, Object?> json) {
    final profile = json['profile'] as String?;
    return AppSettings(
      profile: profile == null ? null : MobilityProfile.values.byName(profile),
      voiceGuidance: (json['voiceGuidance'] as bool?) ?? true,
      hazardAlerts: (json['hazardAlerts'] as bool?) ?? true,
      onlyAccessible: (json['onlyAccessible'] as bool?) ?? false,
      speechRate: (json['speechRate'] as num?)?.toDouble() ?? 0.5,
      textScale: (json['textScale'] as num?)?.toDouble() ?? 1.2,
      orsApiKey: (json['orsApiKey'] as String?) ?? '',
      theme: ThemePreference.values.asNameMap()[json['theme']] ??
          ThemePreference.system,
    );
  }
}
```

lib/core/geo.dart

Cálculos geográficos.

```
/// Cálculos geográficos sin dependencias de Flutter (fáciles de probar).
library;

import 'dart:math' as math;

import 'package:latlong2/latlong.dart';

const _earthRadius = 6371000.0;

double _rad(double deg) => deg * math.pi / 180;
double _deg(double rad) => rad * 180 / math.pi;

/// Distancia en metros entre dos puntos (fórmula de haversine).
double distanceMeters(LatLng a, LatLng b) {
  final dLat = _rad(b.latitude - a.latitude);
  final dLon = _rad(b.longitude - a.longitude);
  final h = math.pow(math.sin(dLat / 2), 2) +
      math.cos(_rad(a.latitude)) *
          math.cos(_rad(b.latitude)) *
          math.pow(math.sin(dLon / 2), 2);
  return 2 * _earthRadius * math.asin(math.sqrt(h));
}

/// Rumbo de a hacia b, en grados desde el norte (0 a 360).
double bearingDegrees(LatLng a, LatLng b) {
  final lat1 = _rad(a.latitude), lat2 = _rad(b.latitude);
  final dLon = _rad(b.longitude - a.longitude);
  final y = math.sin(dLon) * math.cos(lat2);
  final x = math.cos(lat1) * math.sin(lat2) -
      math.sin(lat1) * math.cos(lat2) * math.cos(dLon);
  return (_deg(math.atan2(y, x)) + 360) % 360;
}

/// Ángulo en (-180, 180]: negativo a la izquierda, positivo a la derecha.
double relativeAngle(double bearing, double heading) {
  var d = (bearing - heading) % 360;
```

```
  if (d > 180) d -= 360;
  if (d <= -180) d += 360;
  return d;
}

const _cardinals = [
  'al norte',
  'al noreste',
  'al este',
  'al sureste',
  'al sur',
  'al suroeste',
  'al oeste',
  'al noroeste',
];

/// "al norte", "al sureste"… para cuando no se conoce hacia dónde camina.
String cardinalDirection(double bearing) {
  final index = (((bearing % 360) + 22.5) ~/ 45) % 8;
  return _cardinals[index];
}

/// Dirección relativa a hacia dónde camina la persona.
String relativeDirection(double bearing, double heading) {
  final a = relativeAngle(bearing, heading);
  final abs = a.abs();
  if (abs <= 22.5) return 'al frente';
  if (abs >= 157.5) return 'atrás';
  final side = a > 0 ? 'derecha' : 'izquierda';
  if (abs <= 67.5) return 'adelante a la $side';
  if (abs < 112.5) return 'a tu $side';
  return 'atrás a la $side';
}

/// Dirección relativa si hay rumbo; si no, punto cardinal.
String directionFrom(LatLng from, LatLng to, double? heading) {
  final bearing = bearingDegrees(from, to);
  return heading == null
      ? cardinalDirection(bearing)
      : relativeDirection(bearing, heading);
}

/// Redondea para leer en voz alta: 5 m cerca, 10 m lejos, km a partir de 1 km.
String formatDistance(double meters) {
  if (meters < 10) return 'menos de 10 metros';
  if (meters < 100) return '${(meters / 5).round() * 5} metros';
  if (meters < 1000) return '${(meters / 10).round() * 10} metros';
  final km = (meters / 100).round() / 10;
  final text = km == km.roundToDouble()
      ? km.toStringAsFixed(0)
      : km.toStringAsFixed(1);
  return '$text ${km == 1 ? 'kilómetro' : 'kilómetros'}';
}

/// Cuadro de ~[meters] por lado alrededor de un punto, como anillo GeoJSON
/// ([lon, lat], cerrado). Sirve para pedir rutas que lo eviten.
List<List<double>> squareRing(LatLng center, double meters) {
  final dLat = meters / 2 / 111320;
  final dLon = meters / 2 / (111320 * math.cos(_rad(center.latitude)));
  final s = center.latitude - dLat, n = center.latitude + dLat;
  final w = center.longitude - dLon, e = center.longitude + dLon;
  return [
    [w, s],
    [e, s],
    [e, n],
    [w, n],
    [w, s],
  ];
}
```

```
/// Para leer en pantalla: "120 m", "1.2 km".
String formatDistanceShort(double meters) {
  if (meters < 1000) return '${(meters / 10).round() * 10} m';
  final km = (meters / 100).round() / 10;
  return '${km == km.roundToDouble() ? km.toStringAsFixed(0) : km.toStringAsFixed(1)} km';
}

/// Velocidad de paso tranquila (m/s) para estimar tiempos sin ruta.
const walkingSpeed = 1.0;

/// "2 min", "1 h 5 min". Nunca menos de 1 min.
String formatDuration(double seconds) {
  final minutes = (seconds / 60).ceil().clamp(1, 100000);
  if (minutes < 60) return '$minutes min';
  final h = minutes ~/ 60, m = minutes % 60;
  return m == 0 ? '$h h' : '$h h $m min';
}
```

lib/core/osm_parser.dart

Consulta y lectura de OpenStreetMap.

```
import 'package:latlong2/latlong.dart';

import '../models/place.dart';

/// Consulta de Overpass: lugares con dato de accesibilidad, servicios
/// importantes, paradas y cruces con información de rampa o guía podotáctil.
String buildOverpassQuery(LatLng center, int radiusMeters) {
  final around = '(around:$radiusMeters,${center.latitude},${center.longitude})';
  return '''
[out:json][timeout:25];
(
  nwr["wheelchair"]$around;
  nwr["amenity"~"^(pharmacy|hospital|clinic|doctors|dentist|bank|atm|restaurant|cafe|
fast_food|toilets|school|college|university|police|post_office|marketplace|townhall|library|
place_of_worship)\$"]$around;
  nwr["shop"~"^(supermarket|convenience|chemist|bakery|mall|department_store)\$"]$around;
  node["highway"~"^(bus_stop|elevator)\$"]$around;
  nwr["railway"="station"]$around;
  node["kerb"]$around;
  node["tactile_paving"]$around;
);
out center tags;
''';
}

const _amenity = {
  'pharmacy': 'Farmacia',
  'hospital': 'Hospital',
  'clinic': 'Clínica',
  'doctors': 'Consultorio',
  'dentist': 'Dentista',
  'bank': 'Banco',
  'atm': 'Cajero',
  'restaurant': 'Restaurante',
  'cafe': 'Cafetería',
  'fast_food': 'Comida rápida',
  'toilets': 'Baños públicos',
  'school': 'Escuela',
  'college': 'Escuela',
  'university': 'Universidad',
  'police': 'Policía',
  'post_office': 'Correos',
  'marketplace': 'Mercado',
  'townhall': 'Oficina de gobierno',
  'library': 'Biblioteca',
  'place_of_worship': 'Templo',
```

```
};

const _shop = {
  'supermarket': 'Supermercado',
  'convenience': 'Tienda',
  'chemist': 'Farmacia',
  'bakery': 'Panadería',
  'mall': 'Centro comercial',
  'department_store': 'Tienda departamental',
};

/// Nombre de categoría en español a partir de las etiquetas.
String categoryFor(Map<String, String> tags) {
  final amenity = _amenity[tags['amenity']];
  if (amenity != null) return amenity;
  final shop = _shop[tags['shop']];
  if (shop != null) return shop;
  if (tags.containsKey('shop')) return 'Comercio';
  if (tags['highway'] == 'bus_stop') return 'Parada de autobús';
  if (tags['highway'] == 'elevator') return 'Elevador';
  if (tags['railway'] == 'station' || tags['public_transport'] == 'station') {
    return 'Estación';
  }
  if (_isCrossing(tags)) return 'Cruce peatonal';
  if (tags.containsKey('building')) return 'Edificio';
  return 'Lugar';
}

bool _isCrossing(Map<String, String> tags) =>
    tags.containsKey('kerb') ||
    tags['highway'] == 'crossing' ||
    tags['footway'] == 'crossing' ||
    (tags.containsKey('tactile_paving') && !tags.containsKey('name'));

/// Accesibilidad de un cruce según su borde de banqueta (`kerb`).
Accessibility _crossingAccess(Map<String, String> tags) {
  final explicit = tags['wheelchair'];
  if (explicit != null) return Accessibility.fromTag(explicit);
  return switch (tags['kerb']) {
    'flush' || 'no' => Accessibility.yes,
    'lowered' => Accessibility.yes,
    'rolled' => Accessibility.limited,
    'raised' || 'yes' => Accessibility.no,
    _ => Accessibility.unknown,
  };
}

bool? _yesNo(String? value) => switch (value) {
      'yes' || 'designated' => true,
      'no' => false,
      _ => null,
    };

/// Convierte la respuesta JSON de Overpass en lugares.
///
/// Ignora elementos sin coordenadas y los que no aportan nada al usuario
/// (sin nombre, sin categoría útil y sin dato de accesibilidad).
List<Place> parseOverpass(Map<String, Object?> json) {
  final elements = (json['elements'] as List?) ?? const [];
  final places = <Place>[];

  for (final raw in elements) {
    final e = (raw as Map).cast<String, Object?>();
    final tags = ((e['tags'] as Map?) ?? const {}).cast<String, String>();
    if (tags.isEmpty) continue;

    final center = (e['center'] as Map?)?.cast<String, Object?>();
    final lat = (e['lat'] ?? center?['lat']) as num?;
    final lon = (e['lon'] ?? center?['lon']) as num?;
    if (lat == null || lon == null) continue;
```

```
    final crossing = _isCrossing(tags) && !tags.containsKey('name');
    final category = categoryFor(tags);
    final access =
        crossing ? _crossingAccess(tags) : Accessibility.fromTag(tags['wheelchair']);
    final name = tags['name']?.trim();

    if ((name == null || name.isEmpty) &&
        category == 'Lugar' &&
        access == Accessibility.unknown) {
      continue;
    }

    places.add(Place(
      osmType: (e['type'] as String?) ?? 'node',
      osmId: (e['id'] as num).toInt(),
      name: (name == null || name.isEmpty) ? category : name,
      category: category,
      location: LatLng(lat.toDouble(), lon.toDouble()),
      kind: crossing ? PlaceKind.crossing : PlaceKind.poi,
      access: access,
      accessibleToilet: _yesNo(tags['toilets:wheelchair']),
      tactilePaving: _yesNo(tags['tactile_paving']),
      description: tags['wheelchair:description'],
    ));
  }
  return places;
}
```

lib/core/route_api.dart

Petición y respuesta de openrouteservice.

```
import 'package:latlong2/latlong.dart';

import '../models/route_result.dart';
import '../models/settings.dart';
import 'geo.dart';

/// Lado del cuadro que se evita alrededor de cada peligro reportado.
const hazardBoxMeters = 30.0;

/// Peligros más cerca que esto del origen o destino no se envían:
/// openrouteservice rechaza la ruta si el origen cae dentro de una zona a evitar.
const _endpointClearance = 25.0;

/// Cuerpo JSON de la petición a openrouteservice.
///
/// Coordenadas en orden [lon, lat], como pide GeoJSON.
Map<String, Object?> buildRouteRequest({
  required LatLng start,
  required LatLng end,
  required MobilityProfile profile,
  required List<LatLng> hazards,
}) {
  final options = <String, Object?>{};
  if (profile.avoidSteps) options['avoid_features'] = ['steps'];
  if (profile == MobilityProfile.wheelchair) {
    options['profile_params'] = {
      'restrictions': {
        'maximum_incline': 6,
        'maximum_sloped_kerb': 0.06,
        'smoothness_type': 'good',
      },
    };
  }

  final usable = hazards
```

```
      .where((h) =>
          distanceMeters(h, start) > _endpointClearance &&
          distanceMeters(h, end) > _endpointClearance)
      .toList();
  if (usable.isNotEmpty) {
    options['avoid_polygons'] = {
      'type': 'MultiPolygon',
      'coordinates': [
        for (final h in usable) [squareRing(h, hazardBoxMeters)],
      ],
    };
  }

  return {
    'coordinates': [
      [start.longitude, start.latitude],
      [end.longitude, end.latitude],
    ],
    'language': 'es',
    'instructions': true,
    'units': 'm',
    if (options.isNotEmpty) 'options': options,
  };
}

/// Cuántos peligros se mandaron a evitar en [request].
int avoidedCount(Map<String, Object?> request) {
  final options = request['options'] as Map<String, Object?>?;
  final polygons = options?['avoid_polygons'] as Map<String, Object?>?;
  return (polygons?['coordinates'] as List?)?.length ?? 0;
}

/// Lee la respuesta GeoJSON de openrouteservice.
RouteResult parseRouteResponse(
  Map<String, Object?> json, {
  required String destinationName,
  int avoidedHazards = 0,
}) {
  final features = (json['features'] as List?) ?? const [];
  if (features.isEmpty) throw const FormatException('Respuesta sin ruta');
  final feature = (features.first as Map).cast<String, Object?>();

  final geometry = (feature['geometry'] as Map).cast<String, Object?>();
  final coords = (geometry['coordinates'] as List)
      .map((c) => c as List)
      .map((c) => LatLng((c[1] as num).toDouble(), (c[0] as num).toDouble()))
      .toList(growable: false);

  final props = (feature['properties'] as Map).cast<String, Object?>();
  final summary = ((props['summary'] as Map?) ?? const {}).cast<String, Object?>();
  final steps = <RouteStep>[];
  for (final seg in (props['segments'] as List?) ?? const []) {
    for (final s in ((seg as Map)['steps'] as List?) ?? const []) {
      final step = (s as Map).cast<String, Object?>();
      steps.add(RouteStep(
        instruction: (step['instruction'] as String?) ?? '',
        distanceMeters: (step['distance'] as num?)?.toDouble() ?? 0,
        type: (step['type'] as num?)?.toInt() ?? -1,
      ));
    }
  }

  return RouteResult(
    destinationName: destinationName,
    points: coords,
    distanceMeters: (summary['distance'] as num?)?.toDouble() ?? 0,
    durationSeconds: (summary['duration'] as num?)?.toDouble() ?? 0,
    steps: steps,
    avoidedHazards: avoidedHazards,
  );
```

```
}

/// "Gira a la derecha. Sigue 80 metros."
String spokenStep(RouteStep step) {
  final text = step.instruction.trim();
  if (step.type == 10 || step.distanceMeters < 5) return text;
  return '$text. Sigue ${formatDistance(step.distanceMeters)}.';
}

/// Resumen hablado de la ruta.
String spokenSummary(RouteResult route) {
  final minutes = (route.durationSeconds / 60).ceil();
  final avoided = route.avoidedHazards == 0
      ? ''
      : ' Evita ${route.avoidedHazards} '
          '${route.avoidedHazards == 1 ? 'peligro reportado' : 'peligros reportados'}.';
  final time = minutes <= 1 ? 'un minuto' : 'unos $minutes minutos';
  return 'Ruta a ${route.destinationName}: ${formatDistance(route.distanceMeters)}, '
      '$time caminando.$avoided';
}
```

lib/core/hazard_alerts.dart

Selección de avisos de peligro.

```
import 'package:latlong2/latlong.dart';

import '../models/report.dart';
import 'geo.dart';

/// Reportes que conviene anunciar ahora: vigentes, cerca, de gravedad media
/// o alta y sin aviso reciente. El más cercano va primero.
List<Report> hazardsToAnnounce({
  required List<Report> reports,
  required LatLng me,
  required DateTime now,
  required Map<String, DateTime> lastAlerted,
  required double radiusMeters,
  required Duration cooldown,
}) {
  final due = reports.where((r) {
    if (!r.isActive(now) || r.severity == Severity.low) return false;
    if (distanceMeters(me, r.location) > radiusMeters) return false;
    final last = lastAlerted[r.id];
    return last == null || now.difference(last) >= cooldown;
  }).toList()
    ..sort((a, b) => distanceMeters(me, a.location)
        .compareTo(distanceMeters(me, b.location)));
  return due;
}
```

lib/core/voice_commands.dart

Comandos de voz.

```
import 'text_utils.dart';

/// Acciones que se pueden pedir por voz desde el mapa.
enum VoiceCommand { nearby, report, whereAmI, clearRoute, settings, unknown }

const _keywords = <VoiceCommand, List<String>>{
  // Reportar va primero: "hay un peligro cerca" es un reporte, no una consulta.
  VoiceCommand.report: ['reportar', 'reporta', 'peligro', 'obstaculo', 'avisar'],
  VoiceCommand.clearRoute: ['quitar ruta', 'cancelar ruta', 'borrar ruta'],
  VoiceCommand.whereAmI: ['donde estoy', 'mi ubicacion'],
  VoiceCommand.nearby: [
```

```
    'cerca',
    'que hay',
    'alrededor',
    'lugares',
    'buscar',
    'farmacia',
    'bano',
  ],
  VoiceCommand.settings: ['ajuste', 'configuracion', 'perfil'],
};

/// Convierte una frase reconocida en un comando, sin importar acentos.
VoiceCommand parseVoiceCommand(String phrase) {
  final text = normalize(phrase);
  if (text.isEmpty) return VoiceCommand.unknown;
  for (final entry in _keywords.entries) {
    for (final word in entry.value) {
      if (text.contains(word)) return entry.key;
    }
  }
  return VoiceCommand.unknown;
}
```

lib/core/describe.dart

Descripciones habladas.

```
import 'package:latlong2/latlong.dart';

import '../models/place.dart';
import '../models/report.dart';
import 'geo.dart';

/// "hace 5 minutos", "hace 2 horas", "hace 3 días".
String timeAgo(DateTime then, DateTime now) {
  final d = now.difference(then);
  if (d.inMinutes < 2) return 'hace un momento';
  if (d.inMinutes < 60) return 'hace ${d.inMinutes} minutos';
  if (d.inHours < 24) {
    return d.inHours == 1 ? 'hace una hora' : 'hace ${d.inHours} horas';
  }
  return d.inDays == 1 ? 'hace un día' : 'hace ${d.inDays} días';
}

/// "en 3 horas", "en 2 días".
String timeUntil(DateTime when, DateTime now) {
  final d = when.difference(now);
  if (d.isNegative) return 'ya caducó';
  if (d.inMinutes < 60) return 'en ${d.inMinutes} minutos';
  if (d.inHours < 24) {
    return d.inHours == 1 ? 'en una hora' : 'en ${d.inHours} horas';
  }
  return d.inDays == 1 ? 'en un día' : 'en ${d.inDays} días';
}

/// "a 40 metros, a tu derecha". Vacío si no se conoce la ubicación.
String whereIs(LatLng target, LatLng? me, double? heading) {
  if (me == null) return '';
  final meters = distanceMeters(me, target);
  return 'a ${formatDistance(meters)}, ${directionFrom(me, target, heading)}';
}

/// Lo que se lee en voz alta de un lugar.
String describePlace(Place p, {LatLng? me, double? heading}) {
  final parts = <String>[
    p.name == p.category ? p.name : '${p.name}, ${p.category.toLowerCase()}',
    '${p.access.label}: ${p.access.explanation}',
  ];
```

```
  if (p.accessibleToilet == true) parts.add('Tiene baño accesible');
  if (p.accessibleToilet == false) parts.add('Su baño no es accesible');
  if (p.tactilePaving == true) parts.add('Tiene guía podotáctil');
  final desc = p.description?.trim();
  if (desc != null && desc.isNotEmpty) parts.add(desc);
  final where = whereIs(p.location, me, heading);
  if (where.isNotEmpty) parts.add(_capitalize(where));
  return '${parts.join('. ')}.';
}

/// Lo que se lee en voz alta de un reporte.
String describeReport(Report r, DateTime now, {LatLng? me, double? heading}) {
  final parts = <String>[
    '${r.type.label}, gravedad ${r.severity.label.toLowerCase()}',
  ];
  final where = whereIs(r.location, me, heading);
  if (where.isNotEmpty) parts.add(_capitalize(where));
  final note = r.note.trim();
  if (note.isNotEmpty) parts.add(_capitalize(note));
  parts.add('Confirmado ${timeAgo(r.lastConfirmedAt, now)}');
  return '${parts.join('. ')}.';
}

String _capitalize(String s) =>
    s.isEmpty ? s : s[0].toUpperCase() + s.substring(1);
```

lib/core/text_utils.dart

Utilidades de texto.

```
/// Utilidades de texto sin dependencias de Flutter (fáciles de probar).
library;

const _accents = {
  'á': 'a', 'é': 'e', 'í': 'i', 'ó': 'o', 'ú': 'u', 'ü': 'u', 'ñ': 'n',
};

/// Minúsculas y sin acentos: "Medicinas Ñ" -> "medicinas n".
String normalize(String input) {
  final lower = input.toLowerCase().trim();
  final buffer = StringBuffer();
  for (final rune in lower.runes) {
    final ch = String.fromCharCode(rune);
    buffer.write(_accents[ch] ?? ch);
  }
  return buffer.toString();
}

/// "PARACETAMOL" -> "Paracetamol"; "ácido acetilsalicílico" -> "Ácido Acetilsalicílico".
String titleCase(String input) {
  return input
      .trim()
      .split(RegExp(r'\s+'))
      .where((w) => w.isNotEmpty)
      .map((w) => w[0].toUpperCase() + w.substring(1).toLowerCase())
      .join(' ');
}

/// Junta las líneas del OCR en un texto natural para leer en voz alta:
/// une palabras cortadas con guion y quita espacios repetidos.
String cleanForSpeech(String raw) {
  return raw
      .replaceAll(RegExp(r'-\n'), '')
      .replaceAll(RegExp(r'[ \t]+'), ' ')
      .replaceAll(RegExp(r'\n{2,}'), '\n')
      .trim();
}
```

lib/core/theme.dart

Tema claro y oscuro.

```
import 'package:flutter/material.dart';

/// Colores del diseño. Los de estado (accesible, peligro…) son iguales en
/// claro y oscuro: van sobre un aro blanco, así que contrastan en ambos.
class AppTheme {
  static const fontFamily = 'Atkinson Hyperlegible';

  // Estado de accesibilidad: cada uno va siempre con ícono y forma propios.
  static const accessYes = Color(0xFF157F3D);
  static const accessLimited = Color(0xFFA16207);
  static const accessNo = Color(0xFFB42318);
  static const accessUnknown = Color(0xFF5B6773);
  static const hazard = Color(0xFFC2410C);
  static const userDot = Color(0xFF1D4ED8);

  /// Altura mínima de cualquier control tocable.
  static const minTouch = 48.0;

  static ThemeData light() => _build(RLColors.light, Brightness.light);
  static ThemeData dark() => _build(RLColors.dark, Brightness.dark);

  static ThemeData _build(RLColors c, Brightness b) {
    final scheme = ColorScheme(
      brightness: b,
      primary: c.primary,
      onPrimary: c.onPrimary,
      primaryContainer: c.primarySoft,
      onPrimaryContainer: c.primary,
      secondary: c.primary,
      onSecondary: c.onPrimary,
      error: accessNo,
      onError: Colors.white,
      surface: c.surface,
      onSurface: c.text,
      onSurfaceVariant: c.text2,
      surfaceContainerHighest: c.surface2,
      outline: c.border,
      outlineVariant: c.border,
    );

    TextStyle t(double size, [FontWeight w = FontWeight.w400]) => TextStyle(
        fontFamily: fontFamily, fontSize: size, fontWeight: w, color: c.text, height: 1.3);

    final shape = RoundedRectangleBorder(borderRadius: BorderRadius.circular(16));

    return ThemeData(
      useMaterial3: true,
      brightness: b,
      colorScheme: scheme,
      fontFamily: fontFamily,
      scaffoldBackgroundColor: c.background,
      extensions: [c],
      textTheme: TextTheme(
        displaySmall: t(32, FontWeight.w700),
        headlineMedium: t(28, FontWeight.w700),
        headlineSmall: t(22, FontWeight.w700),
        titleLarge: t(19, FontWeight.w700),
        titleMedium: t(17, FontWeight.w700),
        bodyLarge: t(17),
        bodyMedium: t(15),
        labelLarge: t(17, FontWeight.w700),
      ),
      appBarTheme: AppBarTheme(
        backgroundColor: c.background,
        foregroundColor: c.text,
```

```
      elevation: 0,
      scrolledUnderElevation: 1,
      centerTitle: false,
      titleTextStyle: t(22, FontWeight.w700),
    ),
    elevatedButtonTheme: ElevatedButtonThemeData(
      style: ElevatedButton.styleFrom(
        backgroundColor: c.primary,
        foregroundColor: c.onPrimary,
        elevation: 0,
        minimumSize: const Size.fromHeight(56),
        textStyle: t(18, FontWeight.w700),
        shape: shape,
      ),
    ),
    filledButtonTheme: FilledButtonThemeData(
      style: FilledButton.styleFrom(
        backgroundColor: c.primarySoft,
        foregroundColor: c.primary,
        minimumSize: const Size(minTouch, minTouch),
        textStyle: t(16, FontWeight.w700),
        shape: const StadiumBorder(),
      ),
    ),
    outlinedButtonTheme: OutlinedButtonThemeData(
      style: OutlinedButton.styleFrom(
        foregroundColor: c.text,
        minimumSize: const Size.fromHeight(52),
        side: BorderSide(color: c.border),
        textStyle: t(16, FontWeight.w700),
        shape: shape,
      ),
    ),
    textButtonTheme: TextButtonThemeData(
      style: TextButton.styleFrom(
        foregroundColor: c.primary,
        minimumSize: const Size(minTouch, minTouch),
        textStyle: t(16, FontWeight.w700),
      ),
    ),
    inputDecorationTheme: InputDecorationTheme(
      filled: true,
      fillColor: c.surface,
      labelStyle: t(16).copyWith(color: c.text2),
      hintStyle: t(16).copyWith(color: c.text2),
      contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 16),
      border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(14),
        borderSide: BorderSide(color: c.border),
      ),
      enabledBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(14),
        borderSide: BorderSide(color: c.border),
      ),
      focusedBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(14),
        borderSide: BorderSide(color: c.primary, width: 2),
      ),
    ),
    cardTheme: CardThemeData(
      color: c.surface,
      elevation: 0,
      margin: EdgeInsets.zero,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(16),
        side: BorderSide(color: c.border),
      ),
    ),
    dividerTheme: DividerThemeData(color: c.border, space: 1, thickness: 1),
    listTileTheme: ListTileThemeData(
```

```
        textColor: c.text,
        iconColor: c.text,
        minVerticalPadding: 12,
      ),
      switchTheme: SwitchThemeData(
        thumbColor: WidgetStateProperty.resolveWith((s) =>
            s.contains(WidgetState.selected) ? c.onPrimary : c.text2),
        trackColor: WidgetStateProperty.resolveWith((s) =>
            s.contains(WidgetState.selected) ? c.primary : c.surface2),
      ),
      sliderTheme: SliderThemeData(
        activeTrackColor: c.primary,
        thumbColor: c.primary,
        inactiveTrackColor: c.surface2,
        trackHeight: 6,
      ),
      bottomSheetTheme: BottomSheetThemeData(
        backgroundColor: c.surface,
        showDragHandle: true,
        dragHandleColor: c.handle,
        shape: const RoundedRectangleBorder(
          borderRadius: BorderRadius.vertical(top: Radius.circular(24)),
        ),
      ),
      snackBarTheme: SnackBarThemeData(
        behavior: SnackBarBehavior.floating,
        backgroundColor: c.text,
        contentTextStyle: t(16).copyWith(color: c.background),
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
      ),
      progressIndicatorTheme: ProgressIndicatorThemeData(color: c.primary),
    );
  }
}

/// Tokens de color de la app. Se leen con `context.rl`.
@immutable
class RLColors extends ThemeExtension<RLColors> {
  const RLColors({
    required this.background,
    required this.surface,
    required this.surface2,
    required this.border,
    required this.handle,
    required this.text,
    required this.text2,
    required this.primary,
    required this.primarySoft,
    required this.onPrimary,
    required this.okSoft,
    required this.okText,
    required this.hazardSoft,
    required this.hazardText,
  });

  final Color background;
  final Color surface;
  final Color surface2;
  final Color border;
  final Color handle;
  final Color text;

  /// Texto secundario: cumple 4.5:1 sobre `surface`.
  final Color text2;
  final Color primary;
  final Color primarySoft;
  final Color onPrimary;
  final Color okSoft;
  final Color okText;
  final Color hazardSoft;
```

```
  final Color hazardText;

  static const light = RLColors(
    background: Color(0xFFF4F6F8),
    surface: Color(0xFFFFFFFF),
    surface2: Color(0xFFEEF1F4),
    border: Color(0xFFD5DBE1),
    handle: Color(0xFFC3CAD2),
    text: Color(0xFF10202E),
    text2: Color(0xFF4B5966),
    primary: Color(0xFF1D4ED8),
    primarySoft: Color(0xFFE6EDFC),
    onPrimary: Color(0xFFFFFFFF),
    okSoft: Color(0xFFE7F4EC),
    okText: Color(0xFF0F5B2C),
    hazardSoft: Color(0xFFFDEEE6),
    hazardText: Color(0xFF8A2C07),
  );

  static const dark = RLColors(
    background: Color(0xFF0F151B),
    surface: Color(0xFF161D25),
    surface2: Color(0xFF1F2832),
    border: Color(0xFF2C3742),
    handle: Color(0xFF3A4652),
    text: Color(0xFFE6EDF3),
    text2: Color(0xFFA9B6C2),
    primary: Color(0xFF8AB4FF),
    primarySoft: Color(0xFF1E2D4A),
    onPrimary: Color(0xFF0B1B33),
    okSoft: Color(0xFF12301F),
    okText: Color(0xFF8EE0AE),
    hazardSoft: Color(0xFF3A1D10),
    hazardText: Color(0xFFFFB48F),
  );

  @override
  RLColors copyWith() => this;

  @override
  RLColors lerp(ThemeExtension<RLColors>? other, double t) =>
      other is RLColors && t >= 0.5 ? other : this;
}

extension RLColorsX on BuildContext {
  RLColors get rl => Theme.of(this).extension<RLColors>() ?? RLColors.light;
}
```

lib/services/location_service.dart

Servicio de ubicación.

```
import 'dart:async';

import 'package:flutter/foundation.dart';
import 'package:geolocator/geolocator.dart';
import 'package:latlong2/latlong.dart';

/// Posición y rumbo de la persona.
///
/// El rumbo sale del GPS mientras camina (no hay brújula); quieta, se
/// usan puntos cardinales en las indicaciones.
class LocationService {
  final ValueNotifier<LatLng?> position = ValueNotifier(null);

  /// Precisión de la última lectura en metros.
  double? accuracy;
```

```
  /// Hacia dónde camina, en grados; null si está quieta o no se sabe.
  double? heading;

  StreamSubscription<Position>? _sub;

  /// Por debajo de esta velocidad (m/s) el rumbo del GPS no es confiable.
  static const _minWalkingSpeed = 0.5;

  /// Pide permiso y empieza a seguir la ubicación. Devuelve un mensaje
  /// para el usuario si no se puede.
  Future<String?> start() async {
    if (!await Geolocator.isLocationServiceEnabled()) {
      return 'La ubicación del teléfono está apagada.';
    }
    var permission = await Geolocator.checkPermission();
    if (permission == LocationPermission.denied) {
      permission = await Geolocator.requestPermission();
    }
    if (permission == LocationPermission.denied ||
        permission == LocationPermission.deniedForever) {
      return 'Ruta Libre necesita tu ubicación para mostrar qué hay cerca.';
    }

    await _sub?.cancel();
    _sub = Geolocator.getPositionStream(
      locationSettings: const LocationSettings(
        accuracy: LocationAccuracy.best,
        distanceFilter: 3,
      ),
    ).listen(_onPosition, onError: (Object _) {});
    return null;
  }

  void _onPosition(Position p) {
    accuracy = p.accuracy;
    final moving = p.speed >= _minWalkingSpeed && p.heading >= 0;
    heading = moving ? p.heading : null;
    position.value = LatLng(p.latitude, p.longitude);
  }

  void dispose() {
    _sub?.cancel();
    position.dispose();
  }
}
```

lib/services/places_service.dart

Servicio de lugares con caché.

```
import 'dart:convert';

import 'package:flutter/foundation.dart';
import 'package:http/http.dart' as http;
import 'package:latlong2/latlong.dart';
import 'package:shared_preferences/shared_preferences.dart';

import '../config.dart';
import '../core/osm_parser.dart';
import '../models/place.dart';

class PlacesException implements Exception {
  const PlacesException(this.message);
  final String message;
  @override
  String toString() => message;
}
```

```
/// Descarga lugares de OpenStreetMap por zona y los guarda en el teléfono.
///
/// La API pública de Overpass no debe usarse como backend de una app, así
/// que cada zona se descarga una sola vez y se reutiliza por
/// [AppConfig.placesCacheTtl].
class PlacesService {
  PlacesService(this._prefs, {http.Client? client})
      : _client = client ?? http.Client();

  final SharedPreferences _prefs;
  final http.Client _client;

  /// Zona redondeada a ~100 m para que pequeños cambios usen la misma caché.
  String _zoneKey(LatLng c, int r) =>
      'zone_${c.latitude.toStringAsFixed(3)}_${c.longitude.toStringAsFixed(3)}_$r';

  /// Lugares guardados de la zona, aunque estén vencidos (null si no hay).
  List<Place>? cached(LatLng center, int radius) {
    final raw = _prefs.getString(_zoneKey(center, radius));
    if (raw == null) return null;
    final json = (jsonDecode(raw) as Map).cast<String, Object?>();
    return (json['places'] as List)
        .map((e) => Place.fromJson((e as Map).cast<String, Object?>()))
        .toList();
  }

  bool isFresh(LatLng center, int radius) {
    final raw = _prefs.getString(_zoneKey(center, radius));
    if (raw == null) return false;
    final json = (jsonDecode(raw) as Map).cast<String, Object?>();
    final saved = DateTime.parse(json['savedAt'] as String);
    return DateTime.now().difference(saved) < AppConfig.placesCacheTtl;
  }

  /// Lugares de la zona: de la caché si está vigente; si no, de Overpass.
  Future<List<Place>> load(
    LatLng center, {
    int radius = AppConfig.pilotRadiusMeters,
    bool force = false,
  }) async {
    if (!force && isFresh(center, radius)) return cached(center, radius)!;
    try {
      final places = await _download(center, radius);
      await _prefs.setString(
        _zoneKey(center, radius),
        jsonEncode({
          'savedAt': DateTime.now().toIso8601String(),
          'places': places.map((p) => p.toJson()).toList(),
        }),
      );
      return places;
    } on PlacesException {
      // Sin red o servidor ocupado: mejor datos viejos que nada.
      final old = cached(center, radius);
      if (old != null) return old;
      rethrow;
    }
  }

  Future<List<Place>> _download(LatLng center, int radius) async {
    final http.Response res;
    try {
      res = await _client
          .post(
            Uri.parse(AppConfig.overpassUrl),
            headers: kIsWeb ? null : {'User-Agent': AppConfig.userAgent},
            body: {'data': buildOverpassQuery(center, radius)},
          )
          .timeout(const Duration(seconds: 40));
    } on Exception {
```

```
      throw const PlacesException(
          'No hay conexión para descargar los lugares de esta zona.');
    }
    if (res.statusCode == 429 || res.statusCode == 504) {
      throw const PlacesException(
          'El servidor de OpenStreetMap está ocupado. Intenta en un minuto.');
    }
    if (res.statusCode != 200) {
      throw PlacesException(
          'No se pudieron descargar los lugares (error ${res.statusCode}).');
    }
    final json = (jsonDecode(utf8.decode(res.bodyBytes)) as Map)
        .cast<String, Object?>();
    return parseOverpass(json);
  }
}
```

lib/services/report_repository.dart

Repositorio de reportes.

```
import 'dart:convert';

import 'package:shared_preferences/shared_preferences.dart';

import '../models/report.dart';

/// Dónde viven los reportes.
///
/// El prototipo usa [LocalReportRepository]. Para compartirlos entre
/// usuarios basta otra implementación (p. ej. API Gateway + Lambda +
/// DynamoDB) sin cambiar ninguna pantalla.
abstract interface class ReportRepository {
  Future<List<Report>> loadAll();
  Future<void> save(Report report);
}

/// Guarda los reportes en el teléfono como JSON.
class LocalReportRepository implements ReportRepository {
  LocalReportRepository(this._prefs);

  static const _key = 'reports';

  /// Reportes cerrados o caducados que se conservan antes de borrarse.
  static const _keepAfterExpiry = Duration(days: 30);

  final SharedPreferences _prefs;

  @override
  Future<List<Report>> loadAll() async {
    final raw = _prefs.getString(_key);
    if (raw == null) return [];
    final list = jsonDecode(raw) as List;
    final now = DateTime.now();
    return list
        .map((e) => Report.fromJson((e as Map).cast<String, Object?>()))
        .where((r) => now.difference(r.expiresAt) < _keepAfterExpiry)
        .toList();
  }

  @override
  Future<void> save(Report report) async {
    final all = await loadAll();
    final index = all.indexWhere((r) => r.id == report.id);
    if (index == -1) {
      all.add(report);
    } else {
      all[index] = report;
```

```
    }
    await _prefs.setString(_key, jsonEncode(all.map((r) => r.toJson()).toList()));
  }
}
```

lib/services/routing_service.dart

Cliente de rutas.

```
import 'dart:convert';

import 'package:http/http.dart' as http;
import 'package:latlong2/latlong.dart';

import '../config.dart';
import '../core/route_api.dart';
import '../models/route_result.dart';
import '../models/settings.dart';

class RoutingException implements Exception {
  const RoutingException(this.message);
  final String message;
  @override
  String toString() => message;
}

/// Pide rutas a pie o en silla de ruedas a openrouteservice.
class RoutingService {
  RoutingService({http.Client? client}) : _client = client ?? http.Client();

  final http.Client _client;

  Future<RouteResult> route({
    required String apiKey,
    required LatLng start,
    required LatLng end,
    required String destinationName,
    required MobilityProfile profile,
    required List<LatLng> hazards,
  }) async {
    if (apiKey.trim().isEmpty) {
      throw const RoutingException(
          'Falta la clave de openrouteservice. Agrégala en Ajustes, '
          'en la sección Rutas.');
    }

    final body = buildRouteRequest(
      start: start,
      end: end,
      profile: profile,
      hazards: hazards,
    );

    final http.Response res;
    try {
      res = await _client
          .post(
            Uri.parse('${AppConfig.orsBaseUrl}/${profile.orsProfile}/geojson'),
            headers: {
              'Authorization': apiKey.trim(),
              'Content-Type': 'application/json; charset=utf-8',
              'Accept': 'application/geo+json, application/json',
            },
            body: jsonEncode(body),
          )
          .timeout(const Duration(seconds: 25));
    } on Exception {
      throw const RoutingException('No hay conexión para calcular la ruta.');
```

```
    }

    final json = _decode(res);
    if (res.statusCode == 200) {
      try {
        return parseRouteResponse(
          json,
          destinationName: destinationName,
          avoidedHazards: avoidedCount(body),
        );
      } on Object {
        throw const RoutingException('La respuesta de la ruta no se pudo leer.');
      }
    }
    if (res.statusCode == 429) {
      throw const RoutingException(
          'Se alcanzó el límite de rutas por ahora. Intenta en un minuto.');
    }
    if (res.statusCode == 401 || res.statusCode == 403) {
      throw const RoutingException('La clave de openrouteservice no es válida.');
    }
    final message = ((json['error'] as Map?)?['message'] as String?) ?? '';
    throw RoutingException(message.isEmpty
        ? 'No se encontró una ruta accesible hasta ese lugar.'
        : 'No se encontró una ruta accesible ($message).');
  }

  Map<String, Object?> _decode(http.Response res) {
    try {
      return (jsonDecode(utf8.decode(res.bodyBytes)) as Map)
          .cast<String, Object?>();
    } on FormatException {
      return const {};
    }
  }
}
```

lib/services/voice_service.dart

Voz: hablar y escuchar.

```
import 'dart:async';

import 'package:flutter/foundation.dart';
import 'package:flutter/services.dart';
import 'package:flutter_tts/flutter_tts.dart';
import 'package:speech_to_text/speech_to_text.dart';

/// Habla (texto a voz) y escucha (voz a texto) en español de México.
class VoiceService {
  VoiceService({FlutterTts? tts, SpeechToText? stt})
      : _tts = tts ?? FlutterTts(),
        _stt = stt ?? SpeechToText();

  static const _language = 'es-MX';

  final FlutterTts _tts;
  final SpeechToText _stt;

  bool _sttReady = false;
  String? _sttLocaleId;

  /// Si es falso, [announce] no dice nada (el usuario lo desactivó).
  bool guidanceEnabled = true;

  /// Indica si el micrófono está escuchando, para que la UI lo muestre.
  final ValueNotifier<bool> listening = ValueNotifier(false);
```

```
Future<void> init({required double speechRate}) async {
  // Cada ajuste va por separado: si un motor de voz (o el navegador) no
  // soporta uno, los demás siguen aplicando y la app no se detiene.
  Future<void> attempt(Future<Object?> Function() call) async {
    try {
      await call();
    } catch (e) {
      debugPrint('Ajuste de voz no soportado: $e');
    }
  }

  // Si es-MX no está instalado, el motor usa su español por defecto.
  var language = 'es-ES';
  await attempt(() async {
    if (await _tts.isLanguageAvailable(_language) == true) language = _language;
    return null;
  });
  await attempt(() => _tts.setLanguage(language));
  await attempt(() => _tts.setSpeechRate(speechRate));
  await attempt(() => _tts.setVolume(1.0));
  await attempt(() => _tts.awaitSpeakCompletion(true));
  if (!kIsWeb && defaultTargetPlatform == TargetPlatform.iOS) {
    await attempt(() => _tts.setSharedInstance(true));
  }
}

Future<void> setSpeechRate(double rate) => _tts.setSpeechRate(rate);

/// Lee siempre en voz alta, interrumpiendo lo anterior.
///
/// Espera a que termine de hablar, pero con un tope: en algunos navegadores
/// el motor de voz nunca avisa que terminó y la app no debe quedarse
/// detenida por eso.
Future<void> speak(String text) async {
  if (text.trim().isEmpty) return;
  // ~70 ms por letra en voz pausada, entre 3 y 30 segundos.
  final limit = Duration(
      milliseconds: (text.length * 70).clamp(3000, 30000).toInt());
  try {
    await _tts.stop().timeout(const Duration(seconds: 2));
    await _tts.speak(text).timeout(limit);
  } on TimeoutException {
    // Se sigue: la voz puede seguir sonando, pero la app no se bloquea.
  } catch (e) {
    debugPrint('No se pudo hablar: $e');
  }
}

/// Anuncios de navegación: solo si la guía por voz está activa.
Future<void> announce(String text) async {
  if (guidanceEnabled) await speak(text);
}

Future<void> stop() => _tts.stop();

/// Escucha una frase y la devuelve, o null si no se entendió nada.
Future<String?> listenOnce({
  Duration maxDuration = const Duration(seconds: 8),
}) async {
  if (!_sttReady) {
    try {
      _sttReady = await _stt.initialize();
    } catch (e) {
      debugPrint('Reconocimiento de voz no disponible: $e');
      _sttReady = false;
    }
    if (!_sttReady) return null;
    final locales = await _stt.locales();
    final spanish = locales.where((l) => l.localeId.startsWith('es'));
    _sttLocaleId = spanish
```

```
          .firstWhere(
            (l) => l.localeId.replaceAll('-', '_') == 'es_MX',
            orElse: () => spanish.isNotEmpty
                ? spanish.first
                : LocaleName(_language, 'Español'),
          )
          .localeId;
    }

    await _tts.stop();
    await HapticFeedback.mediumImpact();

    String? finalWords;
    listening.value = true;
    await _stt.listen(
      onResult: (result) {
        if (result.finalResult) finalWords = result.recognizedWords;
      },
      listenOptions: SpeechListenOptions(
        localeId: _sttLocaleId,
        listenFor: maxDuration,
        pauseFor: const Duration(seconds: 3),
        partialResults: false,
        cancelOnError: true,
        listenMode: ListenMode.confirmation,
      ),
    );

    // Espera a que el reconocedor termine (por pausa, tiempo o error).
    final deadline = DateTime.now().add(maxDuration + const Duration(seconds: 2));
    while (_stt.isListening && DateTime.now().isBefore(deadline)) {
      await Future<void>.delayed(const Duration(milliseconds: 150));
    }
    await _stt.stop();
    // El resultado final puede llegar justo después de detenerse.
    await Future<void>.delayed(const Duration(milliseconds: 300));
    listening.value = false;

    final words = finalWords?.trim();
    return (words == null || words.isEmpty) ? null : words;
  }

  void dispose() {
    _tts.stop();
    _stt.cancel();
    listening.dispose();
  }
}
```

lib/widgets/base_map.dart

Mapa base, línea de ruta y crédito.

```
import 'package:flutter/material.dart';
import 'package:flutter_map/flutter_map.dart';
import 'package:url_launcher/url_launcher.dart';

import '../config.dart';
import '../core/theme.dart';
import '../models/route_result.dart';

/// Mosaicos de OpenStreetMap. En modo oscuro se invierten los colores para
/// que el mapa no deslumbre.
TileLayer osmTiles(BuildContext context) => TileLayer(
      urlTemplate: 'https://tile.openstreetmap.org/{z}/{x}/{y}.png',
      userAgentPackageName: AppConfig.packageName,
      maxZoom: 19,
      tileBuilder: Theme.of(context).brightness == Brightness.dark
```

```
          ? darkModeTileBuilder
          : null,
    );

/// Línea de la ruta con borde para que se lea sobre cualquier calle.
PolylineLayer routeLine(BuildContext context, RouteResult route) {
  final dark = Theme.of(context).brightness == Brightness.dark;
  return PolylineLayer(
    polylines: [
      Polyline(
        points: route.points,
        strokeWidth: 7,
        color: context.rl.primary,
        borderStrokeWidth: 3,
        borderColor: dark ? const Color(0xFF0B1B33) : Colors.white,
      ),
    ],
  );
}

/// Crédito de OpenStreetMap, siempre visible sobre el mapa.
class OsmCredit extends StatelessWidget {
  const OsmCredit({super.key});

  @override
  Widget build(BuildContext context) {
    final rl = context.rl;
    return Semantics(
      link: true,
      label: 'Datos del mapa: colaboradores de OpenStreetMap',
      excludeSemantics: true,
      child: Material(
        color: rl.surface.withValues(alpha: 0.9),
        borderRadius: BorderRadius.circular(8),
        child: InkWell(
          borderRadius: BorderRadius.circular(8),
          onTap: () =>
              launchUrl(Uri.parse('https://www.openstreetmap.org/copyright')),
          child: Padding(
            padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
            child: Text('© OpenStreetMap',
                style: TextStyle(fontSize: 12, color: rl.text2)),
          ),
        ),
      ),
    );
  }
}
```

lib/widgets/map_markers.dart

Marcadores y leyenda.

```
import 'package:flutter/material.dart';

import '../core/theme.dart';
import '../models/place.dart';
import '../models/report.dart';

Color accessColor(Accessibility a) => switch (a) {
      Accessibility.yes => AppTheme.accessYes,
      Accessibility.limited => AppTheme.accessLimited,
      Accessibility.no => AppTheme.accessNo,
      Accessibility.unknown => AppTheme.accessUnknown,
    };

/// Ícono distinto por estado: así se distingue sin depender del color.
IconData accessIcon(Accessibility a) => switch (a) {
```

```
      Accessibility.yes => Icons.check,
      Accessibility.limited => Icons.remove,
      Accessibility.no => Icons.close,
      Accessibility.unknown => Icons.question_mark,
    };

Color hazardColor(Severity s) =>
    s == Severity.low ? AppTheme.accessLimited : AppTheme.hazard;

const _markerShadow = [
  BoxShadow(color: Color(0x4D000000), blurRadius: 6, offset: Offset(0, 2)),
];

/// Círculo de estado con su ícono (en el mapa y en las listas).
class AccessBadge extends StatelessWidget {
  const AccessBadge({
    super.key,
    required this.access,
    this.size = 40,
    this.ring = false,
    this.square = false,
  });

  final Accessibility access;
  final double size;

  /// Aro blanco y sombra, para que resalte sobre el mapa.
  final bool ring;

  /// Los cruces son cuadrados; los lugares, círculos.
  final bool square;

  @override
  Widget build(BuildContext context) {
    return Container(
      width: size,
      height: size,
      decoration: BoxDecoration(
        color: accessColor(access),
        shape: square ? BoxShape.rectangle : BoxShape.circle,
        borderRadius: square ? BorderRadius.circular(size * 0.22) : null,
        border: ring ? Border.all(color: Colors.white, width: 3) : null,
        boxShadow: ring ? _markerShadow : null,
      ),
      child: Icon(accessIcon(access), color: Colors.white, size: size * 0.5),
    );
  }
}

/// Lugar en el mapa (cuadrado si es un cruce).
class PlaceMarker extends StatelessWidget {
  const PlaceMarker({
    super.key,
    required this.place,
    required this.semanticLabel,
    required this.onTap,
  });

  final Place place;
  final String semanticLabel;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    final crossing = place.kind == PlaceKind.crossing;
    return Semantics(
      button: true,
      label: semanticLabel,
      excludeSemantics: true,
      child: GestureDetector(
```

```
        onTap: onTap,
        behavior: HitTestBehavior.opaque,
        child: Center(
          child: AccessBadge(
            access: place.access,
            size: crossing ? 26 : 40,
            ring: true,
            square: crossing,
          ),
        ),
      ),
    );
  }
}

/// Triángulo de peligro con aro blanco.
class HazardTriangle extends StatelessWidget {
  const HazardTriangle({super.key, required this.color, this.size = 44});

  final Color color;
  final double size;

  @override
  Widget build(BuildContext context) {
    return SizedBox(
      width: size,
      height: size * 0.92,
      child: CustomPaint(
        painter: _TrianglePainter(color),
        child: Align(
          alignment: const Alignment(0, 0.45),
          child: Icon(Icons.priority_high, color: Colors.white, size: size * 0.42),
        ),
      ),
    );
  }
}

class _TrianglePainter extends CustomPainter {
  _TrianglePainter(this.color);
  final Color color;

  @override
  void paint(Canvas canvas, Size size) {
    final w = size.width, h = size.height;
    final path = Path()
      ..moveTo(w / 2, 3)
      ..lineTo(w - 3, h - 3)
      ..lineTo(3, h - 3)
      ..close();
    canvas.drawShadow(path, Colors.black, 3, false);
    canvas.drawPath(path, Paint()..color = color);
    canvas.drawPath(
      path,
      Paint()
        ..color = Colors.white
        ..style = PaintingStyle.stroke
        ..strokeWidth = 3
        ..strokeJoin = StrokeJoin.round,
    );
  }

  @override
  bool shouldRepaint(_TrianglePainter old) => old.color != color;
}

/// Peligro en el mapa; más grande si la gravedad es alta.
class HazardMarker extends StatelessWidget {
  const HazardMarker({
    super.key,
```

```
    required this.report,
    required this.semanticLabel,
    required this.onTap,
  });

  final Report report;
  final String semanticLabel;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return Semantics(
      button: true,
      label: semanticLabel,
      excludeSemantics: true,
      child: GestureDetector(
        onTap: onTap,
        behavior: HitTestBehavior.opaque,
        child: Center(
          child: HazardTriangle(
            color: hazardColor(report.severity),
            size: report.severity == Severity.high ? 48 : 40,
          ),
        ),
      ),
    );
  }
}

/// Punto azul de "tú estás aquí", con halo.
class UserMarker extends StatelessWidget {
  const UserMarker({super.key});

  @override
  Widget build(BuildContext context) {
    return Semantics(
      label: 'Tu ubicación',
      child: Container(
        decoration: BoxDecoration(
          color: AppTheme.userDot.withValues(alpha: 0.22),
          shape: BoxShape.circle,
        ),
        alignment: Alignment.center,
        child: Container(
          width: 18,
          height: 18,
          decoration: BoxDecoration(
            color: AppTheme.userDot,
            shape: BoxShape.circle,
            border: Border.all(color: Colors.white, width: 3),
            boxShadow: _markerShadow,
          ),
        ),
      ),
    );
  }
}

/// Leyenda de colores e íconos, para quien ve el mapa por primera vez.
class AccessLegend extends StatelessWidget {
  const AccessLegend({super.key});

  @override
  Widget build(BuildContext context) {
    final text = Theme.of(context).textTheme;
    final rl = context.rl;
    Widget row(Widget badge, String title, String detail) => MergeSemantics(
          child: Padding(
            padding: const EdgeInsets.symmetric(vertical: 8),
            child: Row(
```

```
              children: [
                SizedBox(width: 44, child: Center(child: badge)),
                const SizedBox(width: 14),
                Expanded(
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text(title, style: text.titleMedium),
                      Text(detail,
                          style: text.bodyMedium?.copyWith(color: rl.text2)),
                    ],
                  ),
                ),
              ],
            ),
          ),
        );
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      mainAxisSize: MainAxisSize.min,
      children: [
        Text('Qué significa cada marca', style: text.headlineSmall),
        const SizedBox(height: 8),
        for (final a in Accessibility.values)
          row(AccessBadge(access: a, size: 36), a.label, a.explanation),
        row(const HazardTriangle(color: AppTheme.hazard, size: 38),
            'Peligro reportado', 'Lo avisó la comunidad; caduca si nadie lo confirma'),
        const SizedBox(height: 4),
        Text('Los cuadrados son cruces; los círculos, lugares.',
            style: text.bodyMedium?.copyWith(color: rl.text2)),
      ],
    );
  }
}
```

lib/widgets/ui.dart

Componentes compartidos.

```
import 'package:flutter/material.dart';
import 'package:latlong2/latlong.dart';

import '../core/describe.dart';
import '../core/geo.dart';
import '../core/theme.dart';
import '../models/place.dart';
import '../models/report.dart';
import 'map_markers.dart';

/// Piezas visuales que comparten varias pantallas.

const cardShadow = [
  BoxShadow(color: Color(0x1F000000), blurRadius: 10, offset: Offset(0, 2)),
];

/// Barrita para arrastrar una hoja inferior.
class SheetHandle extends StatelessWidget {
  const SheetHandle({super.key});

  @override
  Widget build(BuildContext context) => ExcludeSemantics(
        child: Center(
          child: Container(
            margin: const EdgeInsets.only(top: 10, bottom: 6),
            width: 40,
            height: 5,
            decoration: BoxDecoration(
              color: context.rl.handle,
```

```
              borderRadius: BorderRadius.circular(3),
            ),
          ),
        ),
      );
}

/// Botón cuadrado flotante sobre el mapa (leyenda, centrar, volver).
class MapIconButton extends StatelessWidget {
  const MapIconButton({
    super.key,
    required this.icon,
    required this.tooltip,
    required this.onPressed,
    this.color,
  });

  final IconData icon;
  final String tooltip;
  final VoidCallback onPressed;
  final Color? color;

  @override
  Widget build(BuildContext context) {
    final rl = context.rl;
    return Tooltip(
      message: tooltip,
      child: Semantics(
        button: true,
        label: tooltip,
        excludeSemantics: true,
        child: Container(
          decoration: BoxDecoration(
            borderRadius: BorderRadius.circular(14),
            boxShadow: cardShadow,
          ),
          child: Material(
            color: rl.surface,
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(14),
              side: BorderSide(color: rl.border),
            ),
            child: InkWell(
              borderRadius: BorderRadius.circular(14),
              onTap: onPressed,
              child: SizedBox(
                width: 48,
                height: 48,
                child: Icon(icon, size: 24, color: color ?? rl.text),
              ),
            ),
          ),
        ),
      ),
    );
  }
}

/// Píldora de filtro: azul con palomita si está activa.
class FilterPill extends StatelessWidget {
  const FilterPill({
    super.key,
    required this.label,
    required this.selected,
    required this.onTap,
    this.icon,
  });

  final String label;
  final bool selected;
```

```
  final VoidCallback onTap;
  final IconData? icon;

  @override
  Widget build(BuildContext context) {
    final rl = context.rl;
    final fg = selected ? rl.onPrimary : rl.text;
    return Semantics(
      button: true,
      selected: selected,
      label: 'Filtro: $label',
      excludeSemantics: true,
      child: Material(
        color: selected ? rl.primary : rl.surface,
        shape: StadiumBorder(
          side: selected ? BorderSide.none : BorderSide(color: rl.border),
        ),
        elevation: selected ? 0 : 1,
        shadowColor: Colors.black26,
        child: InkWell(
          customBorder: const StadiumBorder(),
          onTap: onTap,
          child: ConstrainedBox(
            constraints: const BoxConstraints(minHeight: 44),
            child: Padding(
              padding: const EdgeInsets.symmetric(horizontal: 14),
              child: Row(
                mainAxisSize: MainAxisSize.min,
                children: [
                  if (selected) ...[
                    Icon(Icons.check, size: 18, color: fg),
                    const SizedBox(width: 6),
                  ] else if (icon != null) ...[
                    Icon(icon, size: 18, color: rl.text2),
                    const SizedBox(width: 6),
                  ],
                  Text(label,
                      style: TextStyle(
                        fontSize: 15,
                        color: fg,
                        fontWeight: selected ? FontWeight.w700 : FontWeight.w400,
                      )),
                ],
              ),
            ),
          ),
        ),
      ),
    );
  }
}

/// Control segmentado (Lugares | Peligros, Baja | Media | Alta…).
class Segmented<T> extends StatelessWidget {
  const Segmented({
    super.key,
    required this.values,
    required this.selected,
    required this.labelOf,
    required this.onChanged,
    this.semanticPrefix = '',
  });

  final List<T> values;
  final T selected;
  final String Function(T) labelOf;
  final ValueChanged<T> onChanged;
  final String semanticPrefix;

  @override
```

```
  Widget build(BuildContext context) {
    final rl = context.rl;
    return Container(
      padding: const EdgeInsets.all(4),
      decoration: BoxDecoration(
        color: rl.surface2,
        borderRadius: BorderRadius.circular(14),
      ),
      child: Row(
        children: [
          for (final v in values)
            Expanded(
              child: Semantics(
                button: true,
                selected: v == selected,
                label: '$semanticPrefix${labelOf(v)}',
                excludeSemantics: true,
                child: GestureDetector(
                  onTap: () => onChanged(v),
                  behavior: HitTestBehavior.opaque,
                  child: AnimatedContainer(
                    duration: const Duration(milliseconds: 160),
                    constraints: const BoxConstraints(minHeight: 44),
                    alignment: Alignment.center,
                    padding: const EdgeInsets.symmetric(horizontal: 6, vertical: 8),
                    decoration: BoxDecoration(
                      color: v == selected ? rl.surface : Colors.transparent,
                      borderRadius: BorderRadius.circular(10),
                      boxShadow: v == selected
                          ? const [
                              BoxShadow(
                                  color: Color(0x26000000),
                                  blurRadius: 3,
                                  offset: Offset(0, 1)),
                            ]
                          : null,
                    ),
                    child: Text(
                      labelOf(v),
                      textAlign: TextAlign.center,
                      style: TextStyle(
                        fontSize: 16,
                        color: v == selected ? rl.text : rl.text2,
                        fontWeight:
                            v == selected ? FontWeight.w700 : FontWeight.w400,
                      ),
                    ),
                  ),
                ),
              ),
            ),
        ],
      ),
    );
  }
}

/// Etiqueta de color suave ("Sin escaleras", "Evita 1 peligro").
class Tag extends StatelessWidget {
  const Tag({
    super.key,
    required this.label,
    required this.background,
    required this.foreground,
    this.icon,
  });

  final String label;
  final Color background;
  final Color foreground;
```

```
  final IconData? icon;

  @override
  Widget build(BuildContext context) => Container(
        padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
        decoration: BoxDecoration(
          color: background,
          borderRadius: BorderRadius.circular(16),
        ),
        child: Row(
          mainAxisSize: MainAxisSize.min,
          children: [
            if (icon != null) ...[
              Icon(icon, size: 16, color: foreground),
              const SizedBox(width: 6),
            ],
            Flexible(
              child: Text(label,
                  style: TextStyle(
                      fontSize: 14,
                      fontWeight: FontWeight.w700,
                      color: foreground)),
            ),
          ],
        ),
      );
}

/// Fondo suave y color de texto para el estado de accesibilidad.
({Color background, Color foreground}) accessTone(
    BuildContext context, Accessibility a) {
  final rl = context.rl;
  final dark = Theme.of(context).brightness == Brightness.dark;
  if (a == Accessibility.yes) return (background: rl.okSoft, foreground: rl.okText);
  final base = accessColor(a);
  return (
    background: base.withValues(alpha: dark ? 0.22 : 0.12),
    foreground: dark ? Color.lerp(base, Colors.white, 0.55)! : base,
  );
}

/// "Entrada sin escalón · Baño accesible".
String placeSummary(Place p) {
  final parts = <String>[
    switch (p.access) {
      Accessibility.yes => 'Entrada sin escalón',
      Accessibility.limited => 'Acceso limitado',
      Accessibility.no => 'No accesible',
      Accessibility.unknown => 'Sin dato de accesibilidad',
    },
    if (p.accessibleToilet == true) 'Baño accesible',
  ];
  return parts.join(' · ');
}

/// Distancia corta y dirección, en dos renglones a la derecha de una fila.
class _DistanceColumn extends StatelessWidget {
  const _DistanceColumn({required this.target, required this.me, this.heading});
  final LatLng target;
  final LatLng? me;
  final double? heading;

  @override
  Widget build(BuildContext context) {
    final me = this.me;
    if (me == null) return const SizedBox.shrink();
    final rl = context.rl;
    return Column(
      crossAxisAlignment: CrossAxisAlignment.end,
      children: [
```

```
        Text(formatDistanceShort(distanceMeters(me, target)),
            style: TextStyle(
                fontSize: 16, fontWeight: FontWeight.w700, color: rl.text)),
        Text(directionFrom(me, target, heading),
            style: TextStyle(fontSize: 13, color: rl.text2)),
      ],
    );
  }
}

/// Fila de un lugar en "Cerca de ti".
class PlaceRow extends StatelessWidget {
  const PlaceRow({
    super.key,
    required this.place,
    required this.me,
    required this.heading,
    required this.onTap,
  });

  final Place place;
  final LatLng? me;
  final double? heading;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    final rl = context.rl;
    final title = place.name == place.category
        ? place.name
        : '${place.name} · ${place.category}';
    return Semantics(
      button: true,
      label: describePlace(place, me: me, heading: heading),
      excludeSemantics: true,
      child: InkWell(
        onTap: onTap,
        child: Container(
          constraints: const BoxConstraints(minHeight: 64),
          padding: const EdgeInsets.symmetric(vertical: 12),
          decoration: BoxDecoration(
            border: Border(bottom: BorderSide(color: rl.border)),
          ),
          child: Row(
            children: [
              AccessBadge(access: place.access),
              const SizedBox(width: 14),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(title,
                        maxLines: 2,
                        overflow: TextOverflow.ellipsis,
                        style: TextStyle(
                            fontSize: 17,
                            fontWeight: FontWeight.w700,
                            color: rl.text)),
                    const SizedBox(height: 2),
                    Text(placeSummary(place),
                        style: TextStyle(fontSize: 14, color: rl.text2)),
                  ],
                ),
              ),
              const SizedBox(width: 10),
              _DistanceColumn(target: place.location, me: me, heading: heading),
            ],
          ),
        ),
      ),
    ),
```

```
    );
  }
}

/// Fila de un peligro en "Cerca de ti".
class HazardRow extends StatelessWidget {
  const HazardRow({
    super.key,
    required this.report,
    required this.me,
    required this.heading,
    required this.onTap,
  });

  final Report report;
  final LatLng? me;
  final double? heading;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    final rl = context.rl;
    final now = DateTime.now();
    return Semantics(
      button: true,
      label: describeReport(report, now, me: me, heading: heading),
      excludeSemantics: true,
      child: InkWell(
        onTap: onTap,
        child: Container(
          constraints: const BoxConstraints(minHeight: 64),
          padding: const EdgeInsets.symmetric(vertical: 12),
          decoration: BoxDecoration(
            border: Border(bottom: BorderSide(color: rl.border)),
          ),
          child: Row(
            children: [
              SizedBox(
                width: 40,
                child: HazardTriangle(
                    color: hazardColor(report.severity), size: 36),
              ),
              const SizedBox(width: 14),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(report.type.label,
                        style: TextStyle(
                            fontSize: 17,
                            fontWeight: FontWeight.w700,
                            color: rl.text)),
                    const SizedBox(height: 2),
                    Text(
                        'Gravedad ${report.severity.label.toLowerCase()} · '
                        'confirmado ${timeAgo(report.lastConfirmedAt, now)}',
                        style: TextStyle(fontSize: 14, color: rl.text2)),
                  ],
                ),
              ),
              const SizedBox(width: 10),
              _DistanceColumn(target: report.location, me: me, heading: heading),
            ],
          ),
        ),
      ),
    );
  }
}
```

```
/// Mensaje centrado cuando una lista está vacía.
class EmptyNote extends StatelessWidget {
  const EmptyNote(this.message, {super.key, this.icon = Icons.search_off});
  final String message;
  final IconData icon;

  @override
  Widget build(BuildContext context) {
    final rl = context.rl;
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 28),
      child: Column(
        children: [
          Icon(icon, size: 36, color: rl.text2),
          const SizedBox(height: 8),
          Text(message,
              textAlign: TextAlign.center,
              style: TextStyle(fontSize: 16, color: rl.text2)),
        ],
      ),
    );
  }
}

/// Botón redondo de "escuchar" (bocina) con fondo azul suave.
class ListenButton extends StatelessWidget {
  const ListenButton({super.key, required this.onPressed, this.label});

  final VoidCallback onPressed;

  /// Si es null solo se muestra el ícono.
  final String? label;

  @override
  Widget build(BuildContext context) {
    final rl = context.rl;
    if (label == null) {
      return IconButton.filled(
        tooltip: 'Escuchar',
        iconSize: 24,
        style: IconButton.styleFrom(
          backgroundColor: rl.primarySoft,
          foregroundColor: rl.primary,
          minimumSize: const Size(48, 48),
        ),
        onPressed: onPressed,
        icon: const Icon(Icons.volume_up_outlined),
      );
    }
    return FilledButton.icon(
      onPressed: onPressed,
      icon: const Icon(Icons.volume_up_outlined, size: 20),
      label: Text(label!),
    );
  }
}
```

lib/widgets/report_photo.dart

Foto de un reporte.

```
import 'dart:io';

import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';

import '../core/theme.dart';
```

```
/// Foto de un reporte en el teléfono (archivo) o en web (URL del navegador).
///
/// En web la foto vive solo mientras la pestaña está abierta; si ya no
/// existe se muestra un aviso en vez de fallar.
class ReportPhoto extends StatelessWidget {
  const ReportPhoto({super.key, required this.path, this.height = 200});

  final String path;
  final double height;

  /// Si hay algo que mostrar en esta plataforma.
  static bool exists(String? path) {
    if (path == null || path.isEmpty) return false;
    if (kIsWeb) return true;
    return File(path).existsSync();
  }

  @override
  Widget build(BuildContext context) {
    Widget missing(BuildContext _, Object _, StackTrace? _) => SizedBox(
          height: 80,
          child: Center(
            child: Text('La foto ya no está disponible.',
                style: Theme.of(context)
                    .textTheme
                    .bodyMedium
                    ?.copyWith(color: context.rl.text2)),
          ),
        );

    final image = kIsWeb
        ? Image.network(path,
            height: height,
            fit: BoxFit.cover,
            semanticLabel: 'Foto del peligro',
            errorBuilder: missing)
        : Image.file(File(path),
            height: height,
            fit: BoxFit.cover,
            semanticLabel: 'Foto del peligro',
            errorBuilder: missing);

    return ClipRRect(borderRadius: BorderRadius.circular(12), child: image);
  }
}
```

lib/features/onboarding/profile_screen.dart

Pantalla de bienvenida y perfil.

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

import '../../app_state.dart';
import '../../core/theme.dart';
import '../../models/settings.dart';
import '../map/map_screen.dart';

/// Elección del perfil de movilidad. Se muestra al abrir la app por primera
/// vez y desde Ajustes.
class ProfileScreen extends StatefulWidget {
  const ProfileScreen({super.key, this.firstTime = false});

  final bool firstTime;

  @override
  State<ProfileScreen> createState() => _ProfileScreenState();
}
```

```
class _ProfileScreenState extends State<ProfileScreen> {
  MobilityProfile? _selected;
  bool _saving = false;

  @override
  void initState() {
    super.initState();
    _selected = AppScope.read(context).settings.profile;
    WidgetsBinding.instance.addPostFrameCallback((_) {
      final app = AppScope.read(context);
      // La bienvenida se dice siempre: quien no ve aún no ha elegido nada.
      app.voice.speak(widget.firstTime
          ? 'Bienvenido a Ruta Libre. Elige cómo te mueves y toca Continuar. '
              'Opciones: silla de ruedas, bastón o andadera, baja visión o '
              'ceguera.'
          : 'Elige tu perfil y toca Guardar.');
    });
  }

  void _pick(MobilityProfile p) {
    HapticFeedback.selectionClick();
    setState(() => _selected = p);
    AppScope.read(context)
        .voice
        .speak('${p.label}. ${widget.firstTime ? 'Toca Continuar' : 'Toca Guardar'}.');
  }

  Future<void> _confirm() async {
    final p = _selected;
    if (p == null || _saving) return;
    setState(() => _saving = true);
    final app = AppScope.read(context);
    await HapticFeedback.mediumImpact();
    if (widget.firstTime) {
      await app.chooseProfile(p);
    } else {
      await app.updateSettings(app.settings.copyWith(profile: p));
    }
    await app.voice.speak('Perfil: ${p.label}.');
    if (!mounted) return;
    if (widget.firstTime) {
      await Navigator.of(context).pushReplacement(
        MaterialPageRoute<void>(builder: (_) => const MapScreen()),
      );
    } else {
      Navigator.of(context).pop();
    }
  }

  @override
  Widget build(BuildContext context) {
    final text = Theme.of(context).textTheme;
    final rl = context.rl;

    return Scaffold(
      appBar: widget.firstTime ? null : AppBar(title: const Text('Tu perfil')),
      body: SafeArea(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Expanded(
              child: ListView(
                padding: EdgeInsets.fromLTRB(24, widget.firstTime ? 32 : 8, 24, 16),
                children: [
                  if (widget.firstTime) ...[
                    ExcludeSemantics(
                      child: Align(
                        alignment: Alignment.centerLeft,
                        child: Container(
```

```
                        width: 56,
                        height: 56,
                        decoration: BoxDecoration(
                          color: rl.primary,
                          borderRadius: BorderRadius.circular(16),
                        ),
                        child: Icon(Icons.where_to_vote_outlined,
                            color: rl.onPrimary, size: 32),
                      ),
                    ),
                  ),
                  const SizedBox(height: 16),
                  Semantics(
                    header: true,
                    child: Text('Bienvenido a Ruta Libre',
                        style: text.displaySmall?.copyWith(height: 1.15)),
                  ),
                  const SizedBox(height: 12),
                ],
                Text(
                  widget.firstTime
                      ? 'Elige cómo te mueves. Adaptamos el mapa, las rutas y '
                          'los avisos a ti. Puedes cambiarlo después.'
                      : 'Adaptamos el mapa, las rutas y los avisos a tu perfil.',
                  style: text.bodyLarge?.copyWith(color: rl.text2, fontSize: 18),
                ),
                const SizedBox(height: 24),
                for (final p in MobilityProfile.values)
                  Padding(
                    padding: const EdgeInsets.only(bottom: 12),
                    child: _ProfileOption(
                      profile: p,
                      selected: p == _selected,
                      onTap: () => _pick(p),
                    ),
                  ),
              ],
            ),
          ),
          Padding(
            padding: const EdgeInsets.fromLTRB(24, 8, 24, 20),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                ElevatedButton(
                  onPressed: _selected == null || _saving ? null : _confirm,
                  child: Text(_selected == null
                      ? 'Elige una opción'
                      : (widget.firstTime ? 'Continuar' : 'Guardar')),
                ),
                if (widget.firstTime) ...[
                  const SizedBox(height: 12),
                  Text('Datos del mapa: © colaboradores de OpenStreetMap',
                      textAlign: TextAlign.center,
                      style: TextStyle(fontSize: 14, color: rl.text2)),
                ],
              ],
            ),
          ),
        ],
      ),
    ),
  );
  }
}

class _ProfileOption extends StatelessWidget {
  const _ProfileOption({
    required this.profile,
    required this.selected,
```

```
  required this.onTap,
});

final MobilityProfile profile;
final bool selected;
final VoidCallback onTap;

IconData get _icon => switch (profile) {
      MobilityProfile.wheelchair => Icons.accessible,
      MobilityProfile.mobilityAid => Icons.elderly,
      MobilityProfile.lowVision => Icons.visibility_outlined,
      MobilityProfile.blind => Icons.blind,
    };

@override
Widget build(BuildContext context) {
  final rl = context.rl;
  return Semantics(
    inMutuallyExclusiveGroup: true,
    selected: selected,
    button: true,
    label: '${profile.label}. ${profile.description}',
    excludeSemantics: true,
    child: AnimatedContainer(
      duration: const Duration(milliseconds: 150),
      decoration: BoxDecoration(
        borderRadius: BorderRadius.circular(16),
        boxShadow: selected
            ? [BoxShadow(color: rl.primarySoft, spreadRadius: 4)]
            : null,
      ),
      child: Material(
        color: rl.surface,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(16),
          side: BorderSide(
            color: selected ? rl.primary : rl.border,
            width: selected ? 2 : 1,
          ),
        ),
        child: InkWell(
          borderRadius: BorderRadius.circular(16),
          onTap: onTap,
          child: Container(
            constraints: const BoxConstraints(minHeight: 76),
            padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
            child: Row(
              children: [
                Container(
                  width: 48,
                  height: 48,
                  decoration: BoxDecoration(
                    color: selected ? rl.primarySoft : rl.surface2,
                    borderRadius: BorderRadius.circular(14),
                  ),
                  child: Icon(_icon,
                      size: 26, color: selected ? rl.primary : rl.text),
                ),
                const SizedBox(width: 14),
                Expanded(
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text(profile.label,
                          style: TextStyle(
                              fontSize: 18,
                              fontWeight: FontWeight.w700,
                              color: rl.text)),
                      const SizedBox(height: 2),
                      Text(profile.description,
```

```
                          style: TextStyle(fontSize: 15, color: rl.text2)),
                    ],
                  ),
                ),
                const SizedBox(width: 8),
                AnimatedOpacity(
                  opacity: selected ? 1 : 0,
                  duration: const Duration(milliseconds: 150),
                  child: Container(
                    width: 24,
                    height: 24,
                    decoration: BoxDecoration(
                      color: rl.primary,
                      shape: BoxShape.circle,
                    ),
                    child: Icon(Icons.check, size: 16, color: rl.onPrimary),
                  ),
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

lib/features/map/map_screen.dart

Pantalla del mapa.

```
import 'package:flutter/material.dart';
import 'package:flutter_map/flutter_map.dart';
import 'package:latlong2/latlong.dart';

import '../../app_state.dart';
import '../../config.dart';
import '../../core/describe.dart';
import '../../core/geo.dart';
import '../../core/route_api.dart';
import '../../core/text_utils.dart';
import '../../core/theme.dart';
import '../../core/voice_commands.dart';
import '../../models/place.dart';
import '../../models/report.dart';
import '../../widgets/base_map.dart';
import '../../widgets/map_markers.dart';
import '../../widgets/ui.dart';
import '../nearby/nearby_screen.dart';
import '../place/place_screen.dart';
import '../report/new_report_screen.dart';
import '../report/report_screen.dart';
import '../route/route_screen.dart';
import '../settings/settings_screen.dart';

/// Filtros rápidos por tipo de lugar.
enum PlaceFilter {
  pharmacy('Farmacias', Icons.local_pharmacy_outlined),
  toilets('Baños', Icons.wc),
  transport('Transporte', Icons.directions_bus_outlined);

  const PlaceFilter(this.label, this.icon);
  final String label;
  final IconData icon;

  bool matches(Place p) => switch (this) {
        PlaceFilter.pharmacy => p.category == 'Farmacia',
```

```
        PlaceFilter.toilets =>
          p.category == 'Baños públicos' || p.accessibleToilet == true,
        PlaceFilter.transport =>
          const {'Parada de autobús', 'Estación', 'Elevador'}.contains(p.category),
      };
}

/// Pantalla principal: mapa a pantalla completa, búsqueda y filtros arriba,
/// y la hoja "Cerca de ti" abajo.
///
/// Todo lo que muestra el mapa también está en la hoja, que es la vista
/// para quien usa lector de pantalla.
class MapScreen extends StatefulWidget {
  const MapScreen({super.key});

  @override
  State<MapScreen> createState() => _MapScreenState();
}

class _MapScreenState extends State<MapScreen> {
  /// A partir de este zoom se muestran los cruces.
  static const _crossingZoom = 17.0;
  static const _sheetInitial = 0.36;
  static const _sheetMax = 0.92;

  final _map = MapController();
  final _sheet = DraggableScrollableController();
  final _search = TextEditingController();
  final _searchFocus = FocusNode();

  /// Altura actual de la hoja (fracción de la pantalla).
  final _sheetSize = ValueNotifier<double>(_sheetInitial);

  bool _showCrossings = true;
  bool _followMe = true;
  bool _listening = false;
  bool _mapReady = false;
  bool _showLoadHere = false;
  bool _showHazards = false;
  PlaceFilter? _filter;
  String _query = '';
  LatLng _cameraCenter = AppConfig.pilotCenter;

  /// Centros de las zonas ya descargadas.
  final List<LatLng> _loadedCenters = [AppConfig.pilotCenter];

  late final AppState _app;

  @override
  void initState() {
    super.initState();
    _app = AppScope.read(context);
    _app.location.position.addListener(_onMove);
    _searchFocus.addListener(() {
      // Al buscar se sube la hoja para ver los resultados.
      if (_searchFocus.hasFocus && _sheet.isAttached && _sheet.size < 0.55) {
        _sheet.animateTo(0.6,
            duration: const Duration(milliseconds: 250), curve: Curves.easeOut);
      }
    });
    WidgetsBinding.instance.addPostFrameCallback((_) => _start());
  }

  Future<void> _start() async {
    final error = await _app.start();
    if (!mounted) return;
    if (error != null) {
      _snack(error);
      await _app.voice.speak(error);
      return;
```

```
  }
  final accessible = _app.allPlaces
      .where((p) => p.kind == PlaceKind.poi && p.access == Accessibility.yes)
      .length;
  final hazards = _app.activeReports().length;
  await _app.voice.announce(
    'Mapa. $accessible lugares accesibles registrados en la zona'
    '${hazards == 0 ? '' : ' y $hazards peligros reportados'}. '
    'Abajo está la lista Cerca de ti. Arriba puedes buscar o hablar.',
  );
}

void _onMove() {
  final me = _app.me;
  if (me == null || !_followMe || !_mapReady) return;
  _map.move(me, _map.camera.zoom);
}

@override
void dispose() {
  _app.location.position.removeListener(_onMove);
  _sheet.dispose();
  _search.dispose();
  _searchFocus.dispose();
  _sheetSize.dispose();
  super.dispose();
}

void _snack(String message) {
  ScaffoldMessenger.of(context)
    ..hideCurrentSnackBar()
    ..showSnackBar(SnackBar(content: Text(message)));
}

Future<void> _push(Widget screen) =>
    Navigator.of(context).push(MaterialPageRoute<void>(builder: (_) => screen));

// ---------- Filtros y búsqueda ----------

bool _matches(Place p) {
  final f = _filter;
  if (f != null && !f.matches(p)) return false;
  final q = normalize(_query);
  if (q.isEmpty) return true;
  return normalize(p.name).contains(q) || normalize(p.category).contains(q);
}

bool get _searching => _query.trim().isNotEmpty || _filter != null;

List<Place> _listPlaces(AppState app) =>
    app.placesNearby(limit: 200).where(_matches).take(40).toList();

void _toggleOnlyAccessible() {
  final v = !_app.settings.onlyAccessible;
  _app.updateSettings(_app.settings.copyWith(onlyAccessible: v));
  _app.voice.announce(
      v ? 'Mostrando solo lugares accesibles.' : 'Mostrando todos los lugares.');
}

void _toggleFilter(PlaceFilter f) {
  setState(() {
    _filter = _filter == f ? null : f;
    _showHazards = false;
  });
  final count = _listPlaces(_app).length;
  _app.voice.announce(_filter == null
      ? 'Sin filtro.'
      : '${f.label}: $count ${count == 1 ? 'resultado' : 'resultados'}.');
}
```

```
// ---------- Acciones ----------

void _centerOnMe() {
  final me = _app.me;
  if (me == null) {
    _app.voice.speak('Todavía no tengo tu ubicación.');
    _snack('Todavía no tengo tu ubicación.');
    return;
  }
  setState(() => _followMe = true);
  if (_mapReady) _map.move(me, 18);
  _app.voice.announce('Centrado en tu ubicación.');
}

Future<void> _whereAmI() async {
  final me = _app.me;
  if (me == null) {
    await _app.voice.speak('Todavía no tengo tu ubicación.');
    return;
  }
  final nearest = _app.placesNearby(limit: 1);
  if (nearest.isEmpty) {
    await _app.voice.speak('No hay lugares registrados cerca de ti.');
    return;
  }
  final p = nearest.first;
  await _app.voice.speak(
    'Estás a ${formatDistance(distanceMeters(me, p.location))} de ${p.name}, '
    '${directionFrom(me, p.location, _app.heading)}.',
  );
}

Future<void> _speakList() async {
  final me = _app.me;
  final String text;
  if (_showHazards) {
    final list = _app.reportsNearby().take(3).toList();
    final now = DateTime.now();
    text = list.isEmpty
        ? 'No hay peligros reportados cerca.'
        : list
            .map((r) => describeReport(r, now, me: me, heading: _app.heading))
            .join(' ');
  } else {
    final list = _listPlaces(_app).take(5).toList();
    text = list.isEmpty
        ? 'No encontré lugares.'
        : list
            .map((p) => '${p.name}, ${p.access.label.toLowerCase()}, '
                '${whereIs(p.location, me, _app.heading)}.')
            .join(' ');
  }
  await _app.voice.speak(text);
}

Future<void> _listen() async {
  if (_listening) return;
  setState(() => _listening = true);
  await _app.voice.speak('Te escucho');
  final phrase = await _app.voice.listenOnce();
  if (!mounted) return;
  setState(() => _listening = false);
  if (phrase == null) {
    await _app.voice.speak('No escuché nada. Intenta otra vez.');
    return;
  }
  switch (parseVoiceCommand(phrase)) {
    case VoiceCommand.nearby:
      await _push(const NearbyScreen());
    case VoiceCommand.report:
```

```
      await _push(const NewReportScreen());
    case VoiceCommand.whereAmI:
      await _whereAmI();
    case VoiceCommand.clearRoute:
      _app.clearRoute();
      await _app.voice.speak('Ruta quitada.');
    case VoiceCommand.settings:
      await _push(const SettingsScreen());
    case VoiceCommand.unknown:
      // Lo que no es un comando se busca como lugar.
      _search.text = phrase;
      setState(() {
        _query = phrase;
        _showHazards = false;
      });
      final found = _listPlaces(_app);
      await _app.voice.speak(found.isEmpty
          ? 'No encontré "$phrase". Puedes decir: qué hay cerca, reportar '
              'un peligro, dónde estoy o quitar ruta.'
          : 'Encontré ${found.length} resultados para $phrase. '
              'El más cercano: ${found.first.name}, '
              '${whereIs(found.first.location, _app.me, _app.heading)}.');
  }
}

void _showLegend() {
  showModalBottomSheet<void>(
    context: context,
    builder: (_) => const SafeArea(
      child: Padding(
        padding: EdgeInsets.fromLTRB(20, 0, 20, 20),
        child: AccessLegend(),
      ),
    ),
  );
}

bool get _outsideLoadedArea => _loadedCenters.every((c) =>
    distanceMeters(_cameraCenter, c) > AppConfig.pilotRadiusMeters * 0.8);

Future<void> _loadHere() async {
  final center = _cameraCenter;
  await _app.loadPlaces(center);
  if (!mounted) return;
  final error = _app.placesError;
  if (error != null) {
    _snack(error);
    await _app.voice.speak(error);
    return;
  }
  setState(() {
    _loadedCenters.add(center);
    _showLoadHere = _outsideLoadedArea;
  });
  await _app.voice.announce('Lugares de esta zona descargados.');
}

// ---------- Vista ----------

@override
Widget build(BuildContext context) {
  final app = AppScope.of(context);
  final now = DateTime.now();
  final route = app.route;

  final places = app.visiblePlaces
      .where((p) =>
          (_showCrossings && !_searching && p.kind == PlaceKind.crossing) ||
          (p.kind == PlaceKind.poi && _matches(p)))
      .toList()
```

```
    // Los cruces debajo, los lugares encima.
    ..sort((a, b) => b.kind.index.compareTo(a.kind.index));

  return Scaffold(
    body: LayoutBuilder(builder: (context, constraints) {
      final height = constraints.maxHeight;
      final sheetMin = (190 / height).clamp(0.16, 0.4);
      return Stack(
        children: [
          Positioned.fill(
            child: FlutterMap(
              mapController: _map,
              options: MapOptions(
                initialCenter: app.me ?? AppConfig.pilotCenter,
                initialZoom: 17,
                minZoom: 12,
                maxZoom: 19,
                onTap: (_, _) => FocusScope.of(context).unfocus(),
                onMapReady: () {
                  _mapReady = true;
                  _onMove();
                },
                onPositionChanged: (camera, hasGesture) {
                  // Solo se redibuja si cambia algo visible; arrastrar el
                  // mapa no debe reconstruir cientos de marcadores por cuadro.
                  _cameraCenter = camera.center;
                  final crossings = camera.zoom >= _crossingZoom;
                  final outside = _outsideLoadedArea;
                  final follow = hasGesture ? false : _followMe;
                  if (crossings != _showCrossings ||
                      outside != _showLoadHere ||
                      follow != _followMe) {
                    setState(() {
                      _showCrossings = crossings;
                      _showLoadHere = outside;
                      _followMe = follow;
                    });
                  }
                },
              ),
              children: [
                osmTiles(context),
                if (route != null) routeLine(context, route),
                MarkerLayer(
                  markers: [
                    for (final p in places)
                      Marker(
                        point: p.location,
                        width: 44,
                        height: 44,
                        child: PlaceMarker(
                          place: p,
                          semanticLabel: describePlace(p,
                              me: app.me, heading: app.heading),
                          onTap: () => _push(PlaceScreen(place: p)),
                        ),
                      ),
                    for (final r in app.activeReports(now))
                      Marker(
                        point: r.location,
                        width: 52,
                        height: 52,
                        child: HazardMarker(
                          report: r,
                          semanticLabel: describeReport(r, now,
                              me: app.me, heading: app.heading),
                          onTap: () => _push(ReportScreen(reportId: r.id)),
                        ),
                      ),
                    if (app.me != null)
```

```
                  Marker(
                    point: app.me!,
                    width: 32,
                    height: 32,
                    child: const UserMarker(),
                  ),
                ],
              ),
            ],
          ),
        ),

        // Botones flotantes: suben y bajan con la hoja.
        ValueListenableBuilder<double>(
          valueListenable: _sheetSize,
          builder: (context, size, _) {
            final bottom = size * height + 12;
            final hidden = size > 0.62;
            return Positioned(
              left: 0,
              right: 0,
              bottom: bottom,
              child: IgnorePointer(
                ignoring: hidden,
                child: ExcludeSemantics(
                  excluding: hidden,
                  child: AnimatedOpacity(
                    opacity: hidden ? 0 : 1,
                    duration: const Duration(milliseconds: 150),
                    child: _FloatingControls(
                      onLegend: _showLegend,
                      onCenter: _centerOnMe,
                      onReport: () => _push(const NewReportScreen()),
                    ),
                  ),
                ),
              ),
            );
          },
        ),

        // Hoja "Cerca de ti".
        NotificationListener<DraggableScrollableNotification>(
          onNotification: (n) {
            _sheetSize.value = n.extent;
            return false;
          },
          child: DraggableScrollableSheet(
            controller: _sheet,
            initialChildSize: _sheetInitial.clamp(sheetMin, _sheetMax),
            minChildSize: sheetMin,
            maxChildSize: _sheetMax,
            snap: true,
            // Punto intermedio solo si cabe entre el mínimo y el máximo.
            snapSizes: _sheetInitial > sheetMin + 0.02 ? const [_sheetInitial] : null,
            builder: (context, scroll) => _NearbySheet(
              scroll: scroll,
              app: app,
              places: _listPlaces(app),
              hazards: app.reportsNearby(),
              showHazards: _showHazards,
              title: _searching ? 'Resultados' : 'Cerca de ti',
              subtitle: _searching
                  ? [
                      if (_filter != null) _filter!.label,
                      if (_query.trim().isNotEmpty) '“${_query.trim()}”',
                    ].join(' · ')
                  : 'Alrededores de UPIICSA',
              onTab: (h) => setState(() => _showHazards = h),
              onListen: _speakList,
```

```
              onPlace: (p) => _push(PlaceScreen(place: p)),
              onHazard: (r) => _push(ReportScreen(reportId: r.id)),
            ),
          ),
        ),

        // Búsqueda, filtros y avisos arriba.
        Positioned(
          top: 0,
          left: 0,
          right: 0,
          child: SafeArea(
            bottom: false,
            child: Padding(
              padding: const EdgeInsets.fromLTRB(16, 12, 16, 0),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                children: [
                  _SearchBar(
                    controller: _search,
                    focus: _searchFocus,
                    listening: _listening,
                    onChanged: (v) => setState(() {
                      _query = v;
                      _showHazards = false;
                    }),
                    onClear: () {
                      _search.clear();
                      setState(() => _query = '');
                    },
                    onMic: _listen,
                    onSettings: () => _push(const SettingsScreen()),
                  ),
                  const SizedBox(height: 10),
                  if (route != null)
                    _RouteCard(
                      title: route.destinationName,
                      detail: '${formatDuration(route.durationSeconds)} · '
                          '${formatDistanceShort(route.distanceMeters)}',
                      semantic: spokenSummary(route),
                      onSteps: () => _push(const RouteScreen()),
                      onClear: () {
                        app.clearRoute();
                        app.voice.announce('Ruta quitada.');
                      },
                    )
                  else
                    SingleChildScrollView(
                      scrollDirection: Axis.horizontal,
                      clipBehavior: Clip.none,
                      child: Row(
                        children: [
                          FilterPill(
                            label: 'Solo accesibles',
                            selected: app.settings.onlyAccessible,
                            onTap: _toggleOnlyAccessible,
                          ),
                          for (final f in PlaceFilter.values) ...[
                            const SizedBox(width: 8),
                            FilterPill(
                              label: f.label,
                              icon: f.icon,
                              selected: _filter == f,
                              onTap: () => _toggleFilter(f),
                            ),
                          ],
                        ],
                      ),
                    ),
                  if (app.loadingPlaces)
```

```
                        const _StatusPill(
                          icon: null,
                          text: 'Descargando lugares de la zona…',
                        )
                      else if (_showLoadHere)
                        Center(
                          child: _StatusPill(
                            icon: Icons.travel_explore,
                            text: 'Buscar lugares aquí',
                            onTap: _loadHere,
                          ),
                        ),
                    ],
                  ),
                ),
              ),
            ),
          ],
        );
      }),
    );
  }
}

class _SearchBar extends StatelessWidget {
  const _SearchBar({
    required this.controller,
    required this.focus,
    required this.listening,
    required this.onChanged,
    required this.onClear,
    required this.onMic,
    required this.onSettings,
  });

  final TextEditingController controller;
  final FocusNode focus;
  final bool listening;
  final ValueChanged<String> onChanged;
  final VoidCallback onClear;
  final VoidCallback onMic;
  final VoidCallback onSettings;

  @override
  Widget build(BuildContext context) {
    final rl = context.rl;
    return Container(
      decoration: BoxDecoration(
        color: rl.surface,
        borderRadius: BorderRadius.circular(16),
        border: Border.all(color: rl.border),
        boxShadow: cardShadow,
      ),
      padding: const EdgeInsets.fromLTRB(14, 6, 6, 6),
      child: Row(
        children: [
          Icon(Icons.search, color: rl.text2, size: 22),
          const SizedBox(width: 8),
          Expanded(
            child: TextField(
              controller: controller,
              focusNode: focus,
              onChanged: onChanged,
              textInputAction: TextInputAction.search,
              onSubmitted: (_) => focus.unfocus(),
              style: TextStyle(fontSize: 17, color: rl.text),
              decoration: InputDecoration(
                isDense: true,
                filled: false,
                border: InputBorder.none,
```

```
                enabledBorder: InputBorder.none,
                focusedBorder: InputBorder.none,
                contentPadding: const EdgeInsets.symmetric(vertical: 12),
                hintText: 'Buscar farmacia, baño, banco…',
                hintStyle: TextStyle(fontSize: 17, color: rl.text2),
                semanticCounterText: '',
              ),
            ),
          ),
          ValueListenableBuilder<TextEditingValue>(
            valueListenable: controller,
            builder: (context, value, _) => value.text.isEmpty
                ? const SizedBox.shrink()
                : IconButton(
                    tooltip: 'Borrar búsqueda',
                    onPressed: onClear,
                    icon: Icon(Icons.close, color: rl.text2),
                  ),
          ),
          IconButton(
            tooltip: listening ? 'Escuchando' : 'Hablar o buscar por voz',
            onPressed: onMic,
            style: IconButton.styleFrom(
              backgroundColor: listening ? rl.primary : rl.primarySoft,
              foregroundColor: listening ? rl.onPrimary : rl.primary,
              minimumSize: const Size(44, 44),
              shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
            ),
            icon: Icon(listening ? Icons.hearing : Icons.mic_none),
          ),
          const SizedBox(width: 2),
          IconButton(
            tooltip: 'Ajustes y perfil',
            onPressed: onSettings,
            style: IconButton.styleFrom(
              minimumSize: const Size(44, 44),
              shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
            ),
            icon: Icon(Icons.person_outline, color: rl.text2),
          ),
        ],
      ),
    );
  }
}

class _StatusPill extends StatelessWidget {
  const _StatusPill({required this.icon, required this.text, this.onTap});

  /// Null = mostrar un indicador de progreso.
  final IconData? icon;
  final String text;
  final VoidCallback? onTap;

  @override
  Widget build(BuildContext context) {
    final rl = context.rl;
    return Padding(
      padding: const EdgeInsets.only(top: 10),
      child: Semantics(
        liveRegion: onTap == null,
        button: onTap != null,
        child: Material(
          color: rl.surface,
          shape: StadiumBorder(side: BorderSide(color: rl.border)),
          elevation: 2,
          shadowColor: Colors.black26,
          child: InkWell(
            customBorder: const StadiumBorder(),
            onTap: onTap,
```

```
            child: Padding(
              padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
              child: Row(
                mainAxisSize: MainAxisSize.min,
                children: [
                  if (icon == null)
                    const SizedBox(
                        width: 18,
                        height: 18,
                        child: CircularProgressIndicator(strokeWidth: 2.5))
                  else
                    Icon(icon, size: 20, color: rl.primary),
                  const SizedBox(width: 10),
                  Flexible(
                    child: Text(text,
                        style: TextStyle(
                            fontSize: 15,
                            fontWeight: FontWeight.w700,
                            color: onTap == null ? rl.text : rl.primary)),
                  ),
                ],
              ),
            ),
          ),
        ),
      ),
    );
  }
}

class _RouteCard extends StatelessWidget {
  const _RouteCard({
    required this.title,
    required this.detail,
    required this.semantic,
    required this.onSteps,
    required this.onClear,
  });

  final String title;
  final String detail;
  final String semantic;
  final VoidCallback onSteps;
  final VoidCallback onClear;

  @override
  Widget build(BuildContext context) {
    final rl = context.rl;
    return Container(
      decoration: BoxDecoration(
        color: rl.surface,
        borderRadius: BorderRadius.circular(16),
        border: Border.all(color: rl.border),
        boxShadow: cardShadow,
      ),
      padding: const EdgeInsets.fromLTRB(14, 10, 6, 10),
      child: Row(
        children: [
          Container(
            width: 40,
            height: 40,
            decoration: BoxDecoration(
              color: rl.primarySoft,
              borderRadius: BorderRadius.circular(12),
            ),
            child: Icon(Icons.directions, color: rl.primary),
          ),
          const SizedBox(width: 12),
          Expanded(
            child: Semantics(
```

```
              label: semantic,
              excludeSemantics: true,
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text('Ruta a $title',
                      maxLines: 1,
                      overflow: TextOverflow.ellipsis,
                      style: TextStyle(
                          fontSize: 16,
                          fontWeight: FontWeight.w700,
                          color: rl.text)),
                  Text(detail, style: TextStyle(fontSize: 14, color: rl.text2)),
                ],
              ),
            ),
          ),
          TextButton(onPressed: onSteps, child: const Text('Indicaciones')),
          IconButton(
            tooltip: 'Quitar ruta',
            onPressed: onClear,
            icon: Icon(Icons.close, color: rl.text2),
          ),
        ],
      ),
    );
  }
}

class _FloatingControls extends StatelessWidget {
  const _FloatingControls({
    required this.onLegend,
    required this.onCenter,
    required this.onReport,
  });

  final VoidCallback onLegend;
  final VoidCallback onCenter;
  final VoidCallback onReport;

  @override
  Widget build(BuildContext context) {
    final rl = context.rl;
    return Padding(
      padding: const EdgeInsets.symmetric(horizontal: 16),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.end,
        children: [
          const OsmCredit(),
          const Spacer(),
          Column(
            crossAxisAlignment: CrossAxisAlignment.end,
            mainAxisSize: MainAxisSize.min,
            children: [
              MapIconButton(
                icon: Icons.layers_outlined,
                tooltip: 'Qué significan los colores',
                onPressed: onLegend,
              ),
              const SizedBox(height: 10),
              MapIconButton(
                icon: Icons.my_location,
                tooltip: 'Centrar en mi ubicación',
                color: rl.primary,
                onPressed: onCenter,
              ),
              const SizedBox(height: 12),
              Semantics(
                button: true,
                label: 'Reportar peligro',
```

```
                excludeSemantics: true,
                child: Material(
                  color: AppTheme.hazard,
                  shape: const StadiumBorder(),
                  elevation: 4,
                  shadowColor: Colors.black45,
                  child: InkWell(
                    customBorder: const StadiumBorder(),
                    onTap: onReport,
                    child: const Padding(
                      padding: EdgeInsets.symmetric(horizontal: 18, vertical: 14),
                      child: Row(
                        mainAxisSize: MainAxisSize.min,
                        children: [
                          Icon(Icons.warning_amber_rounded,
                              color: Colors.white, size: 22),
                          SizedBox(width: 8),
                          Text('Reportar peligro',
                              style: TextStyle(
                                  fontSize: 16,
                                  fontWeight: FontWeight.w700,
                                  color: Colors.white)),
                        ],
                      ),
                    ),
                  ),
                ),
              ),
            ],
          ),
        ],
      ),
    );
  }
}

class _NearbySheet extends StatelessWidget {
  const _NearbySheet({
    required this.scroll,
    required this.app,
    required this.places,
    required this.hazards,
    required this.showHazards,
    required this.title,
    required this.subtitle,
    required this.onTab,
    required this.onListen,
    required this.onPlace,
    required this.onHazard,
  });

  final ScrollController scroll;
  final AppState app;
  final List<Place> places;
  final List<Report> hazards;
  final bool showHazards;
  final String title;
  final String subtitle;
  final ValueChanged<bool> onTab;
  final VoidCallback onListen;
  final ValueChanged<Place> onPlace;
  final ValueChanged<Report> onHazard;

  @override
  Widget build(BuildContext context) {
    final rl = context.rl;
    final text = Theme.of(context).textTheme;
    final me = app.me;
    final bottomInset = MediaQuery.paddingOf(context).bottom;
```

```
    final List<Widget> items;
    if (showHazards) {
      items = hazards.isEmpty
          ? [
              const EmptyNote('No hay peligros reportados cerca.',
                  icon: Icons.verified_outlined)
            ]
          : [
              for (final r in hazards)
                HazardRow(
                  report: r,
                  me: me,
                  heading: app.heading,
                  onTap: () => onHazard(r),
                ),
            ];
    } else {
      items = places.isEmpty
          ? [
              EmptyNote(app.loadingPlaces
                  ? 'Descargando lugares…'
                  : 'No encontré lugares con ese filtro.')
            ]
          : [
              for (final p in places)
                PlaceRow(
                  place: p,
                  me: me,
                  heading: app.heading,
                  onTap: () => onPlace(p),
                ),
            ];
    }

    return Semantics(
      container: true,
      label: title,
      child: Container(
        decoration: BoxDecoration(
          color: rl.surface,
          borderRadius: const BorderRadius.vertical(top: Radius.circular(24)),
          boxShadow: const [
            BoxShadow(color: Color(0x24000000), blurRadius: 20, offset: Offset(0, -4)),
          ],
        ),
        child: ListView(
          controller: scroll,
          padding: EdgeInsets.fromLTRB(20, 0, 20, 16 + bottomInset),
          children: [
            const SheetHandle(),
            const SizedBox(height: 6),
            Row(
              crossAxisAlignment: CrossAxisAlignment.end,
              children: [
                Expanded(
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Semantics(
                        header: true,
                        child: Text(title, style: text.headlineSmall),
                      ),
                      if (subtitle.isNotEmpty)
                        Text(subtitle,
                            style: TextStyle(fontSize: 15, color: rl.text2)),
                    ],
                  ),
                ),
                ListenButton(onPressed: onListen, label: 'Escuchar'),
              ],
```

```
            ),
            const SizedBox(height: 14),
            Segmented<bool>(
              values: const [false, true],
              selected: showHazards,
              labelOf: (h) => h ? 'Peligros · ${hazards.length}' : 'Lugares',
              onChanged: onTab,
            ),
            if (me == null)
              Padding(
                padding: const EdgeInsets.only(top: 10),
                child: Text(
                  'Sin ubicación todavía: las distancias se miden desde el '
                  'centro de la zona piloto.',
                  style: TextStyle(fontSize: 14, color: rl.text2),
                ),
              ),
            const SizedBox(height: 4),
            ...items,
          ],
        ),
      ),
    );
  }
}
```

lib/features/nearby/nearby_screen.dart

Pantalla Cerca de ti.

```
import 'package:flutter/material.dart';
import 'package:latlong2/latlong.dart';

import '../../app_state.dart';
import '../../core/describe.dart';
import '../../core/text_utils.dart';
import '../../core/theme.dart';
import '../../models/place.dart';
import '../../widgets/ui.dart';
import '../place/place_screen.dart';
import '../report/report_screen.dart';

/// "¿Qué hay cerca?" a pantalla completa. Se abre con el comando de voz y
/// es la misma lista que la hoja del mapa, pensada para lector de pantalla.
class NearbyScreen extends StatefulWidget {
  const NearbyScreen({super.key});

  @override
  State<NearbyScreen> createState() => _NearbyScreenState();
}

class _NearbyScreenState extends State<NearbyScreen> {
  bool _showHazards = false;
  String _query = '';

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addPostFrameCallback((_) {
      final app = AppScope.read(context);
      final hazards = app.reportsNearby().length;
      app.voice.announce(
        'Cerca de ti. Toca Escuchar para oír los más cercanos, o recorre la '
        'lista. ${hazards == 0 ? 'No hay peligros reportados.' : 'Hay $hazards peligros
reportados en la pestaña Peligros.'}',
      );
    });
  }
```

```
List<Place> _filtered(AppState app) {
  final q = normalize(_query);
  final list = app.placesNearby(limit: 60);
  if (q.isEmpty) return list.take(30).toList();
  return list
      .where((p) =>
          normalize(p.name).contains(q) || normalize(p.category).contains(q))
      .take(30)
      .toList();
}

Future<void> _speakTop(AppState app) async {
  final me = app.me;
  final String text;
  if (_showHazards) {
    final list = app.reportsNearby().take(3).toList();
    final now = DateTime.now();
    text = list.isEmpty
        ? 'No hay peligros reportados cerca.'
        : list
            .map((r) => describeReport(r, now, me: me, heading: app.heading))
            .join(' ');
  } else {
    final list = _filtered(app).take(5).toList();
    text = list.isEmpty
        ? 'No encontré lugares.'
        : list
            .map((p) => '${p.name}, ${p.access.label.toLowerCase()}, '
                '${whereIs(p.location, me, app.heading)}.')
            .join(' ');
  }
  await app.voice.speak(text);
}

@override
Widget build(BuildContext context) {
  final app = AppScope.of(context);
  final rl = context.rl;

  return Scaffold(
    appBar: AppBar(
      title: const Text('Cerca de ti'),
      actions: [
        Padding(
          padding: const EdgeInsets.only(right: 12),
          child: ListenButton(onPressed: () => _speakTop(app), label: 'Escuchar'),
        ),
      ],
    ),
    body: SafeArea(
      // Se redibuja al moverse para que distancias y direcciones sigan vigentes.
      child: ValueListenableBuilder<LatLng?>(
        valueListenable: app.location.position,
        builder: (context, me, _) {
          final places = _filtered(app);
          final hazards = app.reportsNearby();
          return ListView(
            padding: const EdgeInsets.fromLTRB(20, 4, 20, 20),
            children: [
              Segmented<bool>(
                values: const [false, true],
                selected: _showHazards,
                labelOf: (h) => h ? 'Peligros · ${hazards.length}' : 'Lugares',
                onChanged: (h) => setState(() => _showHazards = h),
              ),
              const SizedBox(height: 12),
              if (!_showHazards)
                TextField(
                  decoration: const InputDecoration(
```

```
                    hintText: 'Buscar farmacia, banco, baños…',
                    prefixIcon: Icon(Icons.search),
                  ),
                  onChanged: (v) => setState(() => _query = v),
                ),
              if (me == null)
                Padding(
                  padding: const EdgeInsets.only(top: 10),
                  child: Text(
                    'Sin ubicación todavía: las distancias se miden desde '
                    'el centro de la zona piloto.',
                    style: TextStyle(fontSize: 14, color: rl.text2),
                  ),
                ),
              const SizedBox(height: 4),
              if (_showHazards) ...[
                if (hazards.isEmpty)
                  const EmptyNote('No hay peligros reportados cerca.',
                      icon: Icons.verified_outlined),
                for (final r in hazards)
                  HazardRow(
                    report: r,
                    me: me,
                    heading: app.heading,
                    onTap: () => Navigator.of(context).push(
                        MaterialPageRoute<void>(
                            builder: (_) => ReportScreen(reportId: r.id))),
                  ),
              ] else ...[
                if (places.isEmpty)
                  EmptyNote(app.loadingPlaces
                      ? 'Descargando lugares…'
                      : 'No encontré lugares con ese nombre.'),
                for (final p in places)
                  PlaceRow(
                    place: p,
                    me: me,
                    heading: app.heading,
                    onTap: () => Navigator.of(context).push(
                        MaterialPageRoute<void>(
                            builder: (_) => PlaceScreen(place: p))),
                  ),
              ],
            ],
          );
        },
      ),
    ),
  );
 }
}
```

lib/features/place/place_screen.dart

Ficha del lugar.

```
import 'package:flutter/material.dart';
import 'package:flutter_map/flutter_map.dart';
import 'package:latlong2/latlong.dart';
import 'package:url_launcher/url_launcher.dart';

import '../../app_state.dart';
import '../../core/describe.dart';
import '../../core/geo.dart';
import '../../core/theme.dart';
import '../../models/place.dart';
import '../../widgets/base_map.dart';
import '../../widgets/map_markers.dart';
```

```
import '../../widgets/ui.dart';
import '../route/route_screen.dart';

/// Ficha de un lugar: accesibilidad, baño, notas y cómo llegar.
class PlaceScreen extends StatefulWidget {
  const PlaceScreen({super.key, required this.place});

  final Place place;

  @override
  State<PlaceScreen> createState() => _PlaceScreenState();
}

class _PlaceScreenState extends State<PlaceScreen> {
  static const _mapHeight = 260.0;
  bool _feedbackSent = false;

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addPostFrameCallback((_) {
      final app = AppScope.read(context);
      app.voice.announce(
          describePlace(widget.place, me: app.me, heading: app.heading));
    });
  }

  Future<void> _feedback(bool correct) async {
    final app = AppScope.read(context);
    await app.saveFeedback(widget.place, correct: correct);
    setState(() => _feedbackSent = true);
    await app.voice.speak(correct
        ? 'Gracias por confirmarlo.'
        : 'Gracias. Lo revisaremos para corregirlo en OpenStreetMap.');
  }

  @override
  Widget build(BuildContext context) {
    final app = AppScope.of(context);
    final p = widget.place;
    final rl = context.rl;
    final text = Theme.of(context).textTheme;
    final topInset = MediaQuery.paddingOf(context).top;

    return Scaffold(
      body: Stack(
        children: [
          // Mapa fijo con el lugar al centro.
          SizedBox(
            height: _mapHeight + topInset + 24,
            child: ExcludeSemantics(
              child: FlutterMap(
                options: MapOptions(
                  initialCenter: p.location,
                  initialZoom: 18,
                  interactionOptions:
                      const InteractionOptions(flags: InteractiveFlag.none),
                ),
                children: [
                  osmTiles(context),
                  MarkerLayer(markers: [
                    Marker(
                      point: p.location,
                      width: 72,
                      height: 72,
                      child: Center(
                        child: Container(
                          decoration: BoxDecoration(
                            shape: BoxShape.circle,
                            boxShadow: [
```

```
                            BoxShadow(
                              color: accessColor(p.access).withValues(alpha: 0.3),
                              spreadRadius: 8,
                            ),
                          ],
                        ),
                        child: AccessBadge(access: p.access, size: 52, ring: true),
                      ),
                    ),
                  ),
                  if (app.me != null)
                    Marker(
                        point: app.me!,
                        width: 32,
                        height: 32,
                        child: const UserMarker()),
                ]),
              ],
            ),
          ),
        ),
        Positioned(
          top: topInset + 12,
          left: 16,
          child: MapIconButton(
            icon: Icons.arrow_back,
            tooltip: 'Volver al mapa',
            onPressed: () => Navigator.of(context).maybePop(),
          ),
        ),
        Positioned(
          top: topInset + _mapHeight - 34,
          right: 16,
          child: const OsmCredit(),
        ),

        // Ficha.
        Positioned.fill(
          top: topInset + _mapHeight,
          child: Container(
            decoration: BoxDecoration(
              color: rl.surface,
              borderRadius: const BorderRadius.vertical(top: Radius.circular(24)),
              boxShadow: const [
                BoxShadow(
                    color: Color(0x24000000), blurRadius: 20, offset: Offset(0, -4)),
              ],
            ),
            child: ValueListenableBuilder<LatLng?>(
              valueListenable: app.location.position,
              builder: (context, me, _) {
                final meters = me == null ? null : distanceMeters(me, p.location);
                final subtitle = [
                  if (p.name != p.category) p.category,
                  if (me != null && meters != null)
                    '${formatDistanceShort(meters)} ${directionFrom(me, p.location,
app.heading)}',
                  if (meters != null) formatDuration(meters / walkingSpeed),
                ].join(' · ');
                final tone = accessTone(context, p.access);

                return ListView(
                  padding: EdgeInsets.fromLTRB(
                      20, 0, 20, 24 + MediaQuery.paddingOf(context).bottom),
                  children: [
                    const SheetHandle(),
                    const SizedBox(height: 8),
                    Row(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: [
```

```
                    Expanded(
                      child: Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: [
                          Semantics(
                            header: true,
                            child: Text(p.name, style: text.headlineMedium),
                          ),
                          if (subtitle.isNotEmpty) ...[
                            const SizedBox(height: 4),
                            Text(subtitle,
                                style:
                                    TextStyle(fontSize: 16, color: rl.text2)),
                          ],
                        ],
                      ),
                    ),
                    const SizedBox(width: 12),
                    ListenButton(
                      onPressed: () => app.voice.speak(
                          describePlace(p, me: me, heading: app.heading)),
                    ),
                  ],
                ),
                const SizedBox(height: 16),
                MergeSemantics(
                  child: Container(
                    padding: const EdgeInsets.symmetric(
                        horizontal: 16, vertical: 14),
                    decoration: BoxDecoration(
                      color: tone.background,
                      borderRadius: BorderRadius.circular(14),
                    ),
                    child: Row(
                      children: [
                        AccessBadge(access: p.access, size: 36),
                        const SizedBox(width: 12),
                        Expanded(
                          child: Column(
                            crossAxisAlignment: CrossAxisAlignment.start,
                            children: [
                              Text(p.access.label,
                                  style: TextStyle(
                                      fontSize: 18,
                                      fontWeight: FontWeight.w700,
                                      color: tone.foreground)),
                              Text(_capitalize(p.access.explanation),
                                  style: TextStyle(
                                      fontSize: 15, color: rl.text)),
                            ],
                          ),
                        ),
                      ],
                    ),
                  ),
                ),
                const SizedBox(height: 12),
                Row(
                  children: [
                    if (p.kind == PlaceKind.poi) ...[
                      Expanded(
                        child: _Fact(
                          label: 'Baño accesible',
                          value: p.accessibleToilet,
                        ),
                      ),
                      const SizedBox(width: 10),
                    ],
                    Expanded(
                      child: _Fact(
```

```
                          label: 'Guía podotáctil',
                          value: p.tactilePaving,
                        ),
                      ),
                    ],
                  ),
                  if (p.description?.trim().isNotEmpty ?? false) ...[
                    const SizedBox(height: 14),
                    Text.rich(
                      TextSpan(children: [
                        const TextSpan(
                            text: 'Nota: ',
                            style: TextStyle(fontWeight: FontWeight.w700)),
                        TextSpan(text: p.description!.trim()),
                      ]),
                      style: TextStyle(fontSize: 16, height: 1.45, color: rl.text),
                    ),
                  ],
                  const SizedBox(height: 18),
                  ElevatedButton.icon(
                    onPressed: () => Navigator.of(context).push(
                      MaterialPageRoute<void>(
                          builder: (_) => RouteScreen(destination: p)),
                    ),
                    icon: const Icon(Icons.near_me_outlined),
                    label: const Text('Cómo llegar'),
                  ),
                  const SizedBox(height: 18),
                  Divider(color: rl.border),
                  const SizedBox(height: 10),
                  if (_feedbackSent)
                    Semantics(
                      liveRegion: true,
                      child: Text('Gracias por tu ayuda.',
                          style: TextStyle(fontSize: 16, color: rl.okText)),
                    )
                  else
                    Wrap(
                      alignment: WrapAlignment.spaceBetween,
                      crossAxisAlignment: WrapCrossAlignment.center,
                      spacing: 12,
                      runSpacing: 8,
                      children: [
                        Text('¿Es correcta esta información?',
                            style: TextStyle(fontSize: 15, color: rl.text2)),
                        Row(
                          mainAxisSize: MainAxisSize.min,
                          children: [
                            _PillButton(
                                label: 'Sí', onTap: () => _feedback(true)),
                            const SizedBox(width: 8),
                            _PillButton(
                                label: 'No', onTap: () => _feedback(false)),
                          ],
                        ),
                      ],
                    ),
                  const SizedBox(height: 8),
                  Align(
                    alignment: Alignment.centerLeft,
                    child: TextButton.icon(
                      onPressed: () => launchUrl(p.osmUrl),
                      icon: const Icon(Icons.open_in_new, size: 18),
                      label: const Text('Ver o corregir en OpenStreetMap'),
                    ),
                  ),
                ],
              );
            },
          ),
```

```
            ),
          ),
        ],
      ),
    );
  }
}

String _capitalize(String s) =>
    s.isEmpty ? s : s[0].toUpperCase() + s.substring(1);

class _Fact extends StatelessWidget {
  const _Fact({required this.label, required this.value});
  final String label;
  final bool? value;

  @override
  Widget build(BuildContext context) {
    final rl = context.rl;
    final known = value != null;
    return MergeSemantics(
      child: Container(
        padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
        decoration: BoxDecoration(
          borderRadius: BorderRadius.circular(12),
          border: Border.all(color: rl.border),
        ),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(label, style: TextStyle(fontSize: 14, color: rl.text2)),
            const SizedBox(height: 4),
            Text(
              !known ? 'Sin dato' : (value! ? 'Sí' : 'No'),
              style: TextStyle(
                  fontSize: 17,
                  fontWeight: FontWeight.w700,
                  color: known ? rl.text : rl.text2),
            ),
          ],
        ),
      ),
    );
  }
}

class _PillButton extends StatelessWidget {
  const _PillButton({required this.label, required this.onTap});
  final String label;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) => OutlinedButton(
        style: OutlinedButton.styleFrom(
          minimumSize: const Size(64, 44),
          shape: const StadiumBorder(),
          textStyle: const TextStyle(fontSize: 16, fontWeight: FontWeight.w400),
        ),
        onPressed: onTap,
        child: Text(label),
      );
}
```

lib/features/route/route_screen.dart

Pantalla de ruta.

```
import 'package:flutter/material.dart';
```

```
import 'package:flutter_map/flutter_map.dart';

import '../../app_state.dart';
import '../../config.dart';
import '../../core/geo.dart';
import '../../core/route_api.dart';
import '../../core/theme.dart';
import '../../models/place.dart';
import '../../models/route_result.dart';
import '../../services/routing_service.dart';
import '../../widgets/base_map.dart';
import '../../widgets/map_markers.dart';
import '../../widgets/ui.dart';

/// Calcula la ruta a [destination] (o muestra la ruta activa) sobre el mapa,
/// con sus indicaciones en una hoja inferior.
class RouteScreen extends StatefulWidget {
  const RouteScreen({super.key, this.destination});

  /// Null = mostrar la ruta que ya está activa en el mapa.
  final Place? destination;

  @override
  State<RouteScreen> createState() => _RouteScreenState();
}

class _RouteScreenState extends State<RouteScreen> {
  RouteResult? _route;
  String? _error;
  bool _loading = false;
  final _map = MapController();
  bool _mapReady = false;

  @override
  void initState() {
    super.initState();
    if (widget.destination == null) {
      _route = AppScope.read(context).route;
    } else {
      _loading = true;
    }
    WidgetsBinding.instance.addPostFrameCallback((_) {
      final app = AppScope.read(context);
      if (widget.destination == null) {
        if (_route != null) app.voice.announce(spokenSummary(_route!));
      } else {
        _calculate();
      }
    });
  }

  Future<void> _calculate() async {
    final app = AppScope.read(context);
    setState(() {
      _loading = true;
      _error = null;
    });
    await app.voice.announce('Calculando una ruta accesible.');
    try {
      final route = await app.routeTo(widget.destination!);
      if (!mounted) return;
      setState(() => _route = route);
      _fit(route);
      await app.voice.speak(
          '${spokenSummary(route)} Toca Iniciar guía por voz para escucharla.');
    } on RoutingException catch (e) {
      if (!mounted) return;
      setState(() => _error = e.message);
      await app.voice.speak(e.message);
    } finally {
```

```
    if (mounted) setState(() => _loading = false);
  }
}

void _fit(RouteResult route) {
  if (!_mapReady || route.points.length < 2) return;
  _map.fitCamera(CameraFit.bounds(
    bounds: LatLngBounds.fromPoints(route.points),
    padding: _fitPadding,
  ));
}

/// Deja libre la tarjeta de arriba y la hoja de abajo.
EdgeInsets get _fitPadding {
  final h = MediaQuery.sizeOf(context).height;
  return EdgeInsets.fromLTRB(40, 120, 40, h * 0.5 + 24);
}

Future<void> _speakAll(AppState app, RouteResult route) async {
  final text = [
    spokenSummary(route),
    for (final s in route.steps) spokenStep(s),
  ].join(' ');
  await app.voice.speak(text);
}

IconData _stepIcon(int type) => switch (type) {
      0 || 2 || 4 || 12 => Icons.turn_left,
      1 || 3 || 5 || 13 => Icons.turn_right,
      7 || 8 => Icons.roundabout_left,
      9 => Icons.u_turn_left,
      10 => Icons.flag_outlined,
      11 => Icons.directions_walk,
      _ => Icons.straight,
    };

@override
Widget build(BuildContext context) {
  final app = AppScope.of(context);
  final rl = context.rl;
  final route = _route;
  final topInset = MediaQuery.paddingOf(context).top;
  final destination = widget.destination?.name ?? route?.destinationName ?? '';
  final destPoint = widget.destination?.location ??
      (route == null || route.points.isEmpty ? null : route.points.last);

  return Scaffold(
    body: Stack(
      children: [
        Positioned.fill(
          child: ExcludeSemantics(
            child: FlutterMap(
              mapController: _map,
              options: MapOptions(
                initialCenter: destPoint ?? app.me ?? AppConfig.pilotCenter,
                initialZoom: 17,
                minZoom: 12,
                maxZoom: 19,
                onMapReady: () {
                  _mapReady = true;
                  if (_route != null) _fit(_route!);
                },
              ),
              children: [
                osmTiles(context),
                if (route != null) routeLine(context, route),
                MarkerLayer(markers: [
                  for (final r in app.activeReports())
                    Marker(
                      point: r.location,
```

```
                    width: 44,
                    height: 44,
                    child: Center(
                      child: HazardTriangle(
                          color: hazardColor(r.severity), size: 36),
                    ),
                  ),
              if (destPoint != null)
                Marker(
                  point: destPoint,
                  width: 44,
                  height: 44,
                  child: Center(
                    child: AccessBadge(
                      access: widget.destination?.access ??
                          Accessibility.yes,
                      size: 34,
                      ring: true,
                    ),
                  ),
                ),
              if (app.me != null)
                Marker(
                    point: app.me!,
                    width: 32,
                    height: 32,
                    child: const UserMarker()),
            ]),
          ],
        ),
      ),
    ),

    // Tarjeta superior con el destino.
    Positioned(
      top: topInset + 12,
      left: 16,
      right: 16,
      child: Container(
        padding: const EdgeInsets.fromLTRB(6, 8, 14, 8),
        decoration: BoxDecoration(
          color: rl.surface,
          borderRadius: BorderRadius.circular(18),
          border: Border.all(color: rl.border),
          boxShadow: cardShadow,
        ),
        child: Row(
          children: [
            IconButton(
              tooltip: 'Volver',
              onPressed: () => Navigator.of(context).maybePop(),
              icon: Icon(Icons.arrow_back, color: rl.text),
            ),
            const SizedBox(width: 4),
            Expanded(
              child: MergeSemantics(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      app.profile.orsProfile == 'wheelchair'
                          ? 'Ruta en silla de ruedas a'
                          : 'Ruta a pie a',
                      style: TextStyle(fontSize: 14, color: rl.text2),
                    ),
                    Text(destination,
                        maxLines: 1,
                        overflow: TextOverflow.ellipsis,
                        style: TextStyle(
                            fontSize: 19,
```

```
                                fontWeight: FontWeight.w700,
                                color: rl.text)),
                          ],
                        ),
                      ),
                    ),
                  ],
                ),
              ),
            ),
            Positioned(
              right: 16,
              bottom: MediaQuery.sizeOf(context).height * 0.48 + 10,
              child: const OsmCredit(),
            ),

            // Hoja con las indicaciones.
            DraggableScrollableSheet(
              initialChildSize: 0.48,
              minChildSize: 0.3,
              maxChildSize: 0.92,
              builder: (context, scroll) => Container(
                decoration: BoxDecoration(
                  color: rl.surface,
                  borderRadius:
                      const BorderRadius.vertical(top: Radius.circular(24)),
                  boxShadow: const [
                    BoxShadow(
                        color: Color(0x24000000),
                        blurRadius: 20,
                        offset: Offset(0, -4)),
                  ],
                ),
                child: Column(
                  children: [
                    Expanded(
                      child: ListView(
                        controller: scroll,
                        padding: const EdgeInsets.fromLTRB(20, 0, 20, 12),
                        children: [
                          const SheetHandle(),
                          const SizedBox(height: 8),
                          ..._sheetBody(app, route),
                        ],
                      ),
                    ),
                    if (route != null)
                      SafeArea(
                        top: false,
                        child: Padding(
                          padding: const EdgeInsets.fromLTRB(20, 4, 20, 16),
                          child: ElevatedButton.icon(
                            onPressed: () => _speakAll(app, route),
                            icon: const Icon(Icons.volume_up_outlined),
                            label: const Text('Iniciar guía por voz'),
                          ),
                        ),
                      ),
                  ],
                ),
              ),
            ),
          ],
        ),
      );
}

List<Widget> _sheetBody(AppState app, RouteResult? route) {
  final rl = context.rl;
  if (_loading) {
```

```
    return [
      const SizedBox(height: 24),
      Semantics(
        liveRegion: true,
        label: 'Calculando ruta',
        child: const Center(child: CircularProgressIndicator()),
      ),
      const SizedBox(height: 16),
      Center(
        child: Text('Buscando un camino sin obstáculos…',
            style: TextStyle(fontSize: 16, color: rl.text2)),
      ),
    ];
  }
  if (_error != null) {
    return [
      const SizedBox(height: 12),
      const Icon(Icons.wrong_location_outlined,
          size: 48, color: AppTheme.accessNo),
      const SizedBox(height: 12),
      Text(_error!,
          textAlign: TextAlign.center,
          style: TextStyle(fontSize: 18, color: rl.text)),
      const SizedBox(height: 20),
      if (widget.destination != null)
        ElevatedButton(
            onPressed: _calculate, child: const Text('Intentar otra vez')),
    ];
  }
  if (route == null) {
    return [const EmptyNote('No hay una ruta activa.', icon: Icons.route)];
  }

  return [
    Semantics(
      label: spokenSummary(route),
      excludeSemantics: true,
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.baseline,
        textBaseline: TextBaseline.alphabetic,
        children: [
          Text(formatDuration(route.durationSeconds),
              style: TextStyle(
                  fontSize: 30, fontWeight: FontWeight.w700, color: rl.text)),
          const SizedBox(width: 10),
          Text('· ${formatDistanceShort(route.distanceMeters)}',
              style: TextStyle(fontSize: 18, color: rl.text2)),
        ],
      ),
    ),
    const SizedBox(height: 10),
    Wrap(
      spacing: 8,
      runSpacing: 8,
      children: [
        if (app.profile.avoidSteps)
          Tag(
              label: 'Sin escaleras',
              background: rl.okSoft,
              foreground: rl.okText),
        if (route.avoidedHazards > 0)
          Tag(
            label: route.avoidedHazards == 1
                ? 'Evita 1 peligro reportado'
                : 'Evita ${route.avoidedHazards} peligros reportados',
            icon: Icons.warning_amber_rounded,
            background: rl.hazardSoft,
            foreground: rl.hazardText,
          ),
      ],
```

```
      ),
      const SizedBox(height: 6),
      for (var i = 0; i < route.steps.length; i++)
        Semantics(
          button: true,
          label: 'Paso ${i + 1}. ${spokenStep(route.steps[i])}',
          excludeSemantics: true,
          child: InkWell(
            onTap: () => app.voice.speak(spokenStep(route.steps[i])),
            child: Container(
              constraints: const BoxConstraints(minHeight: 56),
              padding: const EdgeInsets.symmetric(vertical: 10),
              decoration: BoxDecoration(
                border: i == route.steps.length - 1
                    ? null
                    : Border(bottom: BorderSide(color: rl.border)),
              ),
              child: Row(
                children: [
                  Container(
                    width: 36,
                    height: 36,
                    decoration: BoxDecoration(
                      color: rl.surface2,
                      borderRadius: BorderRadius.circular(10),
                    ),
                    child: Icon(_stepIcon(route.steps[i].type),
                        size: 20, color: rl.text),
                  ),
                  const SizedBox(width: 14),
                  Expanded(
                    child: Text(route.steps[i].instruction,
                        style: TextStyle(fontSize: 17, color: rl.text)),
                  ),
                  if (route.steps[i].distanceMeters >= 5) ...[
                    const SizedBox(width: 10),
                    Text(formatDistanceShort(route.steps[i].distanceMeters),
                        style: TextStyle(fontSize: 15, color: rl.text2)),
                  ],
                ],
              ),
            ),
          ),
        ),
      const SizedBox(height: 10),
      Text(
        'Ruta sugerida con datos de OpenStreetMap: puede haber tramos sin '
        'información. Confirma el camino al avanzar.',
        style: TextStyle(fontSize: 14, color: rl.text2),
      ),
    ];
  }
}
```

lib/features/report/new_report_screen.dart

Pantalla para reportar.

```
import 'dart:io';

import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:image_picker/image_picker.dart';
import 'package:path_provider/path_provider.dart';

import '../../app_state.dart';
import '../../core/theme.dart';
```

```
import '../../models/report.dart';
import '../../widgets/report_photo.dart';
import '../../widgets/ui.dart';

/// "6 horas", "3 días".
String formatTtl(Duration d) {
  if (d.inHours < 24) return d.inHours == 1 ? '1 hora' : '${d.inHours} horas';
  return d.inDays == 1 ? '1 día' : '${d.inDays} días';
}

IconData reportIcon(ReportType t) => switch (t) {
      ReportType.blockedRamp => Icons.directions_car_outlined,
      ReportType.openDrain => Icons.blur_circular,
      ReportType.brokenSidewalk => Icons.timeline,
      ReportType.construction => Icons.construction,
      ReportType.noRamp => Icons.stairs_outlined,
      ReportType.elevatorOut => Icons.elevator_outlined,
      ReportType.flooding => Icons.water_drop_outlined,
      ReportType.other => Icons.error_outline,
    };

/// Reportar un peligro en tres toques: tipo, gravedad y enviar.
///
/// La ubicación sale del GPS; foto y nota son opcionales.
class NewReportScreen extends StatefulWidget {
  const NewReportScreen({super.key});

  @override
  State<NewReportScreen> createState() => _NewReportScreenState();
}

class _NewReportScreenState extends State<NewReportScreen> {
  ReportType? _type;
  Severity _severity = Severity.medium;
  final _note = TextEditingController();
  String? _photoPath;
  bool _sending = false;

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addPostFrameCallback((_) {
      AppScope.read(context).voice.announce(
            'Reportar peligro. Elige qué encontraste. '
            'Se usará tu ubicación actual.',
          );
    });
  }

  @override
  void dispose() {
    _note.dispose();
    super.dispose();
  }

  void _pickType(ReportType t) {
    HapticFeedback.selectionClick();
    setState(() {
      _type = t;
      _severity = t.defaultSeverity;
    });
    AppScope.read(context).voice.announce(
          '${t.label}. Gravedad ${t.defaultSeverity.label.toLowerCase()}. '
          'Puedes cambiarla, agregar foto o tocar Enviar reporte.',
        );
  }

  Future<void> _takePhoto() async {
    final picked = await ImagePicker().pickImage(
      source: ImageSource.camera,
```

```
    maxWidth: 1280,
    imageQuality: 70,
  );
  if (picked == null) return;
  final String dest;
  if (kIsWeb) {
    // En web es una URL temporal del navegador.
    dest = picked.path;
  } else {
    // La cámara guarda en caché; se copia para que no se borre.
    final dir = await getApplicationDocumentsDirectory();
    dest = '${dir.path}/reporte_${DateTime.now().millisecondsSinceEpoch}.jpg';
    await File(picked.path).copy(dest);
  }
  if (!mounted) return;
  setState(() => _photoPath = dest);
  await AppScope.read(context).voice.announce('Foto agregada.');
}

Future<void> _send() async {
  final app = AppScope.read(context);
  final type = _type;
  final here = app.me;
  if (type == null) return;
  if (here == null) {
    await app.voice.speak(
        'Todavía no tengo tu ubicación. Revisa que el GPS esté encendido.');
    return;
  }
  setState(() => _sending = true);
  await app.addReport(
    type: type,
    severity: _severity,
    at: here,
    note: _note.text.trim(),
    photoPath: _photoPath,
  );
  await HapticFeedback.heavyImpact();
  await app.voice.speak(
      'Reporte enviado: ${type.label}. Gracias por avisar a los demás.');
  if (mounted) Navigator.of(context).pop();
}

@override
Widget build(BuildContext context) {
  final app = AppScope.of(context);
  final rl = context.rl;
  final text = Theme.of(context).textTheme;
  final accuracy = app.location.accuracy;
  final type = _type;
  final canSend = type != null && app.me != null && !_sending;

  Widget heading(String s) => Padding(
        padding: const EdgeInsets.only(bottom: 10),
        child: Semantics(
          header: true,
          child: Text(s,
              style: TextStyle(
                  fontSize: 17, fontWeight: FontWeight.w700, color: rl.text)),
        ),
      );

  return Scaffold(
    appBar: AppBar(
      leading: IconButton(
        tooltip: 'Cerrar',
        icon: const Icon(Icons.close),
        onPressed: () => Navigator.of(context).maybePop(),
      ),
      titleSpacing: 0,
```

```
        title: const Text('Reportar peligro'),
      ),
      body: SafeArea(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Expanded(
              child: ListView(
                padding: const EdgeInsets.fromLTRB(20, 4, 20, 16),
                children: [
                  MergeSemantics(
                    child: Container(
                      padding: const EdgeInsets.symmetric(
                          horizontal: 14, vertical: 12),
                      decoration: BoxDecoration(
                        color: rl.surface,
                        borderRadius: BorderRadius.circular(14),
                        border: Border.all(color: rl.border),
                      ),
                      child: Row(
                        children: [
                          Container(
                            width: 40,
                            height: 40,
                            decoration: BoxDecoration(
                              color: app.me == null
                                  ? rl.hazardSoft
                                  : rl.primarySoft,
                              borderRadius: BorderRadius.circular(12),
                            ),
                            child: Icon(
                                app.me == null
                                    ? Icons.location_searching
                                    : Icons.location_on_outlined,
                                color: app.me == null
                                    ? rl.hazardText
                                    : rl.primary),
                          ),
                          const SizedBox(width: 12),
                          Expanded(
                            child: Column(
                              crossAxisAlignment: CrossAxisAlignment.start,
                              children: [
                                Text(
                                    app.me == null
                                        ? 'Buscando tu ubicación…'
                                        : 'Se reportará donde estás',
                                    style: TextStyle(
                                        fontSize: 16,
                                        fontWeight: FontWeight.w700,
                                        color: rl.text)),
                                Text(
                                    app.me == null
                                        ? 'Revisa que el GPS esté encendido'
                                        : 'Ubicación por GPS'
                                            '${accuracy == null ? '' : ' · precisión ${accuracy.round()} m'}',
                                    style: TextStyle(
                                        fontSize: 14, color: rl.text2)),
                              ],
                            ),
                          ),
                        ],
                      ),
                    ),
                  ),
                  const SizedBox(height: 20),
                  heading('¿Qué encontraste?'),
                  LayoutBuilder(builder: (context, c) {
                    final w = (c.maxWidth - 10) / 2;
```

```
                  return Wrap(
                    spacing: 10,
                    runSpacing: 10,
                    children: [
                      for (final t in ReportType.values)
                        SizedBox(
                          width: w,
                          child: _TypeOption(
                            icon: reportIcon(t),
                            label: t.label,
                            selected: _type == t,
                            onTap: () => _pickType(t),
                          ),
                        ),
                    ],
                  );
                }),
                const SizedBox(height: 20),
                heading('Gravedad'),
                Segmented<Severity>(
                  values: Severity.values,
                  selected: _severity,
                  labelOf: (s) => s.label,
                  semanticPrefix: 'Gravedad ',
                  onChanged: (s) {
                    HapticFeedback.selectionClick();
                    setState(() => _severity = s);
                  },
                ),
                const SizedBox(height: 20),
                OutlinedButton.icon(
                  style: OutlinedButton.styleFrom(
                    side: BorderSide(color: rl.text2),
                    textStyle:
                        const TextStyle(fontSize: 16, fontWeight: FontWeight.w400),
                  ),
                  onPressed: _sending ? null : _takePhoto,
                  icon: Icon(_photoPath == null
                      ? Icons.photo_camera_outlined
                      : Icons.check),
                  label: Text(_photoPath == null
                      ? 'Agregar foto (opcional)'
                      : 'Cambiar foto'),
                ),
                if (_photoPath != null) ...[
                  const SizedBox(height: 10),
                  ClipRRect(
                    borderRadius: BorderRadius.circular(14),
                    child: ReportPhoto(path: _photoPath!, height: 180),
                  ),
                ],
                const SizedBox(height: 6),
                Text('Evita que salgan rostros o placas en la foto.',
                    style: TextStyle(fontSize: 14, color: rl.text2)),
                const SizedBox(height: 16),
                TextField(
                  controller: _note,
                  style: text.bodyLarge,
                  maxLines: 2,
                  maxLength: 140,
                  decoration: const InputDecoration(
                    labelText: 'Nota (opcional)',
                    hintText: 'Ej. frente a la farmacia',
                  ),
                ),
              ],
            ),
          ),
          Container(
            padding: const EdgeInsets.fromLTRB(20, 12, 20, 16),
```

```
              decoration: BoxDecoration(
                color: Theme.of(context).scaffoldBackgroundColor,
                border: Border(top: BorderSide(color: rl.border)),
              ),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                children: [
                  ElevatedButton(
                    style: ElevatedButton.styleFrom(
                      backgroundColor: AppTheme.hazard,
                      foregroundColor: Colors.white,
                    ),
                    onPressed: canSend ? _send : null,
                    child: Text(_sending
                        ? 'Enviando…'
                        : (type == null ? 'Elige qué encontraste' : 'Enviar reporte')),
                  ),
                  const SizedBox(height: 8),
                  Text(
                    type == null
                        ? 'Los reportes caducan solos si nadie confirma que siguen ahí'
                        : 'Caduca en ${formatTtl(type.ttl)} si nadie confirma que sigue ahí',
                    textAlign: TextAlign.center,
                    style: TextStyle(fontSize: 14, color: rl.text2),
                  ),
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }
}

class _TypeOption extends StatelessWidget {
  const _TypeOption({
    required this.icon,
    required this.label,
    required this.selected,
    required this.onTap,
  });

  final IconData icon;
  final String label;
  final bool selected;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    final rl = context.rl;
    return Semantics(
      button: true,
      selected: selected,
      inMutuallyExclusiveGroup: true,
      label: label,
      excludeSemantics: true,
      child: Material(
        color: selected ? rl.hazardSoft : rl.surface,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(14),
          side: BorderSide(
            color: selected ? AppTheme.hazard : rl.border,
            width: selected ? 2 : 1,
          ),
        ),
        child: InkWell(
          borderRadius: BorderRadius.circular(14),
          onTap: onTap,
          child: Container(
```

```
            constraints: const BoxConstraints(minHeight: 64),
            padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 10),
            child: Row(
              children: [
                Icon(icon,
                    size: 22, color: selected ? AppTheme.hazard : rl.text),
                const SizedBox(width: 10),
                Expanded(
                  child: Text(label,
                      style: TextStyle(
                        fontSize: 15,
                        height: 1.2,
                        color: selected ? rl.hazardText : rl.text,
                        fontWeight:
                            selected ? FontWeight.w700 : FontWeight.w400,
                      )),
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

lib/features/report/report_screen.dart

Detalle de un reporte.

```
import 'package:flutter/material.dart';

import '../../app_state.dart';
import '../../core/describe.dart';
import '../../core/geo.dart';
import '../../core/theme.dart';
import '../../models/report.dart';
import '../../widgets/map_markers.dart';
import '../../widgets/report_photo.dart';
import '../../widgets/ui.dart';
import 'new_report_screen.dart';

/// Detalle de un peligro reportado, con "Sigue ahí" y "Ya no está".
class ReportScreen extends StatefulWidget {
  const ReportScreen({super.key, required this.reportId});

  final String reportId;

  @override
  State<ReportScreen> createState() => _ReportScreenState();
}

class _ReportScreenState extends State<ReportScreen> {
  bool _voted = false;

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addPostFrameCallback((_) {
      final app = AppScope.read(context);
      final r = app.reportById(widget.reportId);
      if (r == null) return;
      app.voice.announce(
        '${describeReport(r, DateTime.now(), me: app.me, heading: app.heading)} '
        '¿Sigue ahí?',
      );
    });
  }
```

```
Future<void> _confirm(AppState app, Report r) async {
  await app.confirmReport(r);
  setState(() => _voted = true);
  await app.voice.speak('Gracias. El reporte seguirá activo.');
}

Future<void> _gone(AppState app, Report r) async {
  final updated = await app.voteGone(r);
  setState(() => _voted = true);
  await app.voice.speak(updated.closed
      ? 'Gracias. El reporte se cerró.'
      : 'Gracias. Si alguien más confirma que ya no está, se cerrará.');
}

@override
Widget build(BuildContext context) {
  final app = AppScope.of(context);
  final r = app.reportById(widget.reportId);
  final rl = context.rl;
  final text = Theme.of(context).textTheme;
  final now = DateTime.now();

  if (r == null) {
    return Scaffold(
      appBar: AppBar(),
      body: const EmptyNote('Este reporte ya no existe.',
          icon: Icons.check_circle_outline),
    );
  }

  final active = r.isActive(now);
  final me = app.me;

  Widget fact(String label, String value) => MergeSemantics(
        child: Padding(
          padding: const EdgeInsets.symmetric(vertical: 10),
          child: Row(
            children: [
              Expanded(
                child: Text(label,
                    style: TextStyle(fontSize: 15, color: rl.text2)),
              ),
              Text(value,
                  style: TextStyle(
                      fontSize: 16,
                      fontWeight: FontWeight.w700,
                      color: rl.text)),
            ],
          ),
        ),
      );

  return Scaffold(
    appBar: AppBar(title: const Text('Peligro reportado')),
    body: SafeArea(
      child: ListView(
        padding: const EdgeInsets.fromLTRB(20, 4, 20, 24),
        children: [
          Row(
            children: [
              HazardTriangle(color: hazardColor(r.severity), size: 52),
              const SizedBox(width: 14),
              Expanded(
                child: Semantics(
                  header: true,
                  child: Text(r.type.label, style: text.headlineSmall),
                ),
              ),
              ListenButton(
```

```
              onPressed: () => app.voice
                  .speak(describeReport(r, now, me: me, heading: app.heading)),
            ),
          ],
        ),
        const SizedBox(height: 12),
        Wrap(
          spacing: 8,
          runSpacing: 8,
          children: [
            Tag(
              label: 'Gravedad ${r.severity.label.toLowerCase()}',
              icon: reportIcon(r.type),
              background: rl.hazardSoft,
              foreground: rl.hazardText,
            ),
            if (!active)
              Tag(
                  label: 'Ya no está activo',
                  background: rl.surface2,
                  foreground: rl.text2),
          ],
        ),
        const SizedBox(height: 12),
        Container(
          padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),
          decoration: BoxDecoration(
            color: rl.surface,
            borderRadius: BorderRadius.circular(14),
            border: Border.all(color: rl.border),
          ),
          child: Column(
            children: [
              if (me != null)
                fact('Dónde',
                    '${formatDistanceShort(distanceMeters(me, r.location))} '
                    '${directionFrom(me, r.location, app.heading)}'),
              fact('Reportado', timeAgo(r.createdAt, now)),
              fact('Confirmado', timeAgo(r.lastConfirmedAt, now)),
              fact(active ? 'Caduca' : 'Estado',
                  active ? timeUntil(r.expiresAt, now) : 'Cerrado'),
            ],
          ),
        ),
        if (r.note.isNotEmpty) ...[
          const SizedBox(height: 14),
          Text('“${r.note}”',
              style: TextStyle(fontSize: 17, height: 1.4, color: rl.text)),
        ],
        if (ReportPhoto.exists(r.photoPath)) ...[
          const SizedBox(height: 16),
          ClipRRect(
            borderRadius: BorderRadius.circular(14),
            child: ReportPhoto(path: r.photoPath!, height: 220),
          ),
        ],
        if (active) ...[
          const SizedBox(height: 24),
          Semantics(
            header: true,
            child: Text('¿Sigue ahí?', style: text.titleLarge),
          ),
          const SizedBox(height: 4),
          Text('Tu respuesta mantiene el mapa al día para los demás.',
              style: TextStyle(fontSize: 15, color: rl.text2)),
          const SizedBox(height: 12),
          if (_voted)
            Semantics(
              liveRegion: true,
              child: Text('Gracias por confirmar.',
```

```
                    style: TextStyle(fontSize: 17, color: rl.okText)),
              )
            else
              Row(
                children: [
                  Expanded(
                    child: ElevatedButton(
                      style: ElevatedButton.styleFrom(
                        backgroundColor: AppTheme.hazard,
                        foregroundColor: Colors.white,
                      ),
                      onPressed: () => _confirm(app, r),
                      child: const Text('Sigue ahí'),
                    ),
                  ),
                  const SizedBox(width: 12),
                  Expanded(
                    child: OutlinedButton(
                      style: OutlinedButton.styleFrom(
                          minimumSize: const Size.fromHeight(56)),
                      onPressed: () => _gone(app, r),
                      child: const Text('Ya no está'),
                    ),
                  ),
                ],
              ),
          ],
        ],
      ),
    ),
  );
 }
}
```

lib/features/settings/settings_screen.dart

Ajustes.

```
import 'package:flutter/material.dart';

import '../../app_state.dart';
import '../../config.dart';
import '../../core/theme.dart';
import '../../models/settings.dart';
import '../../widgets/ui.dart';
import '../onboarding/profile_screen.dart';

/// Perfil, voz, avisos, tamaño de letra y privacidad.
class SettingsScreen extends StatefulWidget {
  const SettingsScreen({super.key});

  @override
  State<SettingsScreen> createState() => _SettingsScreenState();
}

class _SettingsScreenState extends State<SettingsScreen> {
  late final TextEditingController _orsKey;
  bool _showKey = false;

  @override
  void initState() {
    super.initState();
    _orsKey = TextEditingController(text: AppScope.read(context).settings.orsApiKey);
    WidgetsBinding.instance.addPostFrameCallback(
        (_) => AppScope.read(context).voice.announce('Ajustes'));
  }

  @override
```

```
void dispose() {
  _orsKey.dispose();
  super.dispose();
}

Future<void> _saveKey(AppState app) async {
  await app.updateSettings(app.settings.copyWith(orsApiKey: _orsKey.text.trim()));
  if (!mounted) return;
  FocusScope.of(context).unfocus();
  await app.voice.speak(app.hasRoutingKey
      ? 'Clave guardada. Ya puedes pedir rutas.'
      : 'Clave borrada.');
}

@override
Widget build(BuildContext context) {
  final app = AppScope.of(context);
  final s = app.settings;
  final rl = context.rl;
  final muted = TextStyle(fontSize: 14, color: rl.text2);
  TextStyle title() =>
      TextStyle(fontSize: 17, fontWeight: FontWeight.w700, color: rl.text);

  return Scaffold(
    appBar: AppBar(title: const Text('Ajustes')),
    body: SafeArea(
      child: ListView(
        padding: const EdgeInsets.fromLTRB(16, 4, 16, 24),
        children: [
          _Section(
            title: 'Perfil',
            children: [
              ListTile(
                contentPadding: const EdgeInsets.symmetric(horizontal: 16),
                leading: Container(
                  width: 44,
                  height: 44,
                  decoration: BoxDecoration(
                    color: rl.primarySoft,
                    borderRadius: BorderRadius.circular(12),
                  ),
                  child: Icon(Icons.person_outline, color: rl.primary),
                ),
                title: Text(app.profile.label, style: title()),
                subtitle: Text(app.profile.description, style: muted),
                trailing: Icon(Icons.chevron_right, color: rl.text2),
                onTap: () => Navigator.of(context).push(MaterialPageRoute<void>(
                    builder: (_) => const ProfileScreen())),
              ),
            ],
          ),
          _Section(
            title: 'Apariencia',
            children: [
              Padding(
                padding: const EdgeInsets.fromLTRB(16, 14, 16, 4),
                child: Text('Tema', style: title()),
              ),
              Padding(
                padding: const EdgeInsets.fromLTRB(16, 6, 16, 14),
                child: Segmented<ThemePreference>(
                  values: ThemePreference.values,
                  selected: s.theme,
                  labelOf: (t) => t.label,
                  semanticPrefix: 'Tema ',
                  onChanged: (t) => app.updateSettings(s.copyWith(theme: t)),
                ),
              ),
              const Divider(indent: 16, endIndent: 16),
              Padding(
```

```
              padding: const EdgeInsets.fromLTRB(16, 14, 16, 0),
              child: Text('Tamaño de letra', style: title()),
            ),
            Padding(
              padding: const EdgeInsets.fromLTRB(16, 4, 16, 0),
              child: Text('Así se verá el texto de la app.', style: muted),
            ),
            Slider(
              value: s.textScale,
              min: 1.0,
              max: 2.0,
              divisions: 5,
              semanticFormatterCallback: (v) =>
                  '${(v * 100).round()} por ciento',
              onChanged: (v) => app.updateSettings(s.copyWith(textScale: v)),
            ),
          ],
        ),
        _Section(
          title: 'Voz y avisos',
          children: [
            SwitchListTile(
              title: Text('Guía por voz', style: title()),
              subtitle: Text('Anunciar cada pantalla y acción', style: muted),
              value: s.voiceGuidance,
              onChanged: (v) =>
                  app.updateSettings(s.copyWith(voiceGuidance: v)),
            ),
            const Divider(indent: 16, endIndent: 16),
            SwitchListTile(
              title: Text('Avisar peligros cercanos', style: title()),
              subtitle: Text(
                  'Voz y vibración a ${AppConfig.hazardAlertMeters.round()} m '
                  'de un peligro reportado',
                  style: muted),
              value: s.hazardAlerts,
              onChanged: (v) =>
                  app.updateSettings(s.copyWith(hazardAlerts: v)),
            ),
            const Divider(indent: 16, endIndent: 16),
            Padding(
              padding: const EdgeInsets.fromLTRB(16, 14, 16, 0),
              child: Text('Velocidad de la voz', style: title()),
            ),
            Slider(
              value: s.speechRate,
              min: 0.3,
              max: 0.7,
              divisions: 4,
              semanticFormatterCallback: (v) =>
                  v < 0.45 ? 'lenta' : (v > 0.55 ? 'rápida' : 'normal'),
              onChanged: (v) => app.updateSettings(s.copyWith(speechRate: v)),
            ),
            Padding(
              padding: const EdgeInsets.fromLTRB(16, 0, 16, 14),
              child: Align(
                alignment: Alignment.centerLeft,
                child: FilledButton.icon(
                  onPressed: () =>
                      app.voice.speak('Así sueno con esta velocidad.'),
                  icon: const Icon(Icons.record_voice_over_outlined, size: 20),
                  label: const Text('Probar voz'),
                ),
              ),
            ),
          ],
        ),
        _Section(
          title: 'Rutas',
          children: [
```

```
              Padding(
                padding: const EdgeInsets.all(16),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.stretch,
                  children: [
                    Row(
                      children: [
                        Icon(
                            app.hasRoutingKey
                                ? Icons.check_circle
                                : Icons.info_outline,
                            color: app.hasRoutingKey
                                ? AppTheme.accessYes
                                : rl.text2),
                        const SizedBox(width: 10),
                        Expanded(
                          child: Text(
                            app.hasRoutingKey
                                ? 'Las rutas están activas.'
                                : 'Para calcular rutas hace falta una clave '
                                    'gratuita de openrouteservice.org. Se '
                                    'guarda solo en este dispositivo.',
                            style: TextStyle(fontSize: 15, color: rl.text),
                          ),
                        ),
                      ],
                    ),
                    const SizedBox(height: 12),
                    TextField(
                      controller: _orsKey,
                      obscureText: !_showKey,
                      autocorrect: false,
                      enableSuggestions: false,
                      decoration: InputDecoration(
                        labelText: 'Clave de openrouteservice',
                        suffixIcon: IconButton(
                          tooltip: _showKey ? 'Ocultar clave' : 'Mostrar clave',
                          icon: Icon(_showKey
                              ? Icons.visibility_off_outlined
                              : Icons.visibility_outlined),
                          onPressed: () => setState(() => _showKey = !_showKey),
                        ),
                      ),
                    ),
                    const SizedBox(height: 12),
                    OutlinedButton(
                      onPressed: () => _saveKey(app),
                      child: const Text('Guardar clave'),
                    ),
                  ],
                ),
              ),
            ],
          ),
          _Section(
            title: 'Datos del mapa',
            children: [
              ListTile(
                title: Text('Actualizar lugares', style: title()),
                subtitle: Text(
                    'Vienen de OpenStreetMap y se guardan en el teléfono por '
                    '${AppConfig.placesCacheTtl.inDays} días.',
                    style: muted),
                trailing: app.loadingPlaces
                    ? const SizedBox(
                        width: 22,
                        height: 22,
                        child: CircularProgressIndicator(strokeWidth: 2.5))
                    : Icon(Icons.refresh, color: rl.primary),
                onTap: app.loadingPlaces
```

```
                      ? null
                      : () async {
                          await app.loadPlaces(AppConfig.pilotCenter, force: true);
                          await app.voice
                              .speak(app.placesError ?? 'Lugares actualizados.');
                        },
                ),
              ],
            ),
            _Section(
              title: 'Privacidad',
              children: [
                Padding(
                  padding: const EdgeInsets.all(16),
                  child: Text(
                    'Tu ubicación se usa en el teléfono para mostrar qué hay '
                    'cerca. En este prototipo los reportes se guardan solo en '
                    'tu teléfono. Cuando se compartan, se enviará la ubicación '
                    'del peligro, nunca tu recorrido.',
                    style: TextStyle(fontSize: 15, height: 1.4, color: rl.text),
                  ),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}

/// Grupo de ajustes con título y tarjeta.
class _Section extends StatelessWidget {
  const _Section({required this.title, required this.children});
  final String title;
  final List<Widget> children;

  @override
  Widget build(BuildContext context) {
    final rl = context.rl;
    return Padding(
      padding: const EdgeInsets.only(top: 16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Padding(
            padding: const EdgeInsets.only(left: 4, bottom: 8),
            child: Semantics(
              header: true,
              child: Text(title.toUpperCase(),
                  style: TextStyle(
                      fontSize: 13,
                      letterSpacing: 0.8,
                      fontWeight: FontWeight.w700,
                      color: rl.text2)),
            ),
          ),
          Card(
            clipBehavior: Clip.antiAlias,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: children,
            ),
          ),
        ],
      ),
    );
  }
}
```

test/geo_test.dart

Pruebas de geografía.

```
import 'package:flutter_test/flutter_test.dart';
import 'package:latlong2/latlong.dart';
import 'package:ruta_libre/core/geo.dart';

void main() {
  const origin = LatLng(19.3958, -99.0912);
  // ~111 m al norte y ~105 m al este de origin.
  const north = LatLng(19.3968, -99.0912);
  const east = LatLng(19.3958, -99.0902);

  test('distancia de haversine', () {
    expect(distanceMeters(origin, north), closeTo(111.2, 1));
    expect(distanceMeters(origin, east), closeTo(104.9, 1));
    expect(distanceMeters(origin, origin), 0);
  });

  test('rumbo', () {
    expect(bearingDegrees(origin, north), closeTo(0, 0.5));
    expect(bearingDegrees(origin, east), closeTo(90, 0.5));
    expect(bearingDegrees(north, origin), closeTo(180, 0.5));
  });

  test('punto cardinal', () {
    expect(cardinalDirection(0), 'al norte');
    expect(cardinalDirection(350), 'al norte');
    expect(cardinalDirection(44), 'al noreste');
    expect(cardinalDirection(90), 'al este');
    expect(cardinalDirection(225), 'al suroeste');
  });

  test('dirección relativa a hacia dónde camina', () {
    expect(relativeDirection(0, 0), 'al frente');
    expect(relativeDirection(90, 0), 'a tu derecha');
    expect(relativeDirection(270, 0), 'a tu izquierda');
    expect(relativeDirection(45, 0), 'adelante a la derecha');
    expect(relativeDirection(180, 0), 'atrás');
    // Cruza el 0: camina al noroeste (350) y el destino está al noreste (30).
    expect(relativeDirection(30, 350), 'adelante a la derecha');
    expect(relativeDirection(200, 10), 'atrás');
  });

  test('sin rumbo usa puntos cardinales', () {
    expect(directionFrom(origin, north, null), 'al norte');
    expect(directionFrom(origin, north, 90), 'a tu izquierda');
  });

  test('distancias redondeadas para voz', () {
    expect(formatDistance(4), 'menos de 10 metros');
    expect(formatDistance(42), '40 metros');
    expect(formatDistance(48), '50 metros');
    expect(formatDistance(234), '230 metros');
    expect(formatDistance(1000), '1 kilómetro');
    expect(formatDistance(1260), '1.3 kilómetros');
  });

  test('cuadro alrededor de un peligro es un anillo cerrado de ~30 m', () {
    final ring = squareRing(origin, 30);
    expect(ring.length, 5);
    expect(ring.first, ring.last);
    final sw = LatLng(ring[0][1], ring[0][0]);
    final se = LatLng(ring[1][1], ring[1][0]);
    final ne = LatLng(ring[2][1], ring[2][0]);
    expect(distanceMeters(sw, se), closeTo(30, 0.5));
    expect(distanceMeters(se, ne), closeTo(30, 0.5));
  });
```

```
  test('distancias y tiempos cortos para pantalla', () {
    expect(formatDistanceShort(118), '120 m');
    expect(formatDistanceShort(1000), '1 km');
    expect(formatDistanceShort(1260), '1.3 km');
    expect(formatDuration(20), '1 min');
    expect(formatDuration(120), '2 min');
    expect(formatDuration(3900), '1 h 5 min');
    expect(formatDuration(7200), '2 h');
  });
}
```

test/osm_parser_test.dart

Pruebas del parser de OSM.

```
import 'package:flutter_test/flutter_test.dart';
import 'package:latlong2/latlong.dart';
import 'package:ruta_libre/core/osm_parser.dart';
import 'package:ruta_libre/models/place.dart';

void main() {
  final sample = <String, Object?>{
    'elements': [
      {
        'type': 'node',
        'id': 1,
        'lat': 19.396,
        'lon': -99.091,
        'tags': {
          'amenity': 'pharmacy',
          'name': 'Farmacia Ejemplo',
          'wheelchair': 'yes',
          'toilets:wheelchair': 'no',
          'wheelchair:description': 'Rampa por la entrada lateral',
        },
      },
      {
        // Edificio como "way": las coordenadas vienen en "center".
        'type': 'way',
        'id': 2,
        'center': {'lat': 19.397, 'lon': -99.092},
        'tags': {'shop': 'supermarket', 'wheelchair': 'limited'},
      },
      {
        'type': 'node',
        'id': 3,
        'lat': 19.395,
        'lon': -99.090,
        'tags': {'kerb': 'lowered', 'tactile_paving': 'yes'},
      },
      {
        'type': 'node',
        'id': 4,
        'lat': 19.395,
        'lon': -99.089,
        'tags': {'kerb': 'raised'},
      },
      {
        // Sin nombre, sin categoría útil y sin dato: se descarta.
        'type': 'node',
        'id': 5,
        'lat': 19.395,
        'lon': -99.088,
        'tags': {'source': 'survey'},
      },
      {
        // Sin coordenadas: se descarta.
```

```
        'type': 'way',
        'id': 6,
        'tags': {'amenity': 'bank', 'name': 'Banco'},
      },
      {
        'type': 'node',
        'id': 7,
        'lat': 19.394,
        'lon': -99.087,
        'tags': {'amenity': 'bank', 'name': 'Banco Norte', 'wheelchair': 'designated'},
      },
    ],
  };

  test('convierte elementos de Overpass en lugares', () {
    final places = parseOverpass(sample);
    expect(places.map((p) => p.osmId), [1, 2, 3, 4, 7]);

    final pharmacy = places.first;
    expect(pharmacy.name, 'Farmacia Ejemplo');
    expect(pharmacy.category, 'Farmacia');
    expect(pharmacy.access, Accessibility.yes);
    expect(pharmacy.accessibleToilet, isFalse);
    expect(pharmacy.description, 'Rampa por la entrada lateral');
    expect(pharmacy.kind, PlaceKind.poi);
  });

  test('usa el centro de un way y la categoría como nombre si no tiene', () {
    final market = parseOverpass(sample)[1];
    expect(market.location, const LatLng(19.397, -99.092));
    expect(market.name, 'Supermercado');
    expect(market.access, Accessibility.limited);
    expect(market.key, 'way/2');
  });

  test('los cruces se califican por su borde de banqueta', () {
    final places = parseOverpass(sample);
    final lowered = places[2];
    final raised = places[3];
    expect(lowered.kind, PlaceKind.crossing);
    expect(lowered.access, Accessibility.yes);
    expect(lowered.tactilePaving, isTrue);
    expect(raised.access, Accessibility.no);
  });

  test('designated cuenta como accesible', () {
    expect(parseOverpass(sample).last.access, Accessibility.yes);
  });

  test('la consulta incluye el radio y el centro', () {
    final q = buildOverpassQuery(const LatLng(19.3958, -99.0912), 1000);
    expect(q, contains('around:1000,19.3958,-99.0912'));
    expect(q, contains('"wheelchair"'));
    expect(q, contains('out center tags;'));
  });

  test('Place sobrevive ida y vuelta por JSON', () {
    final p = parseOverpass(sample).first;
    final back = Place.fromJson(p.toJson());
    expect(back.key, p.key);
    expect(back.access, p.access);
    expect(back.accessibleToilet, p.accessibleToilet);
  });
}
```

test/report_test.dart

Pruebas de reportes y avisos.

```
import 'package:flutter_test/flutter_test.dart';
import 'package:latlong2/latlong.dart';
import 'package:ruta_libre/core/hazard_alerts.dart';
import 'package:ruta_libre/models/report.dart';

Report _report({
  String id = 'r1',
  ReportType type = ReportType.blockedRamp,
  Severity? severity,
  LatLng location = const LatLng(19.3958, -99.0912),
  required DateTime at,
}) =>
    Report(
      id: id,
      type: type,
      severity: severity ?? type.defaultSeverity,
      location: location,
      createdAt: at,
      lastConfirmedAt: at,
    );

void main() {
  final t0 = DateTime(2026, 10, 29, 8);

  group('ciclo de vida', () {
    test('caduca según su tipo', () {
      final r = _report(at: t0); // rampa bloqueada: 6 horas
      expect(r.isActive(t0.add(const Duration(hours: 5))), isTrue);
      expect(r.isActive(t0.add(const Duration(hours: 6))), isFalse);
    });

    test('"sigue ahí" reinicia la caducidad y borra votos en contra', () {
      final r = _report(at: t0).voteGone();
      expect(r.goneVotes, 1);
      final confirmed = r.confirm(t0.add(const Duration(hours: 5)));
      expect(confirmed.goneVotes, 0);
      expect(confirmed.isActive(t0.add(const Duration(hours: 10))), isTrue);
    });

    test('dos "ya no está" seguidos lo cierran', () {
      final once = _report(at: t0).voteGone();
      expect(once.closed, isFalse);
      final twice = once.voteGone();
      expect(twice.closed, isTrue);
      expect(twice.isActive(t0), isFalse);
    });

    test('ida y vuelta por JSON', () {
      final r = _report(at: t0, type: ReportType.openDrain).voteGone();
      final back = Report.fromJson(r.toJson());
      expect(back.type, ReportType.openDrain);
      expect(back.severity, Severity.high);
      expect(back.goneVotes, 1);
      expect(back.expiresAt, r.expiresAt);
    });
  });

  group('avisos de peligro', () {
    const me = LatLng(19.3958, -99.0912);
    const near = LatLng(19.39595, -99.0912); // ~17 m
    const far = LatLng(19.3968, -99.0912); // ~111 m

    List<Report> due(List<Report> reports, Map<String, DateTime> last,
            {DateTime? now}) =>
        hazardsToAnnounce(
          reports: reports,
          me: me,
          now: now ?? t0,
          lastAlerted: last,
          radiusMeters: 30,
```

```
          cooldown: const Duration(minutes: 10),
        );

    test('avisa solo lo cercano, vigente y no leve', () {
      final reports = [
        _report(id: 'cerca', location: near, at: t0),
        _report(id: 'lejos', location: far, at: t0),
        _report(id: 'leve', location: near, severity: Severity.low, at: t0),
        _report(id: 'vencido', location: near,
            at: t0.subtract(const Duration(days: 1))),
      ];
      expect(due(reports, {}).map((r) => r.id), ['cerca']);
    });

    test('no repite el aviso antes del tiempo de espera', () {
      final reports = [_report(id: 'a', location: near, at: t0)];
      final last = {'a': t0.subtract(const Duration(minutes: 5))};
      expect(due(reports, last), isEmpty);
      expect(due(reports, last, now: t0.add(const Duration(minutes: 6))),
          hasLength(1));
    });

    test('el más cercano va primero', () {
      final reports = [
        _report(id: 'b', location: const LatLng(19.39600, -99.0912), at: t0),
        _report(id: 'a', location: near, at: t0),
      ];
      expect(due(reports, {}).map((r) => r.id), ['a', 'b']);
    });
  });
}
```

test/route_api_test.dart

Pruebas de la API de rutas.

```
import 'package:flutter_test/flutter_test.dart';
import 'package:latlong2/latlong.dart';
import 'package:ruta_libre/core/route_api.dart';
import 'package:ruta_libre/models/route_result.dart';
import 'package:ruta_libre/models/settings.dart';

void main() {
  const start = LatLng(19.3958, -99.0912);
  const end = LatLng(19.3990, -99.0880);
  const midHazard = LatLng(19.3974, -99.0896);
  const hazardAtStart = LatLng(19.39585, -99.0912); // ~6 m del origen

  group('petición', () {
    test('coordenadas en orden [lon, lat] e idioma español', () {
      final body = buildRouteRequest(
          start: start, end: end, profile: MobilityProfile.blind, hazards: []);
      expect(body['coordinates'], [
        [-99.0912, 19.3958],
        [-99.0880, 19.3990],
      ]);
      expect(body['language'], 'es');
      expect(body['instructions'], isTrue);
      expect(body.containsKey('options'), isFalse);
    });

    test('silla de ruedas: evita escalones y limita pendiente', () {
      final body = buildRouteRequest(
          start: start,
          end: end,
          profile: MobilityProfile.wheelchair,
          hazards: []);
      final options = body['options'] as Map<String, Object?>;
```

```
    expect(options['avoid_features'], ['steps']);
    final restrictions = (options['profile_params'] as Map)['restrictions'] as Map;
    expect(restrictions['maximum_incline'], 6);
    expect(restrictions['maximum_sloped_kerb'], 0.06);
  });

  test('los peligros se envían como zonas a evitar, salvo junto al origen', () {
    final body = buildRouteRequest(
        start: start,
        end: end,
        profile: MobilityProfile.mobilityAid,
        hazards: [midHazard, hazardAtStart]);
    final polygons =
        (body['options'] as Map<String, Object?>)['avoid_polygons'] as Map;
    expect(polygons['type'], 'MultiPolygon');
    expect((polygons['coordinates'] as List), hasLength(1));
    expect(avoidedCount(body), 1);
  });
});

group('respuesta', () {
  final sample = <String, Object?>{
    'type': 'FeatureCollection',
    'features': [
      {
        'type': 'Feature',
        'geometry': {
          'type': 'LineString',
          'coordinates': [
            [-99.0912, 19.3958],
            [-99.0900, 19.3970],
            [-99.0880, 19.3990],
          ],
        },
        'properties': {
          'summary': {'distance': 480.4, 'duration': 345.9},
          'segments': [
            {
              'distance': 480.4,
              'duration': 345.9,
              'steps': [
                {'instruction': 'Camina hacia el norte', 'distance': 200.0, 'type': 11},
                {'instruction': 'Gira a la derecha', 'distance': 280.4, 'type': 1},
                {'instruction': 'Llegaste a tu destino', 'distance': 0.0, 'type': 10},
              ],
            },
          ],
        },
      },
    ],
  };

  test('lee geometría [lon, lat], resumen y pasos', () {
    final r = parseRouteResponse(sample, destinationName: 'Farmacia');
    expect(r.points.first, start);
    expect(r.points, hasLength(3));
    expect(r.distanceMeters, 480.4);
    expect(r.steps.map((s) => s.type), [11, 1, 10]);
  });

  test('sin ruta lanza error', () {
    expect(() => parseRouteResponse({'features': []}, destinationName: 'x'),
        throwsFormatException);
  });

  test('resumen e indicaciones habladas', () {
    final r = parseRouteResponse(sample,
        destinationName: 'Farmacia', avoidedHazards: 2);
    expect(spokenSummary(r),
        'Ruta a Farmacia: 480 metros, unos 6 minutos caminando. '
```

```
          'Evita 2 peligros reportados.');
      expect(spokenStep(r.steps[1]), 'Gira a la derecha. Sigue 280 metros.');
      expect(spokenStep(r.steps[2]), 'Llegaste a tu destino');
    });

    test('un minuto en singular', () {
      const r = RouteResult(
        destinationName: 'Banco',
        points: [],
        distanceMeters: 50,
        durationSeconds: 40,
        steps: [],
        avoidedHazards: 0,
      );
      expect(spokenSummary(r), 'Ruta a Banco: 50 metros, un minuto caminando.');
    });
  });
}
```

test/describe_test.dart

Pruebas de textos y comandos de voz.

```
import 'package:flutter_test/flutter_test.dart';
import 'package:latlong2/latlong.dart';
import 'package:ruta_libre/core/describe.dart';
import 'package:ruta_libre/core/voice_commands.dart';
import 'package:ruta_libre/models/place.dart';
import 'package:ruta_libre/models/report.dart';

void main() {
  const me = LatLng(19.3958, -99.0912);
  const north = LatLng(19.3962, -99.0912); // ~45 m al norte

  test('tiempo transcurrido y restante', () {
    final now = DateTime(2026, 10, 29, 12);
    expect(timeAgo(now.subtract(const Duration(seconds: 30)), now), 'hace un momento');
    expect(timeAgo(now.subtract(const Duration(minutes: 15)), now), 'hace 15 minutos');
    expect(timeAgo(now.subtract(const Duration(hours: 1)), now), 'hace una hora');
    expect(timeAgo(now.subtract(const Duration(days: 3)), now), 'hace 3 días');
    expect(timeUntil(now.add(const Duration(hours: 5)), now), 'en 5 horas');
    expect(timeUntil(now.subtract(const Duration(hours: 1)), now), 'ya caducó');
  });

  test('descripción hablada de un lugar', () {
    const p = Place(
      osmType: 'node',
      osmId: 1,
      name: 'Farmacia Ejemplo',
      category: 'Farmacia',
      location: north,
      kind: PlaceKind.poi,
      access: Accessibility.yes,
      accessibleToilet: true,
    );
    expect(
      describePlace(p, me: me),
      'Farmacia Ejemplo, farmacia. Accesible: entrada sin escalón. '
      'Tiene baño accesible. A 45 metros, al norte.',
    );
    // Caminando hacia el este, el norte queda a la izquierda.
    expect(describePlace(p, me: me, heading: 90), endsWith('A 45 metros, a tu izquierda.'));
    // Sin ubicación no se inventa distancia.
    expect(describePlace(p), isNot(contains('metros')));
  });

  test('descripción hablada de un reporte', () {
    final now = DateTime(2026, 10, 29, 12);
```

```
    final r = Report(
      id: '1',
      type: ReportType.openDrain,
      severity: Severity.high,
      location: north,
      createdAt: now.subtract(const Duration(hours: 2)),
      lastConfirmedAt: now.subtract(const Duration(hours: 2)),
      note: 'frente al banco',
    );
    expect(
      describeReport(r, now, me: me, heading: 0),
      'Coladera o registro abierto, gravedad alta. A 45 metros, al frente. '
      'Frente al banco. Confirmado hace 2 horas.',
    );
  });

  group('comandos de voz', () {
    test('reconoce cada comando sin importar acentos', () {
      expect(parseVoiceCommand('¿Qué hay cerca?'), VoiceCommand.nearby);
      expect(parseVoiceCommand('busca una farmacia'), VoiceCommand.nearby);
      expect(parseVoiceCommand('Reportar un obstáculo'), VoiceCommand.report);
      expect(parseVoiceCommand('¿Dónde estoy?'), VoiceCommand.whereAmI);
      expect(parseVoiceCommand('quitar ruta'), VoiceCommand.clearRoute);
      expect(parseVoiceCommand('configuración'), VoiceCommand.settings);
    });

    test('un peligro cerca es un reporte, no una consulta', () {
      expect(parseVoiceCommand('hay un peligro aquí cerca'), VoiceCommand.report);
    });

    test('frases sin comando', () {
      expect(parseVoiceCommand(''), VoiceCommand.unknown);
      expect(parseVoiceCommand('hola'), VoiceCommand.unknown);
    });
  });
}
```

test/widget_test.dart

Pruebas de widgets.

```
// También evita que `flutter create .` genere la prueba del contador de ejemplo.
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:latlong2/latlong.dart';
import 'package:ruta_libre/models/place.dart';
import 'package:ruta_libre/widgets/map_markers.dart';

void main() {
  testWidgets('el marcador expone su descripción al lector de pantalla',
      (tester) async {
    var taps = 0;
    final handle = tester.ensureSemantics();
    const place = Place(
      osmType: 'node',
      osmId: 1,
      name: 'Farmacia Ejemplo',
      category: 'Farmacia',
      location: LatLng(19.3958, -99.0912),
      kind: PlaceKind.poi,
      access: Accessibility.limited,
    );

    await tester.pumpWidget(MaterialApp(
      home: Center(
        child: SizedBox(
          width: 44,
          height: 44,
```

```
          child: PlaceMarker(
            place: place,
            semanticLabel: 'Farmacia Ejemplo. Acceso limitado.',
            onTap: () => taps++,
          ),
        ),
      ),
    ));

    expect(find.bySemanticsLabel('Farmacia Ejemplo. Acceso limitado.'),
        findsOneWidget);
    // Acceso limitado se distingue por ícono, no solo por color.
    expect(find.byIcon(Icons.remove), findsOneWidget);

    await tester.tap(find.byType(PlaceMarker));
    expect(taps, 1);
    handle.dispose();
  });

  testWidgets('la leyenda explica cada estado', (tester) async {
    await tester.pumpWidget(const MaterialApp(
      home: Scaffold(body: AccessLegend()),
    ));
    for (final a in Accessibility.values) {
      expect(find.textContaining(a.label), findsOneWidget);
    }
    expect(find.textContaining('Peligro reportado'), findsOneWidget);
  });
}
```

tool/patch_platforms.dart

Configuración nativa de Android e iOS.

```
// Aplica la configuración nativa que Ruta Libre necesita, después de generar
// las carpetas android/ e ios/ con `flutter create`.
//
// Uso (desde la raíz del proyecto):
//   flutter create --org com.drak028 --project-name ruta_libre --platforms android,ios .
//   dart run tool/patch_platforms.dart
//
// Es seguro correrlo varias veces: cada cambio revisa si ya existe.
import 'dart:io';

const _marker = 'ruta-libre-patch';

void main() {
  for (final line in [
    _patchAndroidManifest(),
    _patchAndroidGradle(),
    _patchInfoPlist(),
  ]) {
    stdout.writeln(line);
  }
}

String _patchAndroidManifest() {
  final file = File('android/app/src/main/AndroidManifest.xml');
  if (!file.existsSync()) {
    return '✗ No existe ${file.path}: corre flutter create primero.';
  }
  var xml = file.readAsStringSync();
  if (xml.contains(_marker)) return '• AndroidManifest.xml ya estaba listo.';

  // Sin permiso CAMERA a propósito: image_picker usa la app de cámara del
  // sistema, y declararlo obligaría a pedirlo en tiempo de ejecución.
  const permissions = '''
    <!-- $_marker: permisos -->
```

```
    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
    <uses-permission android:name="android.permission.RECORD_AUDIO" />
''';

  const queries = '''
        <!-- $_marker: servicios y apps que usa Ruta Libre -->
        <intent>
            <action android:name="android.intent.action.TTS_SERVICE" />
        </intent>
        <intent>
            <action android:name="android.speech.RecognitionService" />
        </intent>
        <intent>
            <action android:name="android.media.action.IMAGE_CAPTURE" />
        </intent>
        <intent>
            <action android:name="android.intent.action.VIEW" />
            <data android:scheme="https" />
        </intent>
''';

  final open = RegExp(r'<manifest[^>]*>').firstMatch(xml);
  if (open == null) return '✗ AndroidManifest.xml sin etiqueta <manifest>.';
  xml = xml.replaceRange(open.end, open.end, '\n$permissions');
  xml = xml.contains('<queries>')
      ? xml.replaceFirst('<queries>', '<queries>\n$queries')
      : xml.replaceFirst(
          '</manifest>', '    <queries>\n$queries    </queries>\n</manifest>');
  file.writeAsStringSync(xml);
  return '✓ AndroidManifest.xml: ubicación, micrófono, internet y consultas.';
}

String _patchAndroidGradle() {
  final kts = File('android/app/build.gradle.kts');
  final groovy = File('android/app/build.gradle');
  final file = kts.existsSync() ? kts : groovy;
  if (!file.existsSync()) return '✗ No existe android/app/build.gradle(.kts).';
  var src = file.readAsStringSync();
  if (src.contains('maxOf(24') || src.contains('Math.max(24')) {
    return '• ${file.path} ya estaba listo.';
  }
  // image_picker pide Android 7.0 (API 24) o superior.
  final before = src;
  src = file == kts
      ? src.replaceFirst('minSdk = flutter.minSdkVersion',
          'minSdk = maxOf(24, flutter.minSdkVersion) // $_marker')
      : src.replaceFirst('minSdkVersion flutter.minSdkVersion',
          'minSdkVersion Math.max(24, flutter.minSdkVersion) // $_marker');
  if (src == before) {
    return '✗ ${file.path}: no encontré minSdk; ponlo en 24 a mano.';
  }
  file.writeAsStringSync(src);
  return '✓ ${file.path}: minSdk 24.';
}

String _patchInfoPlist() {
  final file = File('ios/Runner/Info.plist');
  if (!file.existsSync()) return '✗ No existe ios/Runner/Info.plist.';
  var plist = file.readAsStringSync();
  if (plist.contains(_marker)) return '• Info.plist ya estaba listo.';

  const entries = '''
  <!-- $_marker -->
  <key>NSLocationWhenInUseUsageDescription</key>
  <string>Ruta Libre usa tu ubicación para decirte qué lugares accesibles y peligros hay
cerca.</string>
  <key>NSCameraUsageDescription</key>
  <string>Ruta Libre usa la cámara para que agregues una foto al reportar un peligro.</string>
```

```
  <key>NSPhotoLibraryUsageDescription</key>
  <string>Ruta Libre guarda la foto del peligro que reportas.</string>
  <key>NSMicrophoneUsageDescription</key>
  <string>Ruta Libre usa el micrófono para entender lo que le pides por voz.</string>
  <key>NSSpeechRecognitionUsageDescription</key>
  <string>Ruta Libre convierte tu voz en comandos como qué hay cerca o reportar.</string>
  <key>LSApplicationQueriesSchemes</key>
  <array>
    <string>https</string>
  </array>
''';
  final closing = plist.lastIndexOf('</dict>');
  if (closing == -1) return '✗ Info.plist sin </dict> final.';
  plist = plist.replaceRange(closing, closing, entries);
  file.writeAsStringSync(plist);
  return '✓ Info.plist: textos de permisos.';
}
```

Anexo D. Referencias

- Flutter. *Documentation*. docs.flutter.dev
- Flutter. *Accessibility*. docs.flutter.dev/ui/accessibility-and-internationalization/accessibility
- Dart. *Effective Dart*. dart.dev/effective-dart
- fleaflet. *flutter*map documentation_. docs.fleaflet.dev
- OpenStreetMap Wiki. *Key:wheelchair*, *Key:kerb*, *Key:tactile*paving, Key:toilets:wheelchair_. wiki.openstreetmap.org
- OpenStreetMap Foundation. *Tile Usage Policy*. operations.osmfoundation.org/policies/tiles
- Overpass API. *Overpass API User's Manual*. dev.overpass-api.de/overpass-doc
- HeiGIT. *openrouteservice API documentation*. openrouteservice.org/dev/#/api-docs
- W3C. *Web Content Accessibility Guidelines (WCAG) 2.2*. w3.org/TR/WCAG22
- Braille Institute. *Atkinson Hyperlegible Font*. brailleinstitute.org/freefont
- Google. *Android Accessibility Help: TalkBack*. support.google.com/accessibility/android
- Sinnott, R. W. (1984). *Virtues of the Haversine*. Sky and Telescope, 68(2), 159.